分 类 号_____ 学号_____
学校代码_____ 密级_____

華中科技大學

硕士学位论文

(学术型□ 专业型☑)

基于驱动程序替换的物联网设备固件仿真技术研究

学位申请人：

学 科 专 业：

指 导 教 师：

答 辩 日 期：

A Dissertation Submitted in Partial Fulfillment of the Requirements for

Research on IoT Device Firmware Emulation Technology Based on Driver Replacement

Candidate :

Major :

Supervisor :

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

April, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 保　密 □，在 ____ 年解密后适用本授权书。
不保密 □。

（请在以上方框内打“√”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

随着物联网技术的快速发展，物联网设备数量呈现爆炸式增长，其固件安全问题日益突出。固件仿真技术是进行固件安全测试和漏洞挖掘的重要手段，然而现有的固件仿真方法存在外设建模成本高、硬件依赖强、自动化程度不足等问题。传统的基于硬件建模的方法需要为每种外设手动建立仿真模型，工作量大且难以覆盖多样化的硬件生态；基于驱动层替换的方法虽然避免了底层硬件建模，但驱动函数的定位与替换仍高度依赖人工干预，难以实现规模化应用。

针对上述问题，本文提出一种结合大语言模型的自动化固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型的代码理解与生成能力自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的稳定运行。

本文的主要工作包括三个方面。首先，设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块，该模块通过静态分析技术，从驱动程序源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。其次，构建了基于大语言模型的驱动函数分析与替换模块，该模块利用大语言模型理解驱动函数的语义，通过结构化提示词模板和函数分类策略，自动生成可在仿真环境中运行的替代代码，并通过编译修复循环保障代码的可编译性。最后，引入了动态测试反馈闭环机制，当固件在仿真环境中运行出现异常时，系统能够自动分析异常原因并反馈给驱动替换模块，通过”生成—测试—反馈—修正”的闭环流程持续优化仿真质量。

实验结果表明，本文方法能够有效支持多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）的固件仿真，显著降低了固件仿真的技术门槛和人工成本，为漏洞挖掘奠定了基础。

关键词：物联网安全；固件仿真；驱动程序替换；大语言模型；静态分析

Abstract

With the rapid development of Internet of Things (IoT) technology, the number of IoT devices has grown explosively, and firmware security issues have become increasingly prominent. Firmware emulation is an important approach for firmware security testing and vulnerability discovery. However, existing firmware emulation methods face challenges such as high peripheral modeling costs, strong hardware dependencies, and insufficient automation. Traditional hardware modeling-based methods require manually building simulation models for each peripheral, which is labor-intensive and difficult to cover the diverse hardware ecosystem. Although driver layer replacement-based methods avoid low-level hardware modeling, the identification and replacement of driver functions still heavily rely on manual intervention, making it difficult to achieve large-scale application.

To address these issues, this paper proposes an automated firmware emulation method combined with large language models (LLMs). The core idea is to automatically identify hardware-dependent code in firmware through static analysis, use the code understanding and generation capabilities of LLMs to automatically generate alternative driver implementations, and build a dynamic testing feedback loop to iteratively optimize emulation quality, thereby enabling firmware to run stably in emulation environments without manually building peripheral models.

The main contributions of this paper include three aspects. First, a hardware dependency code automatic identification module for embedded firmware is designed and implemented, which automatically extracts hardware-related function call points, peripheral register access patterns, and driver function boundaries from firmware through MMIO pattern recognition, data flow tracking, and recursive induction techniques, constructing a queryable driver knowledge base. Second, a driver function analysis and replacement module based on LLMs is built, which utilizes LLMs to understand driver function semantics, automatically

generates alternative code that can run in emulation environments through structured prompt templates and function classification strategies, and ensures code compilability through a compilation repair loop. Finally, a dynamic testing feedback loop mechanism is introduced. When firmware runs abnormally in the emulation environment, the system can automatically analyze the cause of the abnormality and feed it back to the driver replacement module, continuously optimizing emulation quality through a ”generate-test-feedback-revise” closed-loop process.

Experimental results show that the proposed method can effectively support firmware emulation for multiple embedded platforms (STM32, NXP) and operating systems (FreeRTOS, RT-Thread, bare metal), significantly reducing the technical barrier and manual cost of firmware emulation, laying the foundation for vulnerability discovery.

Keywords: IoT Security; Firmware Emulation; Driver Replacement; Large Language Model; Static Analysis

目　　录

摘　要 I
Abstract II
1　绪论
1.1　研究背景与意义 ..... (1)
1.2　国内外研究现状 ..... (2)
1.3　研究内容与主要贡献 ..... (9)
1.4　论文组织结构 ..... (10)
2　背景知识与相关技术
2.1　物联网设备硬件基础 ..... (13)
2.2　物联网操作系统与驱动 ..... (14)
2.3　大模型代码分析智能体框架 ..... (15)
2.4　静态分析技术 ..... (17)
2.5　固件仿真与模糊测试 ..... (17)
2.6　本章小结 ..... (19)
3　系统设计
3.1　驱动函数接口静态分析与识别模块设计 ..... (21)
3.2　驱动函数分析、替换与编译模块设计 ..... (29)
3.3　动态测试反馈闭环模块设计 ..... (39)
3.4　本章小结 ..... (45)
4　系统实现
4.1　驱动函数接口静态分析与识别模块实现 ..... (47)
4.2　驱动函数分析、替换与编译模块实现 ..... (52)
4.3　动态测试反馈闭环模块实现 ..... (62)
4.4　实现挑战与解决方案 ..... (68)
4.5　本章小结 ..... (72)

5 实验设计与结果分析

5.1 实验设计 ..... (73)

5.2 功能测试 ..... (75)

5.3 性能测试 ..... (85)

5.4 本章小结 ..... (89)

6 总结与展望

6.1 工作总结 ..... (91)

6.2 论文创新点 ..... (92)

6.3 论文贡献 ..... (93)

6.4 局限性与不足 ..... (93)

6.5 未来工作展望 ..... (95)

参考文献 ..... (97)

1 绪论

1.1 研究背景与意义

近年来，随着物联网（Internet of Things, IoT）技术迅速发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据 IoT Analytics 发布的《2026 年企业物联网现状报告》，2025 年企业物联网市场同比增长 13%，达到 3240 亿美元。Statista 的数据显示，截至 2024 年，全球物联网设备连接数已超过 170 亿台，预计到 2030 年将增长至 290 亿台以上[1]。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 OpenHarmony[2]、亚马逊的 FreeRTOS[3]、Linux 基金会的 Zephyr[4] 等。然而，物联网和嵌入式操作系统面临着严峻的安全挑战[5]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。针对物联网恶意软件的安全分析也得到了广泛关注，如 IoTPOT 工具用于分析和检测 IoT 设备中的恶意软件[6]。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为”Mirai” 的恶意软件[7] 利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。根据国家互联网应急中心的 CNVD 统计显示,2020 年公开收录的通用型联网智能设备漏洞 3047 个, 新增漏洞数量呈显著增长趋势, 同比增长 28%[8]。这些安全事件的不断发生表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

固件仿真（Firmware Re-Hosting）技术是一种克服固件分析硬件依赖问题的有效方法，它通过在商用计算机和服务器的软件模拟器上运行嵌入式系统固件，实现对嵌入式设备的软件级模拟，使得包含嵌入式系统的固件无需实际硬件即可进行安全测试[9]。然而，当前固件仿真技术的**自动化程度仍然严重不足**，这主要体现在以下几个方面：首先，现有的外设仿真方法大多需要人工编写仿真模型或基于研究者总结的有限的规则自动推断。针对原始硬件特征进行外设建模需要人工分析 MCU 手

册并编写仿真代码，这项工作费时费力，且通用性较差；利用程序分析自动推断部分外设行为的方案虽然可以自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。其次，驱动层替换方案（如 HALucinator[10]）利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过 HAL 函数替换的方式进行固件仿真，但驱动函数的定位、语义理解和替换代码生成仍然需要大量人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码，这个过程不仅耗时耗力，而且容易出错。这种**自动化程度不足导致的人工成本高昂**，严重制约了固件仿真技术在大规模安全测试中的应用，成为当前物联网设备安全领域亟待解决的关键问题。

针对上述问题，本研究致力于提升固件仿真技术的自动化程度，降低人工成本，实现高效的物联网设备固件安全测试。具体而言，本研究提出一种基于驱动程序替换的固件仿真方法，通过静态分析技术自动识别固件中的硬件依赖代码，利用大语言模型的代码理解能力深入分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。该方法的核心创新在于将自动化技术贯穿于驱动函数定位、语义分析和代码生成的全过程，显著减少了对分析人员人工干预的依赖。通过提升固件仿真的自动化水平，本研究不仅能够降低固件安全测试的成本，还能够提高测试效率和覆盖率，为物联网设备的大规模安全测试提供技术支撑，具有重要的理论意义和应用价值。

1.2 国内外研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性，对固件进行有效的安全测试是一项具有挑战性的任务[8]。现有的二进制漏洞检测方法（如 VulHawk[11] 通过跨架构二进制代码搜索检测已知漏洞）虽然取得了一定成效，但仍难以全面覆盖固件中深层次的安全问题。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了

一些代表性的工具和系统，如 Firmadyne[12]、Fuzzware[13] 等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化。下面将从固件仿真技术、嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[14] 之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如计时器、UART、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。

固件仿真技术的发展经历了从依赖真实硬件到全自动化模拟的演进过程。早期研究采用硬件在环（Hardware in the Loop, HIL）的方法，如 Zaddich 等人提出的 Avatar[15] 和 Muench 等人提出的 Avatar2[16]，通过混合执行的方式将固件分析过程中对外设的访问转发到物理硬件中。然而，这种方案仍然依赖实际设备，且仿真环境和物理设备之间的切换影响了运行速度。为摆脱对真实硬件的依赖，研究者提出了多种自动化程度较高的仿真方案，主要包括以下三类。

1.2.1.1 基于原始硬件的外设建模方案

解决外设依赖的一种方法是在底层内存映射寄存器接口上实现硬件，或通过观察真实硬件的行为来构建外设模型。然而这是一项费力的任务，扩展性不好。当前主流的仿真器 QEMU 也仅能仿真 10 余种设备，远不能满足固件仿真工作的需求。

Gustafson 等人提出的 Pretender[17] 工具通过观察固件与物理设备之间真实发生的交互记录，使用机器学习、模式识别等方法为每个外设构建模型，再结合 QEMU 或 angr[18] 等仿真引擎实现固件仿真。这种方法需要真实设备的支持，但在观测到足够的交互模式后，可以自动生成较为准确的外设模型。Zhou 等人提出的 sEmu[19] 工具通过 NLP（自然语言处理）技术，从 MCU 手册中提取外设的寄存器信息，构建

外设模型，再结合 QEMU 或 angr 等仿真引擎实现固件仿真。这些方案的优点是能够对真实设备的行为进行精确模拟，提高测试效率和覆盖率，但缺点是依赖真实设备观测，通用性差。

1.2.1.2 基于程序分析的外设建模方案

基于程序分析的外设建模方案通过对固件代码进行静态或动态分析，提取固件与外设之间的交互逻辑，构建虚拟外设模型以替代真实硬件。

Feng 等人提出的 P2IM[20] 工具通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，构建与真实设备相似的虚拟模型。Cao 等人提出的 Laelaps[21] 在因访问未实现的外设而卡住时，切换到符号执行以找到正确输入，引导 QEMU 找到与实际执行最相近的路径。Zhou 等人提出的 uEmu[22] 工具利用符号执行分析固件镜像，将未知的外设寄存器表示为符号，构建推断规则并存储在知识库中。Scharnowski 等人提出的 Fuzzware[13] 使用精确的内存映射输入输出（Memory-Mapped I/O，MMIO）建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，自动生成模型以提高模糊测试输入数据的有效性。Shoshitaishvili 等人提出的 Firmalice[23] 通过符号执行自动检测二进制固件中的身份验证绕过漏洞。Johnson 等人提出的 Jetset[24] 利用符号执行推断固件期望的外设行为，自动生成外设模型以实现固件 rehosting。Mera 等人提出的 DICE[25] 针对 DMA 输入通道的自动仿真问题，为固件动态分析提供硬件无关的 DMA 输入通道模拟支持。这些基于程序分析的外设建模方案实现了一定程度的自动化，但这些方案难以完全覆盖复杂外设的所有行为，可能存在误报或漏报。

1.2.1.3 基于系统或驱动层替换的外设仿真方案

基于系统或驱动层替换的仿真技术，又称高级仿真（High Level Emulation, HLE），利用固件中的公共抽象来消除为外设提供低级支持的需要。Chen 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[12]，通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失

性内存、动态文件等仿真技术难题。Li 等人创建了 Para-rehosting[26] 工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利用 x86 上许多现有的复杂软件测试工具对代码进行测试。Clements 等人提出了 HALucinator[10] 工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真。Seidel 等人提出的 SAFIREFUZZ[27] 在 HALucinator 的基础上进一步优化了 HLE 方案的模糊测试性能，通过近原生 rehosting 将 ARM Cortex-M 固件作为 Linux 用户态进程运行，结合动态二进制改写技术降低了仿真开销，相比 HALucinator 实现了平均 690 倍的吞吐量提升。

然而，上述基于系统或驱动层替换的仿真技术自动化程度较差，难以适应大规模安全测试的需要。具体而言，这类技术面临以下两个核心难点：一是难以自动化定位需要进行替换的驱动程序。在嵌入式程序中，识别哪些部分存在硬件依赖并需要替换，往往依赖分析人员的手动干预。手动分析嵌入式程序的复杂性使得这一过程既耗时又容易出错。二是难以自动化生成替换程序。除了识别硬件相关功能，现有的方案也缺乏自动化生成替代程序的能力。分析人员必须手动分析函数的功能，理解其与硬件和中间件的交互，并编写相应的替代代码，这个过程不仅繁琐，而且容易出错，进一步增加了人工成本。

1.2.1.4 固件仿真技术对比

表 1.1 对比了当前主流固件仿真技术的优缺点。

在这三类方案中，基于系统或驱动层替换的固件仿真方法从固件系统层和驱动函数入手，较好地解决了基于程序分析和原始硬件建模的方法对复杂外设难以完全模拟或建模工作量大等痛点。然而，现有驱动层替换方案在自动化方面仍存在明显不足：一方面，驱动函数的定位依赖分析人员的手动干预，识别固件中哪些部分存在硬件依赖并需要替换是一个耗时且容易出错的过程；另一方面，替换程序的生成也缺乏自动化支持，分析人员需要手动分析函数功能、理解其与硬件的交互，并编写相应的替代代码。为解决上述问题，需要设计一种能够自动化定位需要替换的驱

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于原始硬件的外设建模	QEMU, Pretender	可对外设行为进行精确模拟，提高测试效率和覆盖率	需要为每种外设专门建模，工作量大，通用性差，依赖真实设备观测
基于程序分析的外设建模	P2IM, uEmu, Laelaps, Fuzzware	可根据程序自身信息自动推断外设行为，减少人工干预，提高通用性	难以完全覆盖复杂外设的所有行为，可能存在误报或漏报
基于系统或驱动层替换	HALucinator, SAFIREFUZZ, Firmadyne, Para-rehosting	在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	驱动函数的定位与替换依赖人工干预，自动化程度不足，难以适应大规模安全测试

动函数、并利用大模型或代码分析技术自动生成替代程序的方案，以提升固件仿真过程的自动化程度，实现高效的物联网设备程序仿真与动态测试。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷。目前，嵌入式系统模糊测试技术主要以下两个方向：面向真实设备的黑盒模糊测试和基于固件仿真的灰盒模糊测试。

1.2.2.1 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Chen 等人开发的 IoTFuzzer[28] 工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。Feng 等人开发的 Snipuzz[29] 通过分析设备的响应消息，推断消息片段（snippets），基于消息片段进行变异以提升变异质量。Liu 等人提出的 Labrador[30] 利

用设备响应信息引导模糊测试方向，实现了面向黑盒 IoT 设备的高效定向模糊测试。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

1.2.2.2 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[13]、hoedur[31]、FirmAFL[32]、SFuzz[33]、SHiFT[34]等。其中，SFuzz 通过基于切片的模糊测试方法解决 RTOS 单体二进制中任务与服务耦合导致的测试困难，SHiFT 提出半托管式模糊测试方法，通过将嵌入式应用的部分执行委托给宿主机来提升测试效率。为了进一步提升模糊测试的效率和覆盖率，研究者提出了多种混合模糊测试方法，如 Driller[35] 通过选择性符号执行增强模糊测试，QSYM[36] 提供了实用的混合执行引擎用于混合模糊测试，Angora[37] 通过原则性搜索实现高效模糊测试。这种方法的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。此外，Shi 等人提出的 IPEA[38] 采用解耦架构将资源密集的分析任务从微控制器卸载到工作站，仅保留轻量级探针在固件中采集执行信息，实现了非侵入式的真机固件测试。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

1.2.3 大模型程序分析与代码生成技术研究现状

大语言模型（Large Language Model, LLM）在代码理解、生成和程序分析领域取得了突破性进展。以 OpenAI 的 GPT 系列、Anthropic 的 Claude 3.5[39]、Google 的 Gemini 3[40]、DeepSeek-R1[41]、智谱 GLM-5[42] 和 MiniMax M2.7[43] 为代表的大语言模型在代码能力方面展现出卓越的性能。这些模型不仅能够理解和生成高质量的代码，还能够进行复杂的推理、调试和优化任务。这些进展为驱动函数分析和驱动代码生成的自动化提供了新的方向。

近年来，研究者开始探索将 LLM 应用于程序分析和代码生成领域，涌现出多种

增强策略以提升模型在复杂编程任务中的表现。早期 LLM 通过零样本提示能够实现基本的代码生成任务，但在处理复杂编程任务时该方案的准确性和可靠性仍存在不足。为克服这些局限，研究者发展了以下增强策略：

指令微调策略：通过对预训练 LLM 在特定领域数据集上进行精细调整，显著提升模型对特定编程任务的理解和执行能力[44]。DolphCoder 模型通过多样化的指令和自评估机制，结合代码评估目标有效提升了代码生成能力[45]。CodeLSI 框架则通过低秩优化与领域特定指令微调相结合，提高了代码生成的领域特异性和成本效益[46]。

测试驱动迭策略：利用测试用例作为反馈信号指导模型逐步修正生成的代码，直至通过所有测试。这种方法通过隐式执行监督信号，结合搜索、重排序或迭代修正等机制动态优化模型输出[47]。RLCF（Reinforcement Learning with Compiler Feedback）方法通过编译器反馈对 LLM 进行额外训练，解决生成代码中可能存在的语法违规问题[48]。CodeDPO 框架则将偏好学习整合到代码生成过程中，优化模型使其优先选择正确且高效的解决方案[49]。

多步推理策略：思维链（Chain-of-Thought, CoT）和程序思维（Program-of-Thought, PoT）等方法通过将复杂问题分解为多个可管理的子步骤，提升模型解决复杂问题的能力[50]。MoTCoder 引入模块化的思维指令框架来应对复杂编程任务[51]。这种程序思维方法则将中间推理步骤编译为可执行代码片段，实现了”推理即执行”，大幅提升了数值计算和 API 调用类任务的准确性。

工具集成策略：通过集成编译器、调试器、静态分析工具等外部工具，扩展模型的实际应用能力。这种范式将 LLM 定位为”AI 协调者”，通过标准化工具调用协议调度各种工具，形成感知-决策-执行的闭环[52]。MCCoder 系统专注于生成运动控制代码，并结合严格的验证机制以确保代码安全性[53]。RGD 代理调试器利用多 LLM 通过细化和生成指导来自动修复代码中的错误[54]。Agents4PLC 框架通过基于 LLM 的智能体自动化了可编程逻辑控制器（PLC）代码生成与验证，构建了端到端的”分析 → 生成 → 符号验证 → 反馈精炼”流水线[55]。其中形式化规约被用作生成约束，而验证失败轨迹则被结构化为自然语言反馈以驱动下一轮生成，提高了工业控制系统的操作效率和安全性。

1.2.4 现有研究的局限性分析

综合上述研究现状，现有固件仿真技术主要存在以下局限性：

（1）外设建模方案局限性强，推断规则有限难以覆盖复杂外设状态机逻辑

现有基于程序分析的外设建模方案虽然能够自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。网络控制器、USB 设备等复杂外设具有多通道、DMA 传输、中断嵌套等特性，现有自动推断方法往往只能处理简单的 MMIO 寄存器读写，无法准确模拟这些复杂交互行为。硬件在环方案和基于原始硬件的建模方案虽然可以处理复杂外设，但需要为每种外设专门配置物理设备连接或人工分析数据手册并编写仿真代码，工作量巨大且通用性差。

（2）驱动仿真人工依赖大，无法适应大规模需要

现有驱动层替换方案（如 HALucinator）的自动化程度不足，驱动函数的定位与替换高度依赖人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码。这个过程不仅耗时耗力，而且容易出错，导致仿真质量高度依赖分析人员的经验。这种对人工的依赖严重制约了固件仿真技术在大规模安全测试中的应用，难以适应物联网设备固件数量庞大、种类繁多的特点。

（3）缺乏自动化反馈机制，难以迭代优化仿真质量

当前大多数仿真工具采用”一次性配置”的模式：分析人员手动配置仿真环境后运行测试，如果发现问题则需要人工介入调试和修复。这种模式缺乏自动化的反馈闭环机制——当仿真过程中出现异常（如程序卡死、断言失败、外设访问超时）时，系统无法自动分析原因并调整仿真策略。这导致仿真质量的提升高度依赖分析人员的经验，难以适应大规模、多样化的固件测试需求。

1.3 研究内容与主要贡献

为解决上述问题，本文提出一种基于驱动程序替换的物联网设备固件仿真方法。该方法通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。

本文主要的创新点和贡献有三个：

（1）本研究设计并实现了一个面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别、数据流追踪等技术，从固件二进制代码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。与现有依赖人工标注或有限规则推断的方法相比，该模块能够自动处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，显著降低了驱动函数定位的人工成本。

（2）本研究设计并实现了一个基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，本模块能够处理不同类型的驱动函数（如初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。与现有基于预设返回值的简单钩子函数（Hook）方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

（3）本研究设计并实现了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成-测试-反馈-修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

1.4 论文组织结构

本文共分为六章，从绪论、背景知识、系统设计、系统实现、实验验证到总结展望，逐步深入地阐述基于驱动程序替换的物联网设备固件仿真方法。图 1.1 展示了本文的整体组织结构。

第一章：绪论。阐述物联网设备安全问题的研究背景，分析固件仿真技术在物联网安全测试中的作用。梳理国内外在固件仿真、嵌入式系统模糊测试、大模型程

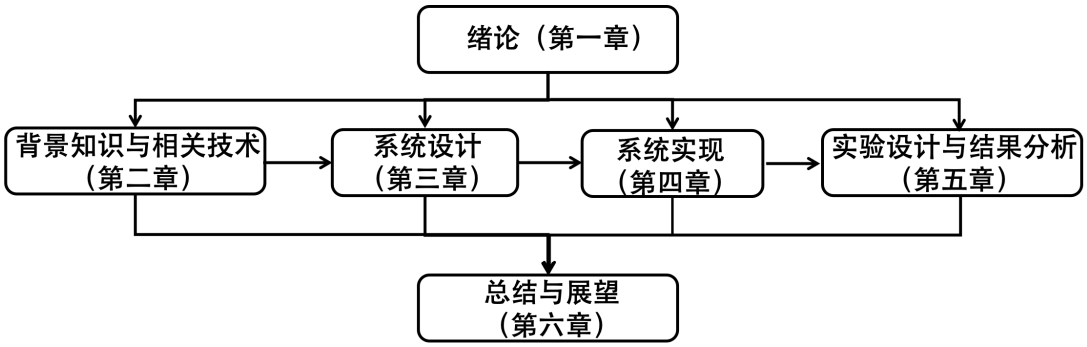

图 1.1 文章架构图

序分析与代码生成等领域的研究现状，分析现有技术存在的局限性，明确本文的研究内容和主要贡献。

第二章：背景知识与相关技术。介绍物联网设备固件仿真研究涉及的基础知识和技术工具。首先阐述 ARM Cortex-M 架构特性和内存映射 I/O 机制，然后介绍实时操作系统与硬件抽象层的概念，接着讲解大语言模型代码分析的智能体框架设计模式，最后介绍 CodeQL 静态分析平台和 QEMU 仿真器与 AFL 模糊测试工具的原理与应用。

第三章：系统设计。阐述基于驱动程序替换的固件仿真方法的整体架构设计。介绍系统的三大核心模块：驱动函数接口静态分析与识别模块负责定位硬件相关代码并构建可查询知识库，驱动函数分析替换与编译模块负责理解驱动函数语义并生成替代实现，动态测试反馈闭环模块负责在仿真环境中运行固件并采集反馈信息。详细说明各模块的职责划分、接口设计和协作机制。

第四章：系统实现。详细阐述各关键模块的具体实现技术。包括 CodeQL 查询规则的设计与实现，用于识别 MMIO 访问模式和追踪数据流；LangGraph 智能体工作流的构建，用于驱动函数分类和替换代码生成；多种替换策略的实现，用于处理不同类型的驱动函数；以及基于 Unicorn 的仿真平台搭建和反馈闭环的实现细节。

第五章：实验设计与结果分析。通过实验验证本文方法的有效性和实用性。首先介绍实验环境的软硬件配置和测试用例的构建，之后从功能测试与性能测试两个方面，多个维度评估系统能力，包括静态分析模块的识别准确率、固件在仿真环境

中的运行正确性、与现有方法的对比分析以及在实际固件上的应用验证。

第六章：总结与展望。总结全文的研究工作，概括本文在硬件依赖代码自动识别、基于大语言模型的驱动函数替换、动态测试反馈闭环等方面的创新点和贡献。客观分析当前方法在源码依赖、LLM 成本、平台覆盖和复杂外设模拟等方面存在的局限性，并对支持无源码固件、扩展平台支持、提升模拟精度和安全性验证等未来研究方向进行展望。

2 背景知识与相关技术

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试技术，为后续章节的论述奠定基础。

2.1 物联网设备硬件基础

物联网设备通常采用嵌入式微控制器（Microcontroller Unit, MCU）作为核心处理器。常见的 MCU 架构包括 ARM Cortex-M 系列、RISC-V 等。这些处理器具有低功耗、低成本、实时性强等特点，广泛应用于物联网领域。本节重点介绍 ARM Cortex-M 架构的特性以及内存映射 I/O 机制。

2.1.1 ARM Cortex-M 架构特性

ARM Cortex-M 是 ARM 架构处理器核心中低端系列的统称，专为低成本、高能效的微控制器设计，广泛应用于物联网设备。该系列采用 Thumb-2 指令集，在提供 32 位性能的同时保持 16 位代码密度。Cortex-M 架构提供了多项增强实时性和可靠性的特性，包括嵌套向量中断控制器（NVIC）用于中断管理、系统定时器用于提供可编程的周期性中断并支持操作系统的任务调度、以及主栈与进程栈的栈模式切换机制以实现多任务并发和内存保护。

2.1.2 内存映射 I/O

内存映射 I/O（Memory-Mapped I/O, MMIO）是嵌入式系统中外设与处理器通信的主要方式。MMIO 将外设寄存器映射到处理器的统一内存地址空间中，处理器通过普通的内存读写指令（如 LDR/STR）访问特定内存地址，即可完成对外设的控制和数据传输。这种设计简化了编程模型，使得外设访问与内存访问具有相同的语法形式。

ARM Cortex-M 架构采用统一的 32 位地址空间，按功能划分为代码区（Flash）、SRAM 区、外设区和系统区。外设区是 MMIO 的核心区域，所有片上外设的寄存器

都被映射到此区域，不同厂商的外设被分配到不同的地址段。系统区位于地址空间高端，包含 NVIC、SysTick、MPU 等系统级组件的控制寄存器。理解 MMIO 地址空间布局对于固件仿真至关重要，仿真器需要针对外设区和系统区实现专门的访问处理逻辑。

外设寄存器按照功能可分为控制寄存器、状态寄存器和数据寄存器三类。控制寄存器用于配置外设的工作模式和参数，如 USART 的 CR1 可设置波特率、数据位长度等。状态寄存器反映外设的当前工作状态，如 USART 的 SR 包含发送缓冲区空（TXE）、接收缓冲区非空（RXNE）等标志位，其值由硬件自动更新。数据寄存器用于数据传输，如 USART 的 DR 用于发送和接收数据，访问数据寄存器通常会触发外设执行相应操作。

2.2 物联网操作系统与驱动

2.2.1 实时操作系统概述

实时操作系统（Real-Time Operating System, RTOS）是物联网设备中广泛采用的管理任务调度和资源分配的软件系统。RTOS 能够及时响应外部事件或数据，并在规定的时间内完成任务，其工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。

物联网领域常见的 RTOS 包括 FreeRTOS[3]、Zephyr[4]、RT-Thread[56] 等。FreeRTOS 是 Amazon 开源的实时操作系统，专为微控制器和小型微处理器设计，内核精简，最小可运行在不到 10KB 的 RAM 上，是物联网领域使用最广泛的 RTOS 之一。Zephyr 是 Linux 基金会管理的开源实时操作系统，支持多种处理器架构，提供更完整的操作系统功能，包括文件系统、网络协议栈、设备驱动模型等，采用模块化设计以平衡功能丰富性和资源占用。RT-Thread 是中国开源社区主导开发的实时操作系统，采用微内核架构，并提供了丰富的组件生态。

2.2.2 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是嵌入式软件架构中的关键组成部分，它位于硬件驱动层和上层应用之间，为应用程序提供统一的硬件访问接口。HAL 层的核心目标是屏蔽底层硬件的差异性，使得上层应用代码可以在不同的硬件平台之间移植，而无需修改业务逻辑。

HAL 层的接口设计遵循统一的命名和功能规范。以 STM32 HAL 库为例，每个外设类型提供初始化函数（如 `HAL_UART_Init`）、数据传输函数（如 `HAL_UART_Transmit`）、状态查询函数（如 `HAL_UART_GetState`）和中断管理函数（如 `HAL_UART_Receive_IT`）。由于 HAL 函数封装了底层硬件操作细节，仿真器可通过替换 HAL 层函数来实现硬件行为模拟，无需深入理解寄存器语义，相比寄存器级模拟更加高效且易于实现。

2.3 大模型代码分析智能体框架

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出强大的能力，为自动化程序分析提供了新的技术途径。本节重点阐述代码分析工作流范式和基于 LLM 的智能体开发框架，为后续章节的驱动函数分析和替换工作奠定技术背景。

2.3.1 代码分析工作流设计模式

基于 LLM 智能体框架构建代码分析系统时，研究者提出了多种工作流设计模式。这些模式为处理代码分析任务中常见的逻辑推理、工具调用、多步骤协作等挑战提供了系统化思路。

ReAct 模式：ReAct（Reasoning + Acting）将推理与行动交替进行[57]。在代码分析中，LLM 可先推理分析目标（如”需要理解该函数的调用关系”），再调用静态分析工具获取信息，观察结果后继续下一步推理。该模式使 LLM 能够动态收集信息并逐步完成复杂分析。

规划-执行模式：该模式将任务分为规划与执行两阶段。规划阶段生成步骤化的执行计划，执行阶段按序完成。其优势在于结构清晰、便于调试和过程监控。

多智能体协作模式：该模式使用多个专门化的智能体协同完成任务[58]。例如在驱动函数替换中，可设计分析智能体、生成智能体、验证智能体与协调智能体，通过消息传递或共享状态完成复杂代码分析与生成任务。本研究在驱动函数替换模块中，综合运用了 ReAct 模式与多智能体协作模式，以应对硬件相关代码替换的复杂性与正确性要求。

2.3.2 LLM 智能体开发框架

为了更好地利用大语言模型构建复杂的应用系统，研究人员开发了多种智能体（Agent）框架。这些框架提供了结构化的方式来组织 LLM 的调用、管理状态、集成外部工具，使开发者能够构建可靠的、可扩展的 LLM 应用。

2.3.2.1 LangChain 框架

LangChain 是目前最流行的 LLM 应用开发框架之一，提供了模型、提示模板、链、代理、工具和记忆等核心组件。模型组件封装了与不同 LLM 提供商的交互接口，便于切换和比较。提示模板允许开发者定义带变量的提示词，在运行时动态填充，提高复用性。链将多个组件按顺序组合，构建复杂的处理流程。代理是更高级的抽象，允许 LLM 根据用户输入自主选择合适的工具（如搜索引擎、代码执行器）并决定后续行动。工具是代理可以调用的外部功能，框架提供了大量内置工具，开发者也可自定义。记忆组件维持应用在多次调用之间的状态，例如在多轮对话中存储历史对话。

2.3.2.2 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图的概念，状态在各个节点之间传递和更新。节点是处理单元，边定义转换关系并支持条件分支和循环，适用于迭代优化等场景。LangGraph 还支持检查点机制，可将工作流状态持久化存储，支持长时间运行任务的暂停和恢复。

2.4 静态分析技术

静态分析技术是软件工程和安全研究的重要基础，旨在通过在不执行程序的情况下，通过分析源代码或二进制代码来推断程序的属性。静态分析的优势在于能够覆盖程序的所有可能执行路径，不受测试用例的限制。常见的静态分析技术包括数据流分析、控制流分析、指针分析、抽象解释等。

2.4.1 CodeQL 代码分析平台

CodeQL 是 GitHub 开发的语义级代码分析平台，支持多种编程语言（包括 C/C++、Java、Python、JavaScript 等）的安全漏洞检测和代码质量分析。CodeQL 的核心思想是将源代码转换为可查询的数据库，然后通过声明式查询语言（QL）进行灵活的分析。与传统基于规则的静态分析工具不同，CodeQL 提供了表达力丰富的查询语言，研究人员可以通过编写查询来查找特定的代码模式、追踪数据流、分析控制依赖等复杂问题。

CodeQL 的工作原理分为两个阶段。第一阶段是数据库构建，CodeQL 对源代码进行解析，生成抽象语法树（AST）、控制流图（CFG）、数据流图（DFG）等多层次的结构化表示，并将这些信息存储在数据库中。第二阶段是查询执行，研究人员使用 QL 语言编写查询，在数据库上执行，获取分析结果。CodeQL 的查询语言基于 Datalog，采用谓词逻辑的表达方式，能够自然地表达”所有存在某种模式的代码片段”或”从源点到汇点的数据流路径”等复杂查询。

CodeQL 在程序分析领域应用广泛，包括安全漏洞检测（如 SQL 注入、缓冲区溢出）、代码质量分析（如死代码检测、复杂度分析）、以及依赖关系分析（如函数调用图、数据流追踪）。本研究利用 CodeQL 的数据流追踪能力，建立 MMIO 访问与函数参数之间的关联关系，为驱动函数识别和语义分析提供支撑。

2.5 固件仿真与模糊测试

固件仿真是物联网安全研究的重要技术手段。由于物联网设备的硬件多样性和封闭性，直接在真实设备上进行安全测试面临诸多困难。固件仿真技术通过软件模拟硬件环境，使研究人员能够在通用计算平台上运行和分析嵌入式固件，从而降低

测试成本并提高效率。模糊测试作为一种高效的漏洞挖掘技术，与固件仿真相结合，能够在仿真环境中自动化地发现固件中的安全漏洞。

2.5.1 QEMU 仿真器

QEMU[14] 是目前应用最广泛的开源机器仿真器，支持多种处理器架构和外设仿真，其设计目标是提供高性能、高可移植性的系统仿真能力，使其成为固件仿真研究的重要基础平台。QEMU 支持两种主要的仿真模式：用户模式仿真（User Mode Emulation）允许在一种架构上运行为另一种架构编译的单个程序，适用于分析用户空间程序；系统模式仿真（System Mode Emulation）提供完整的硬件系统仿真，包括处理器、内存、中断控制器、外设等组件，可以运行完整的操作系统或裸机固件，这是固件仿真的主要工作模式。

QEMU 的核心是动态二进制翻译引擎（Tiny Code Generator, TCG），它将目标架构的指令块翻译为中间表示（IR），然后再将 IR 翻译为宿主机指令，这种两级翻译机制使 QEMU 能够支持多种目标架构和宿主架构的组合。为了提高性能，TCG 采用即时编译（JIT）技术，将翻译结果缓存在内存中供后续执行重复使用。

Unicorn[59] 是基于 QEMU 的 TCG 引擎开发的轻量级 CPU 模拟器，专门针对安全研究和逆向工程场景优化。与 QEMU 提供完整系统级仿真的设计不同，Unicorn 去除了 QEMU 的系统级组件和外设仿真，只保留指令集模拟核心，因此具有更好的灵活性和可控性。Unicorn 提供了丰富的钩子（Hook）机制，允许用户在指令执行、内存访问、中断触发等事件上注册自定义回调函数，从而在不修改模拟器源码的情况下实现对执行过程的细粒度监控和干预。此外，Unicorn 以库的形式提供编程接口（支持 Python、C 等多种语言），便于嵌入到其他工具链中。本研究选用 Unicorn 作为固件动态测试的仿真引擎，利用其 ARM Thumb 模式模拟 ARM Cortex-M 系列微控制器，并通过钩子机制实现驱动函数拦截、运行日志采集和仿真数据注入等功能。

2.5.2 模糊测试技术

模糊测试（Fuzzing）是一种自动化软件测试技术，通过向目标程序输入大量随机、半随机或结构化的测试用例，来发现程序中的安全漏洞和缺陷[60]。模糊测试的

基本工作流程包括：生成或变异测试用例、将测试用例输入目标程序、监控程序执行状态（如崩溃、断言失败、内存错误等）、记录和分析异常。

American Fuzzy Lop（AFL）[61] 是最具影响力的覆盖率引导模糊测试工具之一。AFL 的核心思想是通过代码插桩技术记录程序执行过程中的覆盖率信息，利用这些信息指导测试用例的变异和选择，优先保留能够触发新执行路径的测试用例，从而在有限的测试时间内探索更多的程序状态，提高漏洞发现的概率。AFL 的工作流程包含两个核心环节：覆盖率记录和变异策略。在覆盖率记录环节，AFL 通过编译时插桩或运行时插桩（利用 QEMU 的动态翻译功能）在目标程序中插入覆盖率记录代码，记录程序执行过程中经过的基本块和边。在变异策略环节，AFL 维护一个测试用例队列，不断从队列中选择测试用例，对其进行变异，执行变异后的测试用例，并根据覆盖率变化决定是否将新用例加入队列。AFL 的变异策略包括位翻转、字节删除、字节插入、块拼接、算术运算等多种操作，能够产生多样化的测试输入。通过这种迭代式的测试和筛选机制，AFL 能够逐步探索程序的各种执行路径，自动发现潜在的漏洞和缺陷。

在嵌入式固件模糊测试领域，AFL 常与仿真器（如 QEMU）结合使用。研究者在 QEMU 的动态翻译机制基础上实现插桩，使 AFL 能够对在仿真环境中运行的固件进行模糊测试。这种方法不需要修改固件源码，能够直接对二进制固件进行测试，显著提高了测试的自动化程度和可扩展性。

2.6 本章小结

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动、大模型代码分析智能体框架、静态分析技术、固件仿真与模糊测试技术，为后续章节的论述奠定基础。

3 系统设计

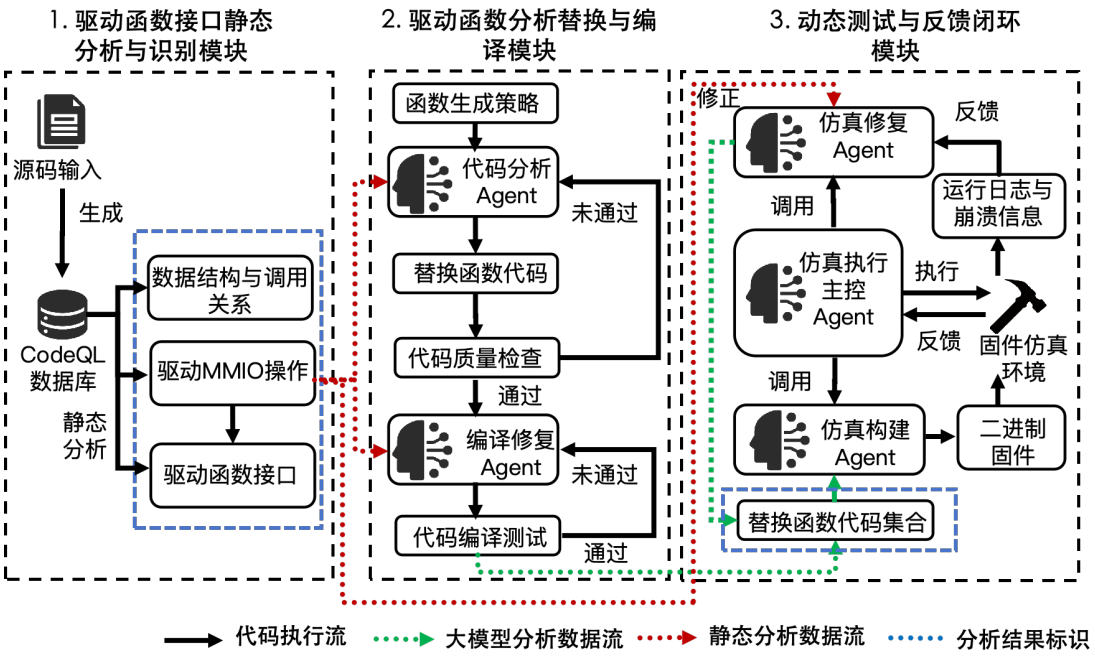

图 3.1 系统整体架构图

本研究设计并实现了基于驱动程序替换的固件仿真系统，该系统分为三个模块：（1）驱动函数接口静态分析与识别模块；（2）驱动函数分析、替换与编译模块；（3）动态测试反馈闭环模块。整体方案架构图如图 3.1所示。

驱动函数接口静态分析与识别模块首先接受固件源码，然后通过 CodeQL 静态分析引擎建立抽象语法树构建的数据库，通过预定义的查询规则从固件源码中提取驱动相关信息，包括驱动函数接口、驱动 MMIO 操作、依赖数据结构与函数间调用关系等信息，并以统一的数据模型对外提供查询接口。

驱动函数分析、替换与编译模块首先接受驱动函数接口静态分析与识别模块输出的驱动函数信息，然后针对各驱动函数进行类型分类并通过合适的函数替换策略生成可替代实现，之后对生成的替换函数代码做质量检查，如果质量检查通过，则将替代实现集成回工程并驱动构建，进一步通过交叉编译验证替换代码是否存在编译

错误。如果存在编译错误，则触发编译修复智能体进行修复，修复完成后再次通过交叉编译验证。该模块采用 LangGraph 智能体框架实现复杂的推理和决策流程，确保生成的替换代码既满足语义保持性要求，又能够正常通过交叉编译工具链的验证。

动态测试反馈闭环模块首先接受驱动函数分析、替换与编译模块输出的替换代码，然后在仿真环境中运行固件，采集运行日志、崩溃信息等，并将错误信息结构化，触发闭环修复智能体对嵌入式程序进行重新分析与替换代码修正，修正完成后再次通过闭环构建智能体交叉编译生成可执行的二进制固件程序，再次在仿真环境中运行固件，如此循环，直至替换代码通过仿真测试。通过多智能体协作，该模块构建”生成-测试-反馈-修正” 的闭环机制，使固件构建仿真与自修复的全过程实现自动化。

为精确定义系统的输入输出关系，以下给出端到端映射的形式化描述。设固件源码工程为 P，固件中的函数集合为 F，MMIO 结构体实例集合为 M，驱动函数类别集合为 C，替换策略集合为 S。系统的端到端功能可形式化为以下复合映射：

$$\mathit{Pipeline} = \mathit{Emulate} \circ \mathit{Verify} \circ \mathit{Replace} \circ \mathit{Classify} \circ \mathit{StaticAnalysis} : P \to (O, E)$$

其中，$\mathit{StaticAnalysis} : P \to (D, R, G)$ 从源码中识别驱动函数集合 $D \subseteq F$、MMIO 访问点集合 R 和函数调用图 $G = (F, E_c)$；$\mathit{Classify} : D \to C$ 将每个驱动函数映射到其语义类别；$\mathit{Replace} : D \times C \to D'$ 对每个驱动函数 $f \in D$ 生成替换实现 $f' \in D'$；$\mathit{Verify} : D' \to \{\mathit{pass}, \mathit{fail}\}$ 对替换实现进行多层验证；$\mathit{Emulate} : P' \to (O, E)$ 在仿真环境中运行替换后的工程 P'，产生运行输出 O 和运行时错误集合 E。系统通过迭代执行” 替换 $\to$ 验证 $\to$ 仿真 $\to$ 反馈” 闭环，直至 $E = \emptyset$ 或达到最大迭代次数。

3.1 驱动函数接口静态分析与识别模块设计

本模块是整个方案的基础支撑层，负责自动、精准地定位固件中与硬件交互的代码，并将其抽象为标准化的” 可查询知识库”。模块的核心职责是建立从源码到硬件依赖信息的映射，识别固件中与硬件交互的函数，确定函数访问硬件的方式（如直接寄存器访问、HAL 库调用或间接句柄操作），并分析函数之间的调用依赖关系。

在静态分析引擎的选型上，需要综合考虑分析能力与工程兼容性。源码级静态分析主要有两种技术路线：基于 LLVM/Clang 的分析和基于 CodeQL 的分析。

基于 LLVM/Clang 的分析方案（如 P2IM[20] 采用基于 LLVM IR 的启发式规则识别外设寄存器访问）在实际应用中面临工程兼容性问题。嵌入式固件的构建工具链通常与 ARM GCC 深度绑定：Zephyr RTOS 通过 west 构建系统默认调用 `arm-none-eabi-gcc` 进行交叉编译，STM32CubeIDE、MCUXpresso 等厂商 IDE 同样基于 ARM GCC 工具链；嵌入式工程中广泛使用的 CMSIS 头文件、芯片厂商提供的设备描述头文件以及 `__attribute__`、`__IO`、`__ASM` 等 ARM GCC 扩展语法，Clang 的兼容性尚不完整。若采用基于 Clang 的源码分析工具，需要将整个构建工具链从 ARM GCC 切换到 Clang，这一过程涉及重新配置构建系统、适配厂商头文件中的编译器扩展语法以及处理链接器脚本差异等问题，迁移成本较高。此外，部分嵌入式工程的构建脚本使用 ARM GCC 特有的编译选项（如 `-mcpu=cortex-m4`、`-mfpu=fpv4-sp-d16` 等），Clang 对这些选项的语义解释存在差异，可能导致分析结果与实际编译行为不一致。

CodeQL 采用与编译器无关的数据库构建方式：通过在构建过程中拦截编译器调用来捕获源码信息，而非替代原始编译器执行编译。因此，CodeQL 可以直接在 ARM GCC 工具链的构建流程上工作，无需修改固件工程的编译配置。CodeQL 将源码构建为包含 AST、控制流图和数据流图的综合数据库，支持声明式查询语言和全局污点追踪，能够在不运行固件的情况下跨文件追踪 MMIO 基址常量的传播路径，为后续的智能替换提供基础数据。与运行时方法相比（如 HALucinator[10] 通过 QEMU 的内存追踪识别外设访问），CodeQL 无需先实现完整的外设模型即可工作。

模块采用静态分析技术构建包含语法树、控制流图和数据流图的综合数据库，通过设计的查询规则识别 MMIO 结构体实例和字段访问，进而识别涉及硬件访问的驱动函数，为替换模块提供精确的语句定位信息。模块输出的分析结果包含驱动函数列表、MMIO 访问操作、函数调用关系图和数据结构信息，这些结构化的分析结果被存储在统一的数据模型中，为下游模块提供可查询的知识库。

3.1.1 MMIO 访问模式识别方法

MMIO（Memory-Mapped I/O，内存映射 I/O）是一种将硬件外设的寄存器映射到处理器内存地址空间的机制。通过 MMIO，软件可以通过访问内存地址的方式与硬件外设进行交互，而不需要特殊的 I/O 指令。嵌入式厂商通常使用 C 语言结构体来定义 MMIO 寄存器映射，结构体的每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移保持一致。

MMIO 结构体的典型代码示例如代码 3.1所示。

```
// STM32 UART寄存器映射示例
typedef struct {
    __IO uint32_t CR1;     // 控制寄存器1
    __IO uint32_t CR2;     // 控制寄存器2
    __IO uint32_t CR3;     // 控制寄存器3
    __IO uint32_t BRR;     // 波特率寄存器
    __IO uint32_t GTPR;    // 保护时间与预分频寄存器
    __IO uint32_t RTOR;    // 接收超时寄存器
    __IO uint32_t RQR;     // 请求寄存器
    __IO uint32_t ISR;     // 中断与状态寄存器
    __IO uint32_t ICR;     // 中断标志清除寄存器
    __IO uint32_t RDR;     // 接收数据寄存器
    __IO uint32_t TDR;     // 发送数据寄存器
} USART_TypeDef;

// 通过强制类型转换创建MMIO实例
#define USART1_BASE    (0x40011000UL)
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;
```

代码清单 3.1 MMIO 结构体示例

定位 MMIO 结构体实例是识别硬件相关代码的关键步骤。本研究总结出三种典型的 MMIO 实例定位模式，分别如代码 3.2、代码 3.3和代码 3.4所示。

模式一：直接地址强转。这类代码将固定外设基地址强制转换为外设寄存器结构体指针，常见于寄存器定义的头文件中。在代码中呈现为类似 `#define USART1 ((USART_TypeDef *)USART1_BASE)` 的宏定义，后续代码直接对该指针进行字段访问如 `USART1->DR`、`USART1->CR1`。

```
// 驱动源码示例：直接地址强转模式
#define USART1_BASE 0x40011000UL
#define USART1 ((USART_TypeDef *)USART1_BASE)
#define USART2 ((USART_TypeDef *)USART2_BASE)
#define USART3 ((USART_TypeDef *)USART3_BASE)

void uart_init_direct(void) {
    USART1->CR1 = 0x2000;  // 直接对强转实例进行寄存器访问
    USART1->BRR = 0x0341;
}
```

代码清单 3.2 直接地址强转模式示例

模式二：间接初始化。这类代码通过句柄（handle）、配置表（descriptor/config table）等间接结构，将外设基地址赋值给某个”寄存器块指针成员”，后续通过该成员完成寄存器访问。在 STM32 HAL 生态中，这类成员常命名为 `Instance`。在代码中呈现为初始化时的赋值语句如 `huart->Instance = COM_USART[COM]`，后续寄存器访问呈现为多级解引用如 `huart->Instance->DR`。在基于配置表的驱动中，也可能呈现为 `handle.Instance = spi_config[i].Instance` 这种从配置表复制的形式。

```
// 驱动源码示例：间接初始化模式（HAL句柄）
UART_HandleTypeDef huart;

void BSP_COM_Init(COM_TypeDef COM) {
    huart.Instance = COM_USART[COM];  // 将外设基地址赋值给Instance成员
    HAL_UART_Init(&huart);
    // 后续访问呈现为多级解引用
    huart.Instance->DR = data;
}

// 驱动源码示例：间接初始化模式（配置表）
typedef struct {
    USART_TypeDef *Instance;
    uint32_t BaudRate;
} SPI_Config;

SPI_Config spi_config[] = {
    {SPI1, 115200},
    {SPI2, 9600}
};

void rt_hw_spi_bus_init(void) {
    for (int i = 0; i < sizeof(spi_config)/sizeof(spi_config[0]); i++) {
        spi_bus_obj[i].handle.Instance = spi_config[i].Instance;
        // 通过配置表中的Instance赋值
    }
}
```

代码清单 3.3 间接初始化模式示例（HAL 句柄和配置表）

模式三：常量指针参数。这类代码通过函数参数接收 MMIO 结构体指针（寄存器块基址），常见于底层驱动或通用外设操作函数。在代码中呈现为函数定义的形参类型是某个寄存器结构体指针，函数体内部通过该参数访问寄存器字段，如 `void mmio_uart_init(USART_TypeDef *uart) { uart->CR1 = 0x2000; }`。调用者可以将模式一或模式二中的 MMIO 实例作为实参传入，如直接调用 `mmio_uart_init(USART1)` 或传入句柄成员 `mmio_uart_init(huart.Instance)`。

```
// 驱动源码示例：常量指针参数模式
void mmio_uart_init_pointer(USART_TypeDef *uart) {
    // 通过指针访问MMIO寄存器
    uart->CR1 = 0x2000;
    uart->BRR = 0x0341;
}
```

```

// 调用示例
void main(void) {
    // 直接传模式一的强转实例
    mmio_uart_init_pointer(USART1);

    // 传模式二的句柄成员
    mmio_uart_init_pointer(huart.Instance);

    return 0;
}
```

代码清单 3.4 常量指针参数模式示例

三种定位模式的对比如表 3.1所示。

设 T 为 MMIO 寄存器结构体类型集合，A 为外设基址常量集合，H 为句柄结构体类型集合，F 为函数集合，则三种定位模式可形式化定义为：

模式一（直接地址强转）：存在常量 $a \in A$ 和类型 $t \in T$，使得 a 被显式转换为 t^* 类型，即存在表达式 $\texttt{cast}(a) \to t^*$。后续通过该指针的字段访问构成 MMIO 操作：

$$DirectCast = \{(a,t) \mid a \in A, t \in T, \exists\, \texttt{cast}(a) : a \to t^*\}$$

模式二（间接初始化）：存在句柄 h（类型为 $\tau \in H$），其成员 $h.inst$ 被赋值为 MMIO 实例的指针，后续通过 $h.inst \to field$ 进行字段访问：

$$IndirectInit = \{(h,t) \mid h.inst = ptr, ptr \to t^*, t \in T\}$$

模式三（常量指针参数）：存在函数 $f \in F$，其参数列表中包含类型为 t^*（$t \in T$）的形式参数，调用点传入 MMIO 实例指针作为实参：

$$PtrParam = \{(f,t,i) \mid f.param_i : t^*, t \in T\}$$

三种模式共同构成 MMIO 实例的完整识别规则集合 $MMIOInst = DirectCast \cup IndirectInit \cup PtrParam$。与已有方法不同，P2IM[20] 通过运行时 fuzzing 探测外设行为，HALucinator[10] 通过 QEMU 内存追踪识别外设访问，本研究从源码层面对 MMIO 访问模式进行了系统性的分类和形式化描述，为驱动函数的精确识别提供了可复用的查询规则。

表 3.1 MMIO 结构体实例定位模式对比

模式	描述	代码呈现
直接地址强转	将固定地址强制转换为外设结构体指针	整数常量 → 结构体指针类型转换
间接初始化	通过配置表将外设基地址赋值给句柄	结构体成员赋值为外设宏
常量指针参数	函数以常量指针形式接收外设基址	参数类型为 MMIO 结构体指针

3.1.1.1 查询规则设计原则

为在 CodeQL 数据库中可靠地识别上述三种 MMIO 定位模式，本研究设计了相应的查询规则体系。查询规则的设计遵循以下原则：

（1）结构体特征识别。首先识别 MMIO 寄存器结构体，判定依据包括：结构体名称包含 `TypeDef`、`_TypeDef`、`_Registers` 等典型后缀；结构体字段类型以 `__IO`（volatile 修饰）为主；字段偏移量与芯片参考手册中的寄存器地址偏移一致。查询规则通过匹配这些特征从数据库中提取候选 MMIO 结构体，过滤掉普通数据结构体。

（2）基址常量识别。MMIO 实例的基址通常以宏或枚举常量的形式定义，其数值范围为典型外设地址区间（如 STM32F4 的 APB1/APB2 总线地址 0x40000000–0x50000000）。查询规则通过识别数值落在已知外设地址区间内的常量表达式，过滤掉地址空间无关的普通整数常量，降低误报率。

（3）类型转换链追踪。对于模式一（直接地址强转），查询规则追踪从基址常量到结构体指针类型的显式转换表达式（C 语言中的 `(Type *)cast` 形式），并进一步追踪通过该指针的解引用和字段访问。对于模式二（间接初始化），查询规则通过数据流追踪识别从 MMIO 实例到句柄成员的赋值传播路径，再追踪句柄成员的字段访问。对于模式三（常量指针参数），查询规则匹配函数参数类型为 MMIO 结构体指针的函数定义，并追踪调用点传入的实际参数来源。

（4）污点传播与边界控制。为防止追踪范围过度扩散，查询规则设置传播边界条件：仅追踪通过指针解引用（`->` 操作符）和结构体字段访问的路径，不追踪算术

运算后的指针偏移；传播深度限制在函数调用图的三层以内，避免跨模块追踪引入无关噪声。对于跨文件和跨编译单元的 MMIO 访问，通过 CodeQL 的全局数据流分析能力实现跨文件追踪。

上述查询规则的设计确保了 MMIO 识别的准确性和可扩展性：结构体特征识别和基址常量识别提供了高精度的起点，类型转换链追踪覆盖了三种定位模式的代码呈现形式，污点传播边界控制则在保证召回率的同时限制了误报范围。当需要支持新的 MCU 平台时，仅需更新基址常量的地址区间和结构体特征的后缀规则即可适配，无需修改核心查询逻辑。

3.1.2 驱动函数定位准则

静态分析模块的核心任务是识别固件中的驱动函数。本研究采用简单直接的定位准则：**凡是包含 MMIO 访问的函数，均定义为驱动函数**。这一定位准则的合理性在于：MMIO 访问是固件与硬件交互的最底层方式，包含寄存器读写的函数必然与硬件强相关，这类函数在仿真环境中无法直接执行，需要进行替换处理；而不包含 MMIO 访问的函数通常是纯软件逻辑，如数据处理、协议解析、状态管理等，在仿真环境中可以直接运行，无需替换。

设 F 为固件中的函数集合，M 为 MMIO 结构体实例集合，$A(f, m)$ 为谓词" 函数 f 访问 MMIO 实例 m 的字段"，则驱动函数集合 D 定义为：

$$D = \{f \in F \mid \exists m \in M : A(f, m)\}$$

驱动函数识别的流程如下：首先通过三种定位模式识别代码中的所有 MMIO 结构体实例；然后对每个 MMIO 结构体实例，追踪其字段访问表达式，记录访问的字段名、访问类型（读/写）、所在语句和所在函数；接着对所有包含 MMIO 字段访问的函数进行汇总，形成驱动函数列表；最后分析驱动函数之间的调用关系，构建函数调用图。

3.1.3 数据流追踪方法

系统以"MMIO 字段访问" 为源点，以" 函数参数、局部变量、返回值" 为汇点，构建数据流配置并追踪路径。追踪目的包括：

1. 确定哪些函数参数最终影响了 MMIO 访问；2. 识别需要保留的业务逻辑与需要替换的硬件操作；3. 为替换模块提供精确的语句定位信息。

3.1.3.1 追踪配置设计

数据流追踪基于 CodeQL 提供的污点分析框架实现。系统配置了两类追踪路径：

正向追踪：从 MMIO 基址常量出发，沿赋值语句、函数参数传递、结构体字段赋值等路径追踪 MMIO 实例的传播，识别所有通过该实例进行字段访问的代码位置。正向追踪的源点为识别出的 MMIO 基址常量表达式，汇点为结构体字段访问表达式（即 `expr->field` 形式的访问）。

反向追踪：从 MMIO 字段访问表达式出发，反向追踪数据来源，确定访问的值是否来源于函数参数或外部输入。反向追踪用于区分”配置型访问”（值由软件写入寄存器，如初始化阶段设置波特率）和”状态型访问”（值从寄存器读出，如轮询等待标志位），前者通常可安全替换，后者需要通过仿真注入接口模拟硬件状态。

3.1.3.2 追踪复杂度分析

设固件中函数数量为 n，平均函数调用深度为 d，平均每个函数的 MMIO 访问点数量为 k。数据流追踪的时间复杂度取决于 CodeQL 污点分析引擎的实现，其理论复杂度为 $O(n \cdot d \cdot k)$：n 为候选源点函数的数量，d 为追踪的最大传播深度（受边界条件约束），k 为每个函数中需要检查的汇点数量。在本研究的测试固件中，n 的取值范围为 10–130，d 通常不超过 5，k 的平均值约为 3，因此单次追踪的计算开销在可接受范围内。空间复杂度为 $O(n \cdot d)$，主要用于存储追踪路径的中间结果。

需要指出的是，CodeQL 的污点分析基于静态近似，可能存在以下局限：一是无法处理通过函数指针或虚函数表进行的间接调用，这类间接调用在嵌入式固件中相对少见；二是对运行时才能确定的数组索引和条件分支采用保守估计，可能引入少量无关追踪路径。在实际应用中，上述局限对驱动函数识别的召回率影响有限，因为嵌入式驱动代码的函数调用关系通常较为明确，间接调用的使用场景较少。

3.1.4 统一数据模型设计

系统定义了统一的数据模型来表示分析结果，包括 `FunctionInfo`（函数基本信息）、`MMIOAccess`（MMIO 访问点）等结构，为上层模块提供标准化查询接口。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化。

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

3.2 驱动函数分析、替换与编译模块设计

驱动函数分析、替换与编译模块负责语义理解和代码生成，采用大语言模型驱动的智能体架构，实现从函数分类、代码生成、质量验证到编译集成的完整自动化流程。该模块采用”语义分析 → 代码生成 → 质量验证 → 编译集成 → 错误修复”的迭代优化模式，通过多轮迭代逐步提升替换质量，直至生成能够在仿真环境中正常运行的替换代码。

在智能体框架的选型上，本模块选择 LangGraph 作为编排框架。驱动函数替换涉及多轮推理决策（函数分类、策略选择、代码生成、质量验证），且各步骤之间存在复杂的条件依赖和迭代关系。LangGraph 基于图式状态机模型，天然支持条件分支和循环结构，能够精确表达”分类 → 生成 → 验证 → 修复”的迭代工作流；同时支持工具调用（Tool Calling）机制，可以将静态分析查询、编译构建等外部能力封装为图节点，实现智能体与工程环境的深度集成。相比纯对话式框架，LangGraph 对执行流程的控制粒度更细，能够精确控制每步的输入输出和状态传递，这对于需要严格保证函数签名一致性和编译正确性的驱动替换场景尤为必要。

3.2.1 分析替换与编译智能体工作流设计

工作流以**单个驱动函数**为粒度调度：每轮从静态分析模块或任务队列中取定一条驱动函数记录（含签名、源码片段、MMIO 访问上下文等），在图式状态机中顺序执行，直至该函数被判定为无需替换并结束，或替换实现通过校验与编译并被采纳。为便于叙述与实现解耦，整条流水线在逻辑上划分为**驱动函数分析与替换代码生成**、**替换代码修正与编译集成**两个子模块；前者由**分析智能体**主导完成上下文拉取、分类与候选替换生成，后者由**编译修复智能体**在工程侧完成定义校验、构建与失败修复。为避免混淆，下文将模块二中的编译修复智能体称为" 编译修复智能体"，将模块三中负责运行时错误后重新构建的智能体称为" 闭环构建智能体"，两者职责不同：前者处理替换代码首次集成时的编译错误（如语法错误、类型不匹配），后者处理动态反馈闭环中经仿真验证发现运行时错误后的重新构建。

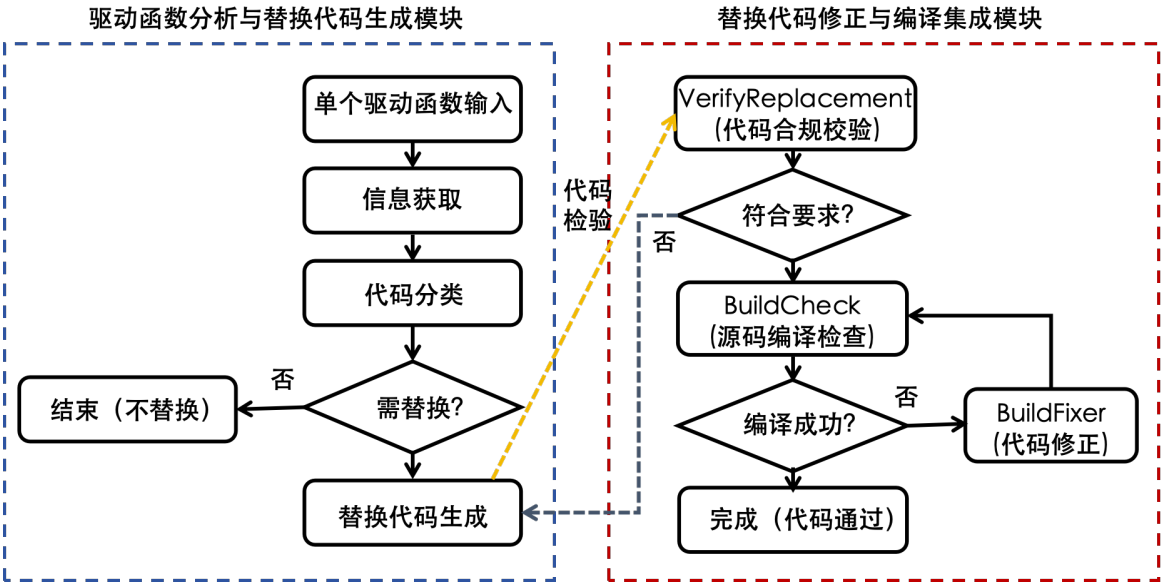

图 3.2 驱动函数分析替换编译模块示意图

3.2.1.1 驱动函数分析与替换代码生成模块设计

该子模块由**分析智能体**驱动，在 LangGraph 状态机中通过" 工具调用—推理—写回" 的循环完成单函数处理。执行顺序固定为：**信息获取** → **代码分类** →（仅在分

类结果认定该函数需要替换时）**替换代码生成**。

信息获取阶段通过工具调用获取该驱动函数在工程中的完整上下文，包括完整实现或相关片段、函数签名与参数类型、调用关系与 MMIO 仿真侧提示等（如 GetFunctionInfo、GetFunctionCallStack、GetMMIOFunctionEmulateInfo）。**代码分类**阶段综合源码与静态分析结果，由大语言模型判定函数语义类别及是否属于必须替换的硬件相关行为；分类所依据的类别体系及提示构造方式见第 3.2.2 节。**替换代码生成**仅在" 需要替换" 分支上触发：模型在既定类别上选择替换策略并产出与签名一致的候选实现，具体策略（如 Skip、Strip、Redirect、Event 等）的选择规则与生成约束同样在第 3.2.2 节展开，本节只给出子模块内的先后次序与数据交接关系。

3.2.1.2 替换代码修正与编译集成模块设计

该子模块在工程目录中对上一子模块输出的**候选替换代码**做**二阶段检验与修复**，目标是在真实构建环境下得到可链接、可编译的替换定义。

第一阶段（代码合规校验）调用 VerifyReplacement，依据预置规则与接口契约检查候选替换是否满足替换定义（例如签名一致、禁止引入不兼容的存储类或明显违背仿真假设的写法等），过滤与定义明显不符的实现；未通过时可将反馈交回分析智能体侧重新生成或局部修订（图 3.2中以虚线表示返回替换生成环节）。

第二阶段（语法编译检查）在通过定义校验后，将替换写入目标源文件（如 ReplaceFuncs，并保留回滚所需的原定义快照），再调用工程构建脚本执行完整编译链接（如 BuildProject）。若**编译失败**，则进入编译修复智能体路径：由编译修复智能体解析编译器诊断信息，结合 GetFunctionInfo、GetMMIOFunctionInfo 等工具定位首个关键错误，通过 UpdateFunctionReplacement 或 FixFunctionBuildErrors 等动作修订替换代码，并再次触发构建，形成" 编译 → 错误解析 → 修复 → 重新编译" 的闭环，直至编译成功或达到步数上限，从而保证最终被采纳的替换函数能够通过工程编译。

编译错误的典型成因包括函数未声明、类型不匹配、缺少头文件、链接错误，以及替换代码引用了目标芯片 HAL 中不存在的结构体成员或与头文件声明不一致等。例如，STM32F401 的 TIM_HandleTypeDef 无 ErrorCode 成员、RCC_PLLInitTypeDef

无 PLLR 成员时，若替换体仍访问这些成员会导致编译失败；将非静态函数误写为 static 亦会破坏链接一致性。编译修复智能体针对不同诊断采取补声明、改类型与转换、补 `#include`、删改非法成员访问、核对依赖等策略。提示词上约束智能体在构建失败时优先根据 stderr 锁定**第一个错误**所涉函数再查上下文，避免对多函数无序轮询而浪费工具调用次数，从而兼顾修复质量与效率。

3.2.2 驱动函数替换策略设计

为实现语义保持的驱动函数替换，本研究在替换前先对驱动函数进行语义分类，并基于分类结果选择对应替换策略，避免硬件依赖被误删或替换不足导致仿真异常。

3.2.2.1 函数分类策略设计

驱动函数替换的核心挑战在于不同类型的驱动函数与硬件交互的方式和语义要求存在显著差异。采用无差别的统一替换策略无法准确处理各类函数的特殊性，容易导致仿真异常或语义丢失。因此，系统在替换前先对驱动函数进行语义分类，根据函数类别选择最合适的替换策略，从而确保替换后代码的正确性和仿真环境的稳定性。

系统基于大语言模型实现驱动函数的自动分类。LLM 通过读取函数源码、识别 MMIO 访问模式、判断函数主要用途等步骤，将函数分类到预定义的类别。分类策略基于对嵌入式驱动函数硬件交互模式的分析，将函数划分为以下类别，如表 3.2所示。

设驱动函数集合 D 上的分类映射定义为：

$$classify : D \to C = \{INIT, RECV, LOOP, IRQ, RETURNOK, SKIP, CORE, NODRIVER\}$$

对于需要替换的函数子集 $D_r = \{f \in D \mid classify(f) \in C \setminus \{CORE, NODRIVER\}\}$，系统进一步为其分配替换策略。替换策略映射定义为：

$$strategy : D_r \to S = \{Skip, Redirect, Event, Steering, ReturnOK\}$$

该分类体系旨在解决已有方法中替换策略与函数语义不匹配的问题。HALucinator[10] 采用统一的 HAL profile 拦截策略，P2IM[20] 通过随机 fuzzing 生成外设响应，

均未对驱动函数的语义类别进行系统性区分。当替换策略与函数语义不匹配时，容易出现仿真异常——例如 LOOP 类函数未移除轮询逻辑引发死循环、RECV 类函数未注入模拟数据导致上层协议栈读取到无效值。本研究通过”先分类后替换”的范式，使每类函数获得与其语义特征相匹配的替换策略，从设计层面降低了替换导致的语义错误风险。

3.2.2.2 分类判定规则

为提高分类的准确性和一致性，系统为每类函数定义了明确的判定规则，LLM 在分类时需综合以下特征进行判定：

（1）INIT 类判定规则。满足以下条件之一的函数归类为 INIT：函数名包含 `Init`、`Config`、`Setup`、`MspInit` 等关键词；函数体内包含对多个 MMIO 寄存器的顺序写操作且无读-改-写循环；函数调用位置位于系统启动阶段或外设初始化流程中（通过调用图判断）。INIT 类函数的特征是对硬件进行一次性配置，执行后不依赖返回的硬件状态。

（2）RECV 类判定规则。满足以下条件之一的函数归类为 RECV：函数名包含 `Receive`、`Read`、`Get`、`Input` 等关键词；函数体内包含从 MMIO 数据寄存器（如 `RDR`、`DR`、`RXDATA`）的读操作；函数参数中包含目标缓冲区指针和长度参数。RECV 类函数的核心语义是从硬件获取数据并传递给上层软件，替换时需要通过仿真注入接口提供模拟数据。

（3）LOOP 类判定规则。满足以下条件之一的函数归类为 LOOP：函数名包含 `Wait`、`Poll`、`Loop` 等关键词；函数体内包含以 MMIO 状态寄存器（如 `ISR`、`SR`）的某一位为条件的循环结构（`while(flag == RESET)` 形式）；循环体内部无实质性的数据处理逻辑，仅为等待硬件状态变化。LOOP 类函数在仿真环境中会造成死循环，必须通过 Skip 策略移除等待逻辑。

（4）IRQ 类判定规则。满足以下条件之一的函数归类为 IRQ：函数名包含 `IRQHandler`、`Callback`、`Handler` 等关键词；函数被注册为中断服务例程（通过中断向量表或 `HAL_RegisterCallback` 等方式）；函数体内包含对 MMIO 中断状态寄存

器的读操作和清标志操作。IRQ 类函数的触发依赖硬件中断信号，在仿真环境中需要通过 Event 策略将其改造为可由仿真器主动触发的事件处理函数。

（5）RETURNOK 类判定规则。满足以下条件之一的函数归类为 RETURNOK：函数仅包含单个或少量简单的 MMIO 读操作且不使用读出值；函数的返回类型为状态枚举（如 `HAL_StatusTypeDef`）且所有执行路径均返回成功状态；函数的语义等价于" 检查硬件状态并返回成功"。RETURNOK 类函数在仿真环境中无需执行具体操作，直接返回成功状态值即可。

（6）SKIP 类判定规则。满足以下条件之一的函数归类为 SKIP：函数返回类型为 `void` 且函数体内的 MMIO 操作对固件状态机无影响；函数名包含 `DeInit`、`MspDeInit`、`Flush` 等资源释放类关键词；函数的功能仅为清理硬件状态或重置外设寄存器。SKIP 类函数在仿真环境中可以直接跳过执行。

（7）CORE 类判定规则。满足以下条件之一的函数归类为 CORE：函数属于操作系统内核代码（如 `rt_thread_init`、`SysTick_Handler`）；函数的实现由仿真器直接提供，不经过替换流程。CORE 类函数由系统预配置，不参与 LLM 分类。

（8）NODRIVER 类判定规则。函数虽位于驱动模块中，但满足以下全部条件：函数体内不包含直接的 MMIO 寄存器访问；函数的功能为纯软件逻辑（如参数校验、错误码转换、辅助计算）；函数的执行不依赖任何硬件状态。NODRIVER 类函数在仿真环境中可直接运行，无需替换。

上述判定规则以结构化的形式嵌入 LLM 的分类提示模板中，指导模型从函数名、MMIO 访问模式、控制流结构和调用上下文四个维度综合判断，从而提高分类的一致性。当函数特征同时满足多个类别的判定条件时，LLM 根据函数的主要语义选择最合适的类别，优先级规则为：CORE > IRQ > RECV > LOOP > INIT > RETURNOK > SKIP > NODRIVER。

这八类函数覆盖了嵌入式固件中与硬件交互的各种典型模式。其中，INIT、RECV、LOOP、IRQ、RETURNOK 和 SKIP 六类函数均直接或间接地涉及 MMIO 寄存器访问，需要通过替换消除硬件依赖；CORE 类函数涉及操作系统核心功能（如系统定时器、线程调度），由仿真器直接提供底层支持，无需替换；NODRIVER 类函

表 3.2 驱动函数分类体系

缩写	描述	典型函数
INIT	外设初始化，影响固件状态机	HAL_UART_Init, BOARD_BootClockRUN
RECV	从硬件/DMA 读取数据	HAL_UART_Receive, ENET_GetRxFrame
LOOP	轮询等待硬件状态	HAL_UART_WaitOnFlag, FLASH_WaitForLastOperation
IRQ	中断处理/回调函数	HAL_UART_IRQHandler, ENET_TransmitIRQHandler
RETURNOK	纯硬件操作，返回状态值	HAL_GPIO_DeInit, HAL_I2C_IsDeviceReady
SKIP	纯硬件操作，无返回值	HAL_ADC_MspInit, FLASH_FlushCaches
CORE	系统区操作，OS 核心函数	SysTick_Handler, rt_thread_init
NODRIVER	存在的驱动操作对固件状态机无影响	main, GPIO_GetInstance

数虽然存在于驱动模块中，但其操作对固件状态机无影响或属于纯业务逻辑，同样无需替换。该分类体系的设计兼顾了完备性和实用性，确保每类函数都有明确的替换策略或处理方式。

3.2.2.3 替换策略体系设计

本研究总结出五种通用替换策略，每种策略适用于特定的函数类别和使用场景。表 3.3展示了各类函数与替换策略的对应关系，策略的选择由 LLM 根据函数的语义特征自动决定。

以下逐一阐述各策略的设计原理和适用场景。

Skip 策略：移除硬件交互操作。Skip 策略移除驱动函数中的硬件交互操作，但保留函数签名和返回值，使函数执行路径能够支持固件正常运行。对于初始化类函数，移除硬件寄存器配置逻辑；对于控制类函数，移除控制命令的硬件执行；对于轮询类函数，移除等待硬件状态的循环逻辑，避免仿真环境中的死循环。Skip 策略跳过与硬件相关的操作，保留了必要的返回值和函数接口，避免固件因硬件依赖而阻塞或异常。

Redirect 策略：重定向到仿真 I/O。Redirect 策略使用自定义的注入函数（如

表 3.3 替换策略选择指南

函数类别	策略类型	说明
INIT	Skip/Steering	移除硬件寄存器操作，保留结构体初始化，避免仿真环境中的阻塞
RECV	Redirect/Steering	利用 HAL_BE_* 注入函数重定向输入输出，实现数据注入
LOOP	Skip	移除轮询等待逻辑，避免仿真环境中的死循环，直接返回成功状态
IRQ	Event	保留回调函数签名，直接触发事件通知，移除硬件条件判断
RETURNOK	ReturnOK	直接返回成功状态值，保留函数签名，不执行硬件操作
SKIP	Skip	直接跳过函数执行，保留函数签名和调用路径，函数体为空
CORE	NoReplacement	完全不替换，保留原函数代码和调用关系，由仿真器提供底层支持
NODRIVER	NoReplacement	不进行任何替换，保留原函数代码和调用关系

HAL_BE_In、HAL_BE_ENET_ReadFrame、HAL_BE_Out）替代硬件访问，为替换后的驱动代码提供与仿真环境交互的标准接口。对于接收类函数，使用 HAL_BE_In（固定长度）或 HAL_BE_ENET_ReadFrame（变长报文）将仿真输入数据填充到缓冲区；对于发送类函数，使用 HAL_BE_Out 重定向输出或在不影响状态时跳过写操作。

Event 策略：事件驱动模拟。Event 策略通过事件队列模拟硬件中断触发。在仿真环境中，硬件中断信号由仿真器生成的事件触发，而非真实的硬件信号。该策略保留中断处理函数中的数据处理和回调触发逻辑，移除硬件中断标志寄存器的操作，通过控制流改写让中断处理函数必然执行到 OS 通知和回调触发分支。

Steering 策略：执行流导向。Steering 策略通过控制流旁路和改写，将因硬件依赖而不可达的执行路径导向到仿真可达且语义保持的分支。该策略适用于依赖硬件标志的循环、基于 DMA+RingBuffer 的数据接收状态机等复杂场景。当固件因为硬件依赖导致执行路径不可达时，Steering 策略改写控制流把执行导向到仿真可用的路

径，同时保证数据传输和状态更新的语义一致性。

ReturnOK 策略：直接返回成功。ReturnOK 策略针对 RETURNOK 类函数，这类函数通常是简单的辅助函数，主要功能是设置状态、清理资源或执行简单的检查。该策略保留函数签名和必要的参数校验，删除函数体的具体实现，直接返回成功状态（如 HAL_OK）。

与已有方法替换策略的差异。已有固件仿真方法在处理硬件依赖时主要采用两种策略：一是 HALucinator[10] 通过手工编写 HAL profile 来拦截和替换函数调用，该方法依赖领域专家针对每种芯片平台逐一手动编写适配规则，可扩展性有限；二是 P2IM[20] 和 Pretender[17] 采用随机或学习策略生成外设响应，在运行时通过 fuzzing 方式探索外设行为空间，但随机生成的响应可能导致固件执行偏离正常路径。本系统采用的”先分类后替换”策略具有三点优势：（1）通过语义分类识别函数的硬件交互模式，针对性地选择替换策略，避免了”一刀切”替换导致的语义丢失；（2）利用 LLM 的代码理解能力自动生成替换代码，无需针对每种芯片平台手动编写适配规则；（3）通过 Redirect 和 Steering 等策略提供结构化的数据注入机制，相比随机响应更能保证固件执行路径的语义正确性。

3.2.3 替换验证机制设计

替换代码生成后，需要在实际集成到固件工程之前进行严格的质量检查。由于替换代码由大语言模型生成，可能存在语法错误、签名不一致或语义偏移等问题，若未经校验直接写入工程，将导致编译失败或运行时异常。因此，系统设计了多层验证机制，在代码写入工程前尽可能拦截不合格的替换实现，减少后续编译修复的迭代次数，提高整体效率。

系统对生成的替换代码进行以下四层验证：

设原始函数为 f，生成的替换函数为 f'，替换函数的替换代码为 $code(f')$。四层验证的形式化定义如下：

（1）**语法正确性检查**：验证 $code(f')$ 的语法结构完整性：

$$V_1(f') = syntax_valid(code(f'))$$

（2）**签名匹配检查**：验证替换函数签名与原始函数完全一致：

$$V_2(f') = (sig(f') = sig(f))$$

其中 $sig(f) = (name(f), ret_type(f), params(f))$ 表示函数签名三元组。

（3）**返回值检查**：验证所有执行路径的返回值与返回类型一致：

$$V_3(f') = \forall \pi \in paths(f') : ret_value(\pi, f') \in ret_domain(f)$$

其中 $paths(f')$ 表示函数 f' 中从入口到出口的所有执行路径。

（4）**替换规则检查**：验证替换代码遵守策略约束条件：

$$V_4(f') = rule_check(code(f'), strategy(f'))$$

替换函数通过验证当且仅当四层检查全部通过：

$$valid(f') = V_1(f') \wedge V_2(f') \wedge V_3(f') \wedge V_4(f')$$

3.2.3.1 语义保持性定义

替换验证的目标是确保替换代码在仿真环境中保持原始函数的业务语义。由于驱动函数的语义由其与固件其他部分的交互决定，语义保持性无法仅通过静态分析完全验证，需要通过动态测试闭环进一步确认。本研究将语义保持性定义为以下约束条件：

设原始驱动函数 f 的语义为 $\mathcal{S}(f)$，替换函数 f' 的语义为 $\mathcal{S}(f')$。语义保持性要求替换后的固件工程 P' 在仿真环境中满足以下条件：

（1）控制流可达性：设 $B(P)$ 为固件工程 P 中从入口到各关键节点（如 `main` 函数入口、RTOS 内核启动点、协议栈初始化完成点）的执行路径集合，则：

$$\forall \pi \in B(P_{target}) : \exists \pi' \in B(P') : key_nodes(\pi') \supseteq key_nodes(\pi)$$

其中 P_{target} 为目标执行路径集，$key_nodes(\pi)$ 表示路径 π 上的关键节点集合。该条件确保固件在替换后仍能到达所有关键执行节点。

（2）状态一致性：设 Σ 为固件状态空间，$\sigma_0 \in \Sigma$ 为初始状态，$step(P, \sigma)$ 为固件执行一步的状态转移函数。对于固件中不受替换影响的软件模块（如操作系统调度器、协议栈逻辑），其可观测状态在替换前后应保持一致：

$$\forall m \in M_{soft} : obs(step^*(P, \sigma_0), m) = obs(step^*(P', \sigma_0), m)$$

其中 M_{soft} 为纯软件模块集合，$obs(\sigma, m)$ 为对模块 m 的可观测状态提取函数，$step^*$ 表示多步执行。

（3）接口兼容性：替换函数 f' 对调用方的可观测行为应与原始函数 f 一致，包括返回值类型正确、参数列表不变、对非硬件相关的全局状态的副作用保持一致：

$$\forall c \in callers(f) : contract(f', c) = contract(f, c)$$

其中 $contract(f, c)$ 表示函数 f 对调用方 c 的行为契约（返回值、参数副作用）。

这三个条件共同构成替换的语义保持性要求。控制流可达性通过动态测试验证——仿真运行时检查固件是否到达预设的关键节点；状态一致性通过仿真运行中的状态采集和比对验证；接口兼容性通过前述四层静态验证（$V_1 \wedge V_2 \wedge V_3 \wedge V_4$）和动态测试联合保障。由于语义保持性无法通过纯静态分析完全保证，系统设计了”静态验证 $\rightarrow$ 动态测试 $\rightarrow$ 反馈修正”的多层保障机制，通过迭代闭环逐步满足语义保持性要求。

这四层验证形成层次化的质量过滤机制：语法检查过滤最基本的格式错误，签名和返回值检查确保函数接口的一致性，替换规则检查验证替换策略的执行规范性。未通过验证的替换代码将被退回分析智能体重新生成，并附上具体的错误描述，以便 LLM 在下一轮生成中针对性地修正问题。

3.3 动态测试反馈闭环模块设计

动态测试反馈闭环模块是连接”代码替换”与”仿真验证”的关键环节，负责将经过静态分析和替换处理的固件部署到仿真平台中执行，通过实际运行检验替换代码的正确性，并将运行过程中发现的错误进行迭代修复。已有固件仿真方法在验证环节普遍存在不足：P2IM[20] 和 Pretender[17] 通过 fuzzing 反馈优化外设响应生成，但

其反馈目标是改进随机策略的参数，而非修复固件本身的执行问题；HALucinator[10] 依赖人工分析仿真失败原因并手动调整 HAL profile，缺乏自动化反馈机制。本研究提出编译错误与运行时错误的双层闭环修复机制：模块二中的编译修复智能体处理替换代码集成阶段的静态错误（语法错误、类型不匹配），模块三中的闭环修复智能体处理仿真运行阶段的动态错误（崩溃、死循环、语义偏差），两层闭环通过仿真执行智能体统一调度，实现了从静态替换到动态验证的全自动化。本系统设计了”运行 → 诊断 → 修复 → 回归” 的自动化闭环：首先在基于 Unicorn 引擎的仿真平台上加载并执行替换后的固件，通过多种钩子机制全面采集指令执行、基本块覆盖、函数调用和中断触发等运行时信息；当仿真运行出现崩溃、死循环或内存异常等错误时，仿真执行智能体自动分析运行日志定位问题函数，触发闭环修复智能体生成修复方案，修复后重新构建并再次运行仿真，直至固件能够正常完成执行或达到最大迭代次数。本节依次阐述动态执行环境设计、仿真执行智能体设计、反馈信息结构化设计和迭代优化机制设计。

3.3.1 动态执行环境设计

3.3.1.1 仿真平台架构设计

系统采用基于 Unicorn 引擎的固件仿真框架进行动态测试。在仿真平台的选型上，本模块选择 Unicorn 而非 QEMU，主要基于以下理由。Firmadyne[12] 和 HALucinator[10] 等系统基于 QEMU 构建仿真环境，QEMU 提供了完整的外设模型和操作系统支持，适合运行基于 Linux 的固件，但对于裸机固件而言，QEMU 的设备模型过于重量级，且难以精确控制函数级别的执行拦截。Unicorn 是库级别的 CPU 模拟器，支持 ARM Thumb 模式，可以直接执行 Cortex-M 固件；同时提供细粒度的 Hook 机制（指令级、基本块级、内存访问级），可以在执行过程中拦截注入函数调用并转移至宿主机侧处理。P2IM[20] 同样基于 Unicorn 构建仿真平台，验证了 Unicorn 在裸机固件仿真场景中的适用性，但 P2IM 采用随机生成策略模拟外设响应，而本系统通过驱动函数替换和结构化数据注入实现更精确的仿真。

固件仿真子模块采用双层架构设计，由**仿真引擎层**与 **Hook 机制层**构成。仿真

引擎层负责提供指令集执行、内存映射等基础仿真能力；Hook 机制层则在指令流和访存路径上设置回调机制，实现函数入口的拦截和宿主侧逻辑的调用，从而支持驱动替换后注入接口的执行。

仿真引擎层基于 Unicorn ARM 模拟器实现 ARMv7-M Thumb 指令集的模拟执行，包括寄存器管理、内存访问以及固件映像的加载和地址空间映射配置。

Hook 机制层通过代码 Hook、基本块 Hook、内存读/写 Hook 和中断 Hook 等机制，实现执行过程中的信息采集和干预。该机制主要用于代码覆盖率统计、异常内存访问检测和执行轨迹记录等分析任务。在与替换策略配套时，Hook 机制层承担了关键作用：通过预留 `HAL_BE_*` 系列注入接口进行函数入口级拦截。具体而言，链接阶段的弱符号桩在宿主机侧被接管，当执行流到达约定入口时，不再在模拟 CPU 上执行原函数体，而是转入宿主机例程，完成缓冲区读写、数据帧搬运或块传输等操作，并按照调用约定返回结果和更新内存状态，随后继续执行。通过这种方式，驱动函数使用 Redirect 策略写入固件的 `HAL_BE_*` 接口调用可以被固件仿真模块拦截并执行，以支持固件与仿真平台的数据交互，完成固件与仿真平台的协同工作。

表 3.4归纳**固件仿真模块**中与驱动替换配套的注入接口及其使用场景。

3.3.2 仿真执行智能体设计

本节设计仿真运行主控智能体（Emulator Runner Agent），该智能体作为动态测试反馈闭环的核心调度单元，负责在仿真环境中执行固件并采集运行日志、崩溃信息等执行数据。为行文简洁，下文将仿真运行主控智能体简称为” 仿真智能体”。

3.3.2.1 职责定义与工具链设计

仿真智能体的核心职责包括以下五个方面：（1）启动仿真运行并实时监控执行状态；（2）分析仿真输出并判断运行是否成功；（3）在运行失败时触发故障诊断，定位问题函数；（4）在修复完成后调用闭环构建智能体重新构建；（5）执行回归验证并判断是否需要继续迭代。为实现上述职责，该智能体绑定了三类工具：仿真执行工具（EmulateProject），用于启动固件仿真并采集运行结果；信息查询工具（GetMMIOFunctionEmulateInfo、GetFunctionCallsEmulateInfo），用于获取 MMIO 函

表 3.4 固件仿真模块注入接口与使用场景

注入接口	使用场景
HAL_BE_return_0	恒定返回 0，多用作成功码或空操作占位；替换侧用于仅需固定”成功”语义、无需真实硬件操作。
HAL_BE_return_1	恒定返回 1，多用作布尔真或约定非零语义。
HAL_BE_Out	以输出长度表征一次发送语义，常见于类 SEND 的输出重定向策略。
HAL_BE_In	定长接收：将仿真输入写入缓冲区，常见于类 RECV 的输入重定向策略。
HAL_BE_ENET_ReadFrame	在变长上界内填充一帧网络类数据，常见于类 RECV 的输入重定向策略。
HAL_BE_Block_Read	按块大小与块数从虚拟介质读入缓冲区，常见于存储或块设备类替换。
HAL_BE_Block_Write	按块粒度写出缓冲区内容，常见于块设备或大块 I/O 类替换。

数调用日志和函数调用栈信息，支撑故障诊断；以及修复工具和构建工具，通过工具调用接口与闭环修复智能体和闭环构建智能体交互。

3.3.2.2 决策逻辑设计

仿真智能体的决策逻辑由大语言模型（LLM）驱动，采用条件分支的状态机模型。LLM 根据当前仿真运行的状态和日志信息，在以下决策点进行判断：（1）仿真结果评估——根据退出码、异常循环检测标志等判断运行是否成功，若成功则终止闭环并返回结果；（2）修复需求判断——根据标准输出和标准错误中的错误信息，判断是否需要调用闭环修复智能体进行修复；（3）构建触发判断——在修复完成后，判断是否需要触发闭环构建智能体执行重新编译；（4）迭代终止判断——根据当前迭代轮次与最大迭代次数限制的比较结果，决定继续迭代或终止闭环并返回失败报告。该决策机制使得智能体能够根据运行时状态自主调整执行路径，而非依赖预定义的固定流程。

3.3.2.3 仿真执行流程设计

仿真智能体的仿真执行流程分为四个阶段。首先为环境准备阶段，仿真智能体根据目标固件的硬件平台配置 CPU 架构模式和内存映射，包括设置程序计数器（PC）和栈指针（SP）的初始值，以及加载链接脚本和中断向量表等配置信息。其次为固件加载阶段，仿真智能体将编译生成的 ELF 文件加载到 Unicorn 引擎的虚拟内存空间中，完成符号解析和地址重定位。第三为执行监控阶段，仿真智能体启动仿真器运行固件，通过指令级 Hook 和内存访问 Hook 实时采集运行日志、程序计数器变化和内存访问记录，同时监控异常信号（如段错误、非法指令）和运行超时情况。第四为结果分析阶段，仿真智能体对采集的日志和状态信息进行综合分析，判断运行是否成功，若失败则提取崩溃地址、错误类型和调用栈等关键信息，生成结构化的反馈摘要传递给故障诊断模块。

3.3.3 反馈信息结构化设计

系统在仿真运行过程中采集多种类型的运行信息，用于评估替换效果和指导迭代优化。运行监控采集的信息包括执行状态（成功、失败、超时）、程序计数器的最终值、执行的基本块数量、执行的指令总数、标准输出和标准错误输出等。系统定义了统一的数据结构来表示执行结果，包含上述所有采集的信息，该数据结构作为动态测试反馈模块的输出传递给上游模块。

覆盖率统计用于评估固件代码的执行覆盖程度，系统支持基本块级覆盖率统计，通过块 Hook 记录执行过程中访问的所有基本块的地址。错误分类系统将运行过程中发生的各种错误按照类型进行分类，包括执行崩溃错误（访问无效内存、执行非法指令等）、执行超时错误（死循环、阻塞等待等）以及外设或 MMIO 访问相关的异常。

仿真运行产生的原始日志通常是大量的非结构化文本信息，系统对原始日志进行结构化处理，提取关键信息形成反馈摘要。反馈摘要包含迭代轮次、失败类型、错误地址、崩溃信息、相关驱动函数列表和修复建议等字段。反馈信息结构化处理流程包括日志解析、关键信息提取、相关驱动函数分析和摘要生成四个阶段，通过调

用栈分析确定错误发生时正在执行的驱动函数及其调用链。

3.3.4 迭代优化机制设计

当运行失败时，闭环修复智能体被触发进行针对性修复。需要指出的是，闭环修复智能体与模块二中的编译修复智能体职责不同：编译修复智能体处理替换代码首次集成时的编译错误（如语法错误、类型不匹配等静态问题），闭环修复智能体处理仿真运行时发现的动态错误（如崩溃、死循环、语义偏差等运行时问题），两类错误的诊断方式和修复策略存在本质差异。设第 i 轮迭代的状态为 $S_i = (P_i, E_i)$，其中 P_i 为当前固件工程，E_i 为仿真运行产生的错误集合。闭环修复过程定义为状态转移序列：

$$
\begin{aligned}
S_0 &= (P_0, E_0) \\
S_{i+1} &= \begin{cases}
(P_i, \emptyset) & \text{if } E_i = \emptyset \quad \textit{(成功终止)} \\
\mathit{Fix}(P_i, E_i) & \text{if } E_i \neq \emptyset \wedge i < K \wedge \neg \mathit{stagnant}(i) \quad \textit{(继续修复)} \\
S_i & \text{otherwise} \quad \textit{(失败终止)}
\end{cases}
\end{aligned}
\tag{3.1}
$$

其中 K 为最大迭代次数，$\mathit{Fix}(P_i, E_i)$ 表示对工程 P_i 施加基于错误 E_i 的修复操作，$\mathit{stagnant}(i)$ 表示连续 N 轮迭代未能引入新的修复。修复操作包括以下步骤：

3.3.4.1 错误诊断算法设计

系统采用基于规则与大语言模型相结合的错误诊断算法，根据运行失败的类型选择相应的诊断策略：

编译错误诊断。编译错误的诊断信息由编译器直接提供（如 `error: unknown type name`、`warning: implicit declaration`），系统通过正则表达式从编译器输出中提取错误类型、源文件路径和行号，精确定位到具体的替换函数和出错语句。诊断算法优先锁定编译器报告的第一个错误，原因在于后续错误往往由首个错误的连锁反应引起，修复首个错误后重新编译可大幅减少需要处理的错误数量。

运行崩溃诊断。运行崩溃由 Unicorn 引擎捕获的异常信号触发，常见类型包括访问无效内存地址（如 `SIGSEGV`）、执行非法指令（如 `SIGILL`）和栈溢出等。诊断算法

从崩溃现场提取以下信息：程序计数器（PC）的值、触发崩溃的指令地址、当前函数调用栈和崩溃时的寄存器状态。通过将崩溃地址与静态分析模块输出的函数地址范围进行比对，确定崩溃发生在哪个替换函数内部还是固件业务逻辑中。若崩溃发生在替换函数内部，则直接定位到该函数进行修复；若崩溃发生在业务逻辑中，则通过调用栈分析追溯到最近一次调用的替换函数，判断是否因替换不当导致传入参数异常。

运行超时诊断。运行超时由预设的执行时间上限或指令数上限触发，通常表明固件进入了死循环或阻塞等待状态。诊断算法通过分析超时时刻的程序计数器分布，识别反复执行的指令区间和对应的循环结构，判断超时是否由未被替换的轮询类驱动函数引起。若确认为 LOOP 类函数未被正确替换，则触发该函数的重新分析和替换。

3.3.4.2 迭代终止条件

系统的迭代优化过程设置以下终止条件，确保闭环不会无限运行：

成功终止条件。当仿真运行满足以下全部条件时终止迭代：固件执行到预设的关键节点（如 `main` 函数入口、RTOS 内核启动、协议栈初始化完成等）；仿真运行过程中未发生崩溃或超时；替换代码通过编译验证且无编译器警告。

失败终止条件。当满足以下任一条件时终止迭代并输出失败报告：连续 N 轮迭代未能引入新的修复（N 的默认值为 3），表明当前错误已超出自动修复能力；迭代轮次达到预设的最大迭代次数上限（默认值为 10），防止过长时间占用计算资源；诊断算法判断错误类型为不可自动修复的类别（如固件依赖的第三方库函数缺失、链接器脚本配置错误等）。

通过”静态分析 → 智能替换 → 编译验证 → 动态测试 → 反馈优化”的完整闭环，系统不断迭代优化替换策略。

3.4 本章小结

本章围绕”基于驱动程序替换的固件仿真”给出了系统级设计方案。系统采用”静态识别 → 智能替换 → 构建验证 → 动态反馈”的分层模块化架构，各模块职责分

工和数据流关系如第 3 节所述。在此基础上，本章分别完成了三个核心模块的设计：

（1）静态分析模块设计：提出了基于 CodeQL 污点追踪的 MMIO 访问模式识别方法，形式化定义了直接地址强转、间接初始化和常量指针参数三种定位模式，设计了驱动函数定位准则和数据流追踪方法，构建了统一的数据模型。

（2）替换模块设计：提出了基于语义分类的驱动函数替换方法，构建了涵盖八类函数的分类体系和五种对应的替换策略，设计了四层验证机制和语义保持性定义，通过大语言模型驱动的智能体架构实现了分类和替换的自动化。

（3）动态反馈模块设计：设计了编译错误与运行时错误的双层闭环修复机制，基于 Unicorn 引擎构建了仿真平台，设计了仿真智能体和迭代优化流程，实现了”运行 → 诊断 → 修复 → 回归”的自动化闭环。

这三个模块协同工作，实现了”静态识别 → 智能替换 → 构建验证 → 动态反馈”的完整闭环。

4 系统实现

4.1 驱动函数接口静态分析与识别模块实现

4.1.1 CodeQL 查询实现

本节详细阐述分析与识别模块的具体实现技术。模块采用 CodeQL 静态分析引擎作为底层支撑，CodeQL 通过构建包含语法树、控制流图、数据流图的综合数据库，支持对 C/C++ 代码的语义级查询分析。

4.1.1.1 MMIO 操作定位查询实现

针对三种 MMIO 定位模式，本研究设计了相应的查询规则。查询从 CodeQL 数据库中提取所有符合 MMIO 结构体特征的变量及其初始化表达式。查询的核心设计思想是将”寄存器字段访问”追溯到一个可被 CodeQL 稳定描述的基对象表达式，如代码 4.1所示。

```
import cpp
import libraries.MMIOAnalyzer

from
    Function func,
    InterestingMMIOExpr expr
where
    isInterestingMMIOFunction(func) and
    expr.getEnclosingFunction() = func
select expr.toString(),
       expr.getEnclosingFunction().getName(),
       expr.getFile().getAbsolutePath().toString(),
       expr.getLocation().getStartLine(),
       expr.checkExprType(),
       expr.checkEnclosingType()
```

代码清单 4.1 MMIO 表达式收集查询示例

该查询通过 `import libraries.MMIOAnalyzer` 引入 MMIO 分析库，从 CodeQL 数据库中提取所有符合条件的 MMIO 表达式。查询的输出包括表达式字符串、所属函数名、文件路径、行号、表达式类型和封闭类型等信息。

对于”间接初始化”和”常量指针参数”模式，查询采用数据流追踪策略，通过追踪赋值语句和函数参数来识别 MMIO 实例。常量基址的追踪利用 CodeQL 提供的数据流和污点追踪能力，将”常量大值表达式（外设基址）”追踪到后续的变量访问/解

引用位置，如代码 4.2所示。

```
predicate isMMIOConstantExpr(ConstantExpr src_mmio) {
    exists( DataFlow::Node source, DataFlow::Node sink, Expr varAccess,
            PointerDereferenceExpr deref |
        source.asExpr() = src_mmio and
        isConstantBigValueExpr(src_mmio) and
        sink.asExpr() = varAccess and
        deref.getOperand() = varAccess and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}

predicate isMMIOTracedExpr(Expr e) {
    exists( DataFlow::Node source, DataFlow::Node sink, ConstantExpr src_mmio |
        source.asExpr() = src_mmio and
        isMMIOConstantExpr(src_mmio) and
        sink.asExpr() = e and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}
```

代码清单 4.2 常量基址数据流追踪查询示例

该查询定义了两个谓词：isMMIOConstantExpr 识别常量大值表达式（外设基址），isMMIOTracedExpr 通过污点追踪将源点追踪到汇点。查询的逻辑可形式化表示为：当存在一个常量大值表达式作为数据流源点，经过数据流传播到达某个变量访问或解引用操作时，该操作即为 MMIO 相关的表达式。

```
// 示例：常量基址在代码中的传播
#define USART1_BASE 0x40011000UL
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;  // 基址强转
huart->Instance = USART1;                              // 赋值给Instance成员
huart->Instance->DR = data;                            // 最终的字段访问点
```

代码清单 4.3 常量基址在代码中的传播示例

查询通过污点追踪，将上述代码中的 USART1_BASE（源点）追踪到最终的 huart->Instance->DR（汇点），从而识别出整个 MMIO 使用链路，如代码 4.3所示。

对于” 函数第 0 个参数是指针类型，实参是常量大值表达式” 的情况，查询通过函数调用关系和参数类型匹配，定位” 把常量基址传给接受结构体指针的驱动函数” 的调用点，如代码 4.4所示。

```
predicate isMMIOStructure_MK(Struct s){
    exists( FunctionCall call, Expr e, TypedefType typeDef |
        call.getTarget().getParameter(0).getType() instanceof PointerType and
        (
            (call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = typeDef and
             typeDef.getBaseType() = s) or
            call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = s
        ) and
```

```
        call.getArgument(0) = e and
        isConstantBigValueExpr(e)
    )
}
```

代码清单 4.4 函数参数类型匹配查询示例

该查询通过参数类型匹配（PointerType）和实参的常量大值表达式检查，识别出 MMIO 结构体类型的函数调用点。查询的逻辑可表示为：当函数调用的第 0 个参数类型为结构体指针，且该参数对应的实参是经过数据流追踪的 MMIO 常量表达式时，该调用点即为候选的 MMIO 使用点。

```
// 示例：常量指针参数传递
void uart_send_data(USART_TypeDef *uart, uint8_t *data, uint16_t len);

// 调用时传递MMIO实例
uart_send_data(USART1, buffer, len);     // 直接传强转实例
uart_send_data(huart->Instance, buffer, len);  // 传Instance成员
```

代码清单 4.5 常量指针参数传递示例

查询通过参数类型匹配（USART_TypeDef *）和实参的数据流追踪，识别出上述调用点中的 MMIO 参数传递关系，如代码 4.5所示。

4.1.1.2 驱动函数定位查询实现

驱动函数定位模块的核心任务是根据前述定位准则，从固件源代码中识别出所有驱动函数，并构建驱动函数调用图。模块采用 CodeQL 静态分析技术，通过定义驱动函数的判定规则来实现自动化识别。

模块的核心判定规则是：**包含 DriverFieldAccess 且不包含 MMIOFieldAccess 的函数被定义为驱动函数**。DriverFieldAccess 指的是通过句柄或间接方式访问 MMIO 寄存器块的字段操作，而 MMIOFieldAccess 指的是直接通过 MMIO 结构体指针访问寄存器的字段操作。这一定义确保了识别出的函数是真正需要替换的驱动层函数，而非底层的硬件抽象层函数。判定规则的 CodeQL 实现如代码 4.6所示。

```
class DriverFunction extends Function
{
    DriverFunction() {
        exists(DriverFieldAccess fa | fa.getEnclosingFunction() = this) and
        not exists(MMIOFieldAccess fa | fa.getEnclosingFunction() = this)
    }
}
```

代码清单 4.6 驱动函数判定规则

该查询定义了 DriverFunction 类，该类继承自 Function，通过检查函数内部是否包含 DriverFieldAccess 且不包含 MMIOFieldAccess 来判定函数是否为驱动函数。

在识别出驱动函数列表后，模块进一步分析驱动函数之间的调用关系，构建函数调用图。调用图以驱动函数为节点，函数调用关系为边，记录调用方、被调用方、调用位置等信息。调用图记录了函数间的调用依赖关系，有助于理解函数的调用上下文和语义关联。

```
from DriverToCall call
select call.toString2(),
       call.getTargetFunction().toString(),
       call.getEnclosingFunction().toString(),
       call.isDirectCall()
```

代码清单 4.7 驱动函数调用关系查询

该查询从 DriverToCall 关系中提取驱动函数之间的调用关系，输出调用边的字符串表示、目标函数名、封装函数名以及调用是否为直接调用等信息，如代码 4.7所示。

4.1.2 数据模型实现

为便于后续模块使用，静态分析结果被组织为统一的数据模型。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化，如代码 4.8所示。

代码清单 4.8 数据模型定义示例

```
# 示例：数据模型定义
@dataclass
class MmioAccessInfo:
    struct_type: str      # 结构体类型，如"USART_TypeDef"
    field_name: str       # 字段名，如"DR"
    access_type: str      # 访问类型：READ/WRITE
    line_number: int      # 行号
    function_name: str    # 所在函数名
    context_code: str     # 上下文代码

@dataclass
class FunctionInfo:
    name: str             # 函数名
    file_path: str        # 文件路径
    line_number: int      # 行号
    signature: str         # 函数签名
    mmio_accesses: List[MmioAccessInfo]  # MMIO访问列表
```

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、

字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

4.1.2.1 查询工具函数实现

各模块的数据通过一组工具函数对外暴露，供分析智能体和修复智能体在 LangGraph 工作流中调用。工具函数基于 LangChain 的 `@tool` 装饰器实现，将底层的数据查询封装为 LLM 可直接调用的工具接口。以下列举系统中的主要工具函数。

GetFunctionInfo 工具接收函数名作为输入，从静态分析结果中检索对应的 FunctionInfo 对象，返回函数的完整源码、签名、参数列表、返回类型、所在文件和行号等信息。当查询的函数名不存在时，工具返回空结果并提示可能的函数名变体（如去掉前缀/后缀后的匹配），供 LLM 纠正查询参数。

GetMMIOFunctionInfo 工具接收函数名作为输入，返回静态分析阶段提取的该函数所有 MMIO 访问点信息，包括访问的结构体类型、字段名、访问类型（读/写）和上下文代码，供分析智能体在分类阶段判断函数与硬件交互的具体方式。

GetFunctionCallStack 工具接收函数名作为输入，从静态分析构建的函数调用图中提取该函数的调用者列表和被调用者列表，返回调用关系的双向视图，供分析智能体理解函数在驱动架构中的定位。

除以上基于静态分析结果的查询工具外，系统还提供了基于仿真运行结果的查询工具。**GetMMIOFunctionEmulateInfo** 返回 MMIO 函数在仿真运行中的调用日志（包括调用次数、参数值和返回值），**GetFunctionCallsEmulateInfo** 返回仿真运行中的函数调用栈信息。这两个工具的数据来源为仿真运行阶段采集的运行时日志，而非静态分析结果，主要用于仿真执行智能体在故障诊断时定位问题函数的调用上下文。

此外，系统还提供 GetStructOrEnumInfo（查询数据结构定义）、GetDriverInfo（获取驱动模块上下文）等辅助工具，供智能体按需调用。各工具函数的返回结果均

经过格式化处理，以结构化的文本形式呈现给 LLM，避免原始 JSON 数据对模型推理的干扰。基于静态分析的工具函数内部实现了缓存机制，对同一函数的重复查询直接返回缓存结果，减少对 CodeQL 数据库的重复访问。

4.1.3 分析流程实现

静态分析模块的分析流程包括四个主要步骤：CodeQL 数据库构建、查询执行、结果聚合和模型存储。

步骤 1：数据库构建。调用 `codeql database create` 命令构建 CodeQL 数据库，该过程包括源码解析、AST 生成、控制流图和数据流图的构建。数据库构建完成后，可以通过 CodeQL 查询语言对代码进行语义级分析。

步骤 2：查询执行。依次执行前述设计的查询规则，包括 MMIO 表达式收集查询、常量基址数据流追踪查询、驱动函数判定查询和调用关系查询。每个查询返回一组结果，这些结果被转换为 Python 对象供后续处理。

步骤 3：结果聚合。将各个查询的结果进行聚合和关联。例如，将 MMIO 表达式与函数信息关联，构建完整的函数-MMIO 访问映射；将驱动函数与调用关系关联，构建函数调用图。

步骤 4：模型存储。将聚合后的结果存储到统一的数据模型中，并通过 JSON 序列化保存到文件。存储的数据模型包括项目分析结果、函数信息列表、MMIO 访问点列表和调用关系图。这些数据通过文件系统缓存，支持后续模块的快速查询和断点续跑。

4.2 驱动函数分析、替换与编译模块实现

4.2.1 智能体工作流实现

本节阐述基于 LangGraph 框架的智能体工作流实现。LangGraph 是一个用于构建有状态、多智能体应用的框架，它通过图（Graph）来表示应用的状态转换和逻辑流，特别适合本系统这种包含多轮迭代和条件分支的复杂场景。

4.2.1.1 驱动函数分析与替换代码生成模块实现

驱动函数分析与替换代码生成模块的设计逻辑见第 3.2.1.1 节，本节阐述其基于 LangGraph 框架的具体实现。该模块的核心功能是以单个驱动函数为粒度，通过分析智能体（Analyzer Agent，下文简称分析智能体）依次完成信息获取、函数分类和替换代码生成三个阶段，实现从原始驱动函数到仿真兼容实现的自动化转换。

分析智能体基于 LangGraph 的 `StateGraph` 类构建，采用图式状态机架构。智能体的核心状态包括当前处理的函数信息、函数分类结果、替换代码、验证结果和错误信息。智能体的转换逻辑由条件函数决定，根据验证结果决定是否需要重新生成替换代码或进入下一个处理阶段。在实现上，智能体通过 `@tool` 装饰器将 GetFunctionInfo、GetFunctionCallStack、GetMMIOFunctionEmulateInfo 等工具函数注册到 LangGraph 框架中，通过 `compile()` 方法编译为可执行的 `CompiledGraph` 对象，由执行器调度节点的执行顺序并管理状态传递。

4.2.1.2 替换代码修正与编译集成模块实现

替换代码修正与编译集成模块的设计逻辑见第 3.2.1.2 节，本节阐述其具体实现。该模块负责对上一模块输出的候选替换代码进行二阶段检验与修复，由编译修复智能体（BuilderFixer Agent，下文简称编译修复智能体）统一完成代码集成、编译构建和错误修复，目标是在真实构建环境下得到可编译、可链接的替换定义。

代码集成机制：系统采用”原位替换”策略将替换代码写入工程。对于每个需要替换的驱动函数，系统在目标源文件中保留原始函数定义的快照（用于回滚），然后将替换实现写入函数体。同时，系统在工程中添加弱符号桩函数文件，为仿真注入接口（`HAL_BE_*` 系列函数）提供链接占位，确保在非仿真环境中不会因未定义符号而导致链接失败。这种设计使得同一个工程既可以在原始硬件环境中编译运行，也可以在仿真环境中使用替换代码运行，无需维护两套代码副本。

编译构建与错误修复流程：编译修复智能体将替换代码应用到工程后执行编译构建，若编译失败则自动进入错误解析与修复循环。替换代码应用通过 ReplaceFuncs 工具实现，准确定位函数在源文件中的位置、保留注释和格式、生成回滚快照；工

程构建通过 BuildProject 工具执行 CMake 构建，配置 ARM 工具链环境并捕获编译输出。内置的编译错误解析器支持 GCC、Clang、ARMCC 等多种编译器，能够将诊断信息转换为包含错误类型、位置和消息的结构化错误对象。编译失败时，编译修复智能体自动解析编译器输出的错误信息，定位到具体的替换函数和错误原因，然后生成针对性的修复方案并重新构建，形成”编译 → 错误解析 → 修复 → 重新编译”的闭环。修复策略的选择由 LLM 驱动，LLM 根据错误信息和函数上下文智能地选择补声明、改类型与转换、补 `#include`、删改非法成员访问等策略。编译修复智能体采用”锁定首个错误”策略，优先修复编译器报告的第一个错误所涉及的函数，避免同时处理多个错误导致修复方案相互冲突。编译错误的典型成因包括：函数未声明或声明不匹配、类型不兼容（如 LLM 将 `uint32_t` 参数改写为 `int`）、缺少头文件包含、替换代码引用了目标芯片外设库中不存在的结构体成员等。每次修复后重新触发编译，直至编译成功或达到最大修复轮次。

4.2.1.3 替换代码验证机制实现

第 3 章第 3.2.3 节定义了替换代码的四层验证机制，本节阐述其具体实现。

语法正确性检查（V_1）通过 Python 的正则表达式和括号匹配算法实现。验证器对替换代码进行以下检查：括号（花括号、圆括号、方括号）是否匹配；语句是否以分号结尾（排除控制结构行）；是否存在未闭合的字符串字面量；函数体是否非空。对于语法检查未通过的代码，验证器返回具体的语法错误描述（如”第 3 行缺少右花括号”），帮助 LLM 在下一轮生成中定位修正。

签名匹配检查（V_2）通过将替换代码的函数签名与静态分析模块提供的原始函数签名进行逐字段比对实现。签名提取器通过正则表达式从替换代码的首行解析函数名、返回类型和参数列表，然后与 FunctionInfo 中记录的签名进行匹配。检查内容包括：函数名完全一致、返回类型一致、参数数量一致、每个参数的类型和名称一致。常见的不匹配情形包括 LLM 遗漏参数、将指针参数写为值参数、将 `const` 修饰符遗漏等。

返回值检查（V_3）通过遍历替换函数中所有执行路径的返回语句实现。验证器

首先检查函数的返回类型：对于 void 类型函数，验证不存在带值的 return 语句；对于非 void 类型函数，验证所有执行路径（通过简化的控制流分析识别分支和循环的出口）均包含返回语句，且返回值的类型与声明的返回类型兼容。例如，返回类型为 HAL_StatusTypeDef 的函数，所有返回值应为该枚举的有效成员（如 HAL_OK、HAL_ERROR、HAL_TIMEOUT）。

替换规则检查（V_4）根据函数的分类类别执行策略特定的约束检查。各策略的检查规则包括：Skip 策略——替换代码不应包含任何 MMIO 寄存器访问（通过检查是否包含结构体指针的解引用操作 ->）；Redirect 策略——替换代码必须包含至少一个 HAL_BE_* 系列注入函数调用；Event 策略——替换代码必须保留回调函数的调用；ReturnOK 策略——替换代码的所有返回语句必须返回成功状态值。规则检查器的实现基于正则表达式模式匹配和关键字检索，对未满足策略约束的替换代码生成具体的违规描述。

四层验证通过后，验证器生成通过报告，替换代码进入后续的编译集成流程。若任一层验证失败，验证器将失败原因和修改建议附加到分析智能体的上下文中，触发重新生成。

4.2.2 替换策略实现

各替换策略的设计意图与语义定义见第 3.2.2 节，本节通过代码示例展示各策略的具体替换效果。

4.2.2.1 Skip 策略实现

Skip 策略适用于两类场景：初始化类函数（INIT）和轮询等待类函数（LOOP）。对于初始化类函数，Skip 策略移除所有硬件寄存器配置操作，仅保留函数签名，使调用路径不被中断，同时通过参数透传确保结构体指针在后续代码中仍然可用。对于轮询等待类函数，Skip 策略移除基于硬件标志位的 while 循环等待逻辑，直接返回成功状态（HAL_OK），避免仿真环境中因硬件标志永远不置位而导致死循环。代码 4.9展示了 Skip 策略在 UART 初始化和轮询函数上的替换效果。

```
1 // 原始代码（初始化类）
2 void HAL_UART_Enable(UART_HandleTypeDef *huart) {
```

```
    huart->Instance->CR1 |= USART_CR1_UE;
}

// 替换代码 (Skip 策略)
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    (void)huart;
}

// 原始代码 (轮询类)
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t
    Timeout) {
    while (!(__HAL_UART_GET_FLAG(huart, Flag))) {
        if (Timeout != HAL_MAX_DELAY) {
            if ((Timeout--) == 0) return HAL_TIMEOUT;
        }
    }
    return HAL_OK;
}

// 替换代码 (Skip 策略)
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t
    Timeout) {
    (void)huart;
    (void)Flag;
    (void)Timeout;
    return HAL_OK;
}
```

代码清单 4.9 Skip 策略替换示例（初始化类和轮询类）

4.2.2.2 Redirect 策略实现

Redirect 策略适用于接收类函数（RECV），其核心思想是将硬件寄存器轮询替换为仿真注入函数调用，同时保留函数中不影响 I/O 的业务逻辑（如状态字段更新、计数器维护等）。对于定长接收场景，使用 HAL_BE_In 将仿真输入数据填充到目标缓冲区，覆盖原始的寄存器读取逻辑；对于变长报文接收场景（如以太网驱动），使用 HAL_BE_ENET_ReadFrame 在当前有效 buffer 描述符处注入一帧网络数据。代码 4.10展示了 UART 接收函数的替换效果，代码 4.11展示了变长报文接收场景的替换效果。

```
// 原始代码
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    while (!(huart->Instance->SR & USART_SR_RXNE));
    *pData = huart->Instance->DR;
    return HAL_OK;
}

// 替换代码 (Redirect 策略)
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    // Redirect to simulation input function
    HAL_BE_In(pData, Size);
    // Preserve non-I/O logic
    huart->RxXferCount = 0;
```

```
    huart->RxXferSize = Size;
    huart->RxState = HAL_UART_STATE_READY;
    return HAL_OK;
}
```

代码清单 4.10 Redirect 策略替换示例（UART 接收函数）

对于变长报文接收场景（如以太网驱动），代码 4.11展示了在当前有效 buffer 指针处调用 HAL_BE_ENET_ReadFrame 注入数据，完成一次 buffer descriptor 更新后立即返回。

```
// 变长报文接收
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
if (rxBuffer->buffer != NULL && rxBuffer->length > 0) {
    HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);
}
rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
```

代码清单 4.11 Redirect 策略替换示例（变长报文接收）

4.2.2.3 Event 策略实现

Event 策略适用于中断处理和回调函数类（IRQ），其核心思想是绕过硬件中断标志的判断条件，直接触发事件通知和回调函数。在原始代码中，中断处理函数通常通过检查硬件中断标志寄存器（如 ETH_DMA_FLAG_R）来决定是否执行回调，而仿真环境中这些硬件标志永远不会被置位，导致回调逻辑永远无法触发。Event 策略通过移除硬件标志判断，直接调用目标回调函数，使中断处理逻辑能够在仿真环境中正常执行。代码 4.12展示了 ETH 中断处理函数的替换效果。

```
// 原始代码
void HAL_ETH_IRQHandler(ETH_HandleTypeDef *heth) {
    if (__HAL_ETH_DMA_GET_FLAG(heth, ETH_DMA_FLAG_R)) {
        HAL_ETH_RxCpltCallback(heth);
    }
    __HAL_ETH_DMA_CLEAR_FLAG(heth, ETH_DMA_FLAG_R);
}

// 替换代码 (Event 策略)
void HAL_ETH_IRQHandler(ETH_HandleTypeDef *heth) {
    // Trigger Rx complete callback event directly
    HAL_ETH_RxCpltCallback(heth);
    // No need to check hardware flags or clear registers
}
```

代码清单 4.12 Event 策略替换示例

4.2.2.4 Steering 策略实现

Steering 策略适用于依赖复杂硬件状态机的场景，其核心思想是通过控制流改写将因硬件依赖而” 走不到正确路径” 的执行导向到” 仿真可达且语义保持” 的分支。

典型的应用场景是 DMA+RingBuffer 机制：原始代码通过轮询硬件 buffer descriptor 的状态标志位来遍历接收数据，但在仿真环境中这些标志位不会更新，导致循环无法终止。Steering 策略跳过依赖硬件标志的状态扫描循环，在当前 buffer descriptor 处直接调用 HAL_BE_ENET_ReadFrame 注入仿真数据，同时保留 Ring 索引更新和状态字段维护逻辑，确保数据传输和状态更新的语义一致性。代码 4.13和代码 4.14展示了 NXP 网卡驱动中 ENET_GetRxFrame 函数的原始代码和替换后的代码对比。

```
// 原始代码：依赖硬件标志扫描 Ring，仿真中将陷入死循环
index = rxBdRing->rxGenIdx;
while (0U == (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_EMPTY_MASK))
{
    if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
        result = ENET_GetRxFrameErr(curBuffDescrip, &rxFrame->rxFrameError);
        break;
    }
    index = ENET_IncreaseIndex(index, rxBdRing->rxRingLen);
    curBuffDescrip = rxBdRing->rxBdBase + index;
    if (index == rxBdRing->rxGenIdx) break;
}

// 后续：遍历 BD，拷贝数据到 rxFrame
do {
    newBuff = handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U) {
        rxBuffer = &rxFrame->rxBuffArray[index];
        if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
            isLastBuff = true;
            rxFrame->totLen = curBuffDescrip->length;
            rxBuffer->length = curBuffDescrip->length - buffLen;
        } else {
            rxBuffer->length = curBuffDescrip->length;
            buffLen += rxBuffer->length;
        }
        // ... buffer 管理、BD 状态更新 ...
    }
} while (!isLastBuff);
```

代码清单 4.13 Steering 策略原始代码示例（NXP ENET_GetRxFrame）

替换后的代码如代码 4.14所示，跳过依赖硬件标志的 while 循环，在当前 buffer descriptor 调用 HAL_BE_ENET_ReadFrame 注入数据，并保留 Ring 更新逻辑。

```
// 替换代码：跳过硬件标志扫描，直接注入数据
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
index = 0;
do {
    newBuff = (uintptr_t)(uint8_t *)handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U) {
        rxBuffer = &rxFrame->rxBuffArray[index];
        rxBuffer->buffer = (void *)(uint8_t *)address;

        if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
            isLastBuff       = true;
            rxFrame->totLen  = curBuffDescrip->length;
            rxBuffer->length = curBuffDescrip->length - buffLen;
        } else {
            rxBuffer->length = curBuffDescrip->length;
```

```
            buffLen += rxBuffer->length;
        }
    }

    // Steering 核心：注入仿真数据
    if (rxBuffer->buffer != NULL && rxBuffer->length > 0)
        HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);

    // 保留 Ring 更新，防止卡在同一 BD
    index++;
    rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
    curBuffDescrip     = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
} while (!isLastBuff);
```

代码清单 4.14 Steering 策略替换代码示例（NXP ENET_GetRxFrame）

4.2.2.5 ReturnOK 策略实现

ReturnOK 策略适用于 RETURNOK 类函数，这类函数的主要功能是执行简单的硬件操作并返回状态值（如 HAL_OK 或 HAL_ERROR），如 GPIO 配置、外设复位等。在仿真环境中，这些硬件操作既无实际效果也无必要执行，因此 ReturnOK 策略保留函数签名和必要的参数校验（如空指针检查），直接返回成功状态值，跳过所有硬件相关操作。这样既保证了调用方通过返回值判断执行流程的连贯性，又避免了不必要的硬件依赖。代码 4.15展示了 GPIO 去初始化函数的替换效果。

```
    if (GPIOx == NULL) return HAL_ERROR;
    GPIOx->MODER = 0x00000000;
    GPIOx->OTYPER = 0x00000000;
    // ... more register operations ...
    return HAL_OK;
}

// 替换代码 (ReturnOK 策略)
HAL_StatusTypeDef HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx) {
    if (GPIOx == NULL) return HAL_ERROR;
    (void)GPIOx;
    return HAL_OK;
}
```

代码清单 4.15 ReturnOK 策略替换示例

4.2.3 LLM 提示词工程实现

大语言模型的分类和生成质量直接影响替换代码的正确性，因此提示词的设计是系统的关键技术实现之一。本节阐述分析智能体和修复智能体的提示词模板结构设计。提示词工程的目标是：（1）为 LLM 提供函数上下文信息，使其能够理解函数语义；（2）通过结构化的分类规则指导 LLM 做出一致的分类决策；（3）通过生成约束限制 LLM 的输出格式，降低后处理复杂度。

4.2.3.1 分类提示词模板设计

分析智能体的提示词模板采用分阶段结构设计，包含角色定义、工具说明、分类策略和输出格式四个部分。其整体骨架如代码 4.16所示。

```
**Role**: You are an embedded software engineer tasked with
replacing driver library functions to eliminate dependencies
on peripheral hardware while preserving normal functionality
and MCU-related operations (including OS scheduling and
interrupt triggering).
**Overall Process**:
1. Information Gathering: Obtain the function implementation
   and context using available tools.
2. Analysis Phase: Analyze MMIO accesses, call relationships,
   data structures, and core logic.
3. Classification Phase: Categorize the function into one of
   the defined types.
4. Processing Phase:
   - RECV, IRQ, INIT, LOOP: Generate replacement code.
   - RETURNOK, SKIP, NEEDCHECK, NODRIVER: Classification only.
**Tools**: GetFunctionInfo, GetMMIOFunctionInfo,
  GetStructOrEnumInfo, GetFunctionCallStack, GetDriverInfo
  (详见代码~\ref{lst:prompt-tools})
**Categories**: RECV / IRQ / INIT / LOOP / RETURNOK / SKIP /
  NEEDCHECK / NODRIVER (详见代码~\ref{lst:prompt-recv}等)
**Rules**: (详见代码~\ref{lst:prompt-rules})
**Output Format** (JSON):
{
  "function_name": "<name>",
  "function_type": "<RECV|IRQ|INIT|LOOP|...>",
  "functionality": "<brief description>",
  "classification_reason": "<detailed explanation>",
  "has_replacement": <true|false>,
  "function_replacement": "<code or empty string>"
}
```

代码清单 4.16 分类提示词模板骨架

角色定义与工具说明。提示词以角色设定开场，将 LLM 定位为嵌入式软件工程师，明确任务是消除驱动函数的硬件依赖。随后声明五个可调用的工具函数及其调用顺序，如代码 4.17所示。

```
1. GetFunctionInfo(func_name): Get function implementation.
   -> Always call this first.
2. GetMMIOFunctionInfo(func_name): Identify hardware accesses.
   -> Call immediately after GetFunctionInfo.
3. GetStructOrEnumInfo(struct_name): Query data structures.
4. GetFunctionCallStack(func_name): Get call relationships.
5. GetDriverInfo(driver_name): Get driver module context.
   Tool Sequence: GetFunctionInfo -> GetMMIOFunctionInfo
                  -> [3|4|5] (as needed)
```

代码清单 4.17 提示词中的工具声明与调用顺序

提示词要求 LLM 先调用 GetFunctionInfo 获取函数源码，再调用 GetMMIOFunctionInfo 确认硬件访问方式，最后按需调用其余工具补充上下文。固定的工具调用顺序减少了信息遗漏。

分类策略。提示词将八类函数的识别特征和替换策略以结构化形式嵌入。代码 4.18展示了 RECV 类函数的分类描述片段。

```
RECV (Data Transmission/Reception Functions)
  Identification: Functions performing critical data I/O,
    DMA buffer management, or peripheral data transfer.
  Examples: HAL_UART_Receive, ETH_ReadFrame, SPI_Receive
  Strategy:
    Data reception -> HAL_BE_In(buf, len) or
                      HAL_BE_ENET_ReadFrame(buf, length)
    Data transmission -> HAL_BE_Out(len) or skip
  Critical: Preserve all non-I/O logic (buffer pointer
    updates, state flags, data processing).
```

代码清单 4.18 提示词中 RECV 类函数的分类描述

每类函数的定义包含识别特征（函数行为描述和典型函数名）、替换策略（使用哪些注入函数）和关键约束（必须保留的逻辑）。提示词按照 RECV > IRQ > INIT > LOOP 的优先级排列，要求 LLM 先判断高影响类别，减少误分类。

代码生成约束。对于需要生成替换代码的函数类别，提示词设置了以下约束，如代码 4.19所示。

```
General Replacement Rules:
1. Do not change the function's return type.
2. Do not add extern declarations or #include directives.
3. Preserve all OS-related calls (scheduling, semaphores).
4. Only use provided helper functions (HAL_BE_*).
5. Do not fabricate undefined constants or variables.
```

代码清单 4.19 提示词中的通用替换规则

规则（1）和（2）与第 3 章的四层验证机制中的签名匹配检查（V_2）和替换规则检查（V_4）对应，在生成阶段即约束 LLM 遵守接口规范。规则（3）确保操作系统调用不被误删。规则（4）和（5）限制了 LLM 的生成范围，要求仅使用原函数中出现的类型、变量和系统提供的注入函数，从源头减少幻觉问题。

输出格式。提示词要求 LLM 以 JSON 格式输出分类结果，包含函数名、分类类别、分类理由和替换代码字段。结构化输出使后续模块能够自动解析，避免自由格式文本的不确定性。

编译修复智能体的提示词设计较为精炼，主要任务是接收编译器输出的错误信息和对应的函数名，要求 LLM 定位错误原因并生成修复后的替换代码，修复后重新触发编译验证。提示词约束修复时不得修改函数签名、不得引入新的头文件或外部声明，与分析智能体的通用替换规则一致。

4.3 动态测试反馈闭环模块实现

本节阐述动态测试反馈闭环模块的具体实现，其设计方案见第 3 章第 3.3 节。

4.3.1 仿真平台实现

4.3.1.1 Unicorn 引擎初始化与内存映射

系统在初始化阶段通过 Unicorn 引擎的 C API 完成仿真环境的搭建。首先调用 `uc_open(UC_ARCH_ARM, UC_MODE_THUMB, &uc)` 创建 ARM Thumb 模式的模拟器实例，随后依次完成以下配置：（1）内存区域映射，根据目标芯片的链接脚本解析 Flash、RAM 和外设区域的起始地址与大小，调用 `uc_mem_map()` 为各区域分配独立的内存空间并配置读写权限，系统定义了 flash、ram、peripheral_ram 等受保护的内存类型，防止不同区域被错误合并；（2）固件映像加载，支持 ELF 和 BIN 两种格式，通过自动识别代码段（而非数据段）作为 Flash 基址，调用 `uc_mem_write()` 将固件二进制写入虚拟内存；（3）寄存器初始化，通过 `uc_reg_write()` 设置程序计数器（PC）和栈指针（SP）等关键寄存器的初始值，确保固件从正确的入口点开始执行；（4）浮点单元支持，为兼容包含浮点运算的固件（如图像处理、网络协议栈），启用 VFP 浮点单元支持。

4.3.1.2 钩子机制的实现

第 3 章阐述了本系统使用的四类钩子（代码钩子、基本块钩子、内存钩子、中断钩子）的设计原理，本节阐述其具体实现。

在钩子注册阶段，系统通过 `uc_hook_add()` 函数批量注册四类钩子，每类钩子绑定独立的 Python 回调函数。各回调函数接收 Unicorn 实例、触发地址和访问参数，将事件信息写入结构化的日志队列。具体实现中，代码钩子通过 `UC_HOOK_CODE` 类型注册，回调函数接收当前指令地址和指令大小，用于指令计数和执行轨迹记录；基本块钩子通过 `UC_HOOK_BLOCK` 类型注册，回调函数记录基本块入口地址，用于覆盖率统计；内存钩子通过 `UC_HOOK_MEM_READ` 和 `UC_HOOK_MEM_WRITE` 类型注册，回调函数接收访问地址、访问大小和数据内容，用于检测对未映射地址空间的非法访问；

中断钩子通过 UC_HOOK_INTR 类型注册，回调函数接收中断号，用于处理 SVC 指令等软件中断。

在与驱动替换策略配合时，钩子机制承担了关键的拦截角色。系统在仿真初始化阶段通过 ELF 符号表解析所有 HAL_BE_* 注入函数的虚拟地址，然后在这些地址处注册代码钩子。当固件执行到注入函数入口时，钩子回调函数执行以下流程：（1）从 Unicorn 寄存器中读取函数参数（ARM 调用约定下参数通过 r0-r3 寄存器传递）；（2）调用宿主机侧的 Python handler 接口完成数据注入或输出操作；（3）将返回值写入 r0 寄存器；（4）从栈中读取返回地址并设置 PC 指针，跳过原函数体继续执行。通过这一机制，固件中调用 HAL_BE_* 系列接口时，执行流被拦截并转移到宿主机侧的处理例程。

4.3.1.3 弱符号桩函数实现

弱符号桩函数是连接固件代码和仿真环境的桥梁，其设计原理见第 3 章。在工程构建阶段，这些弱符号桩被编译为空函数体，仅作为链接占位符存在，确保固件在没有仿真环境时仍能编译通过。在仿真运行阶段，这些函数的地址被 Unicorn 钩子拦截，实际功能由宿主机侧的 Python 处理器完成。

弱符号桩函数的实现如代码4.20所示。所有桩函数均使用 __attribute__((weak)) 修饰，链接器在存在强符号定义时优先使用强符号，否则使用弱符号桩。在仿真配置中，系统通过替换函数机制将这些桩函数的实际执行逻辑重定向到钩子回调中。通过弱符号桩与 Unicorn Hook 的协同机制，固件代码在硬件环境和仿真环境之间可以零修改切换，同一份源码工程无需维护两套代码副本即可同时支持两种运行模式。

```
__attribute__((weak)) void HAL_BE_In(uint8_t *buffer, uint16_t size) {}
__attribute__((weak)) void HAL_BE_Out(const uint8_t *data, uint16_t size) {}
__attribute__((weak)) void HAL_BE_ENET_ReadFrame(uint8_t *buffer,
                                                 uint16_t size) {}
__attribute__((weak)) uint32_t HAL_BE_return_0(void) { return 0; }
__attribute__((weak)) uint32_t HAL_BE_return_1(void) { return 1; }
__attribute__((weak)) void HAL_BE_Block_Read(uint8_t *buffer,
                                             uint32_t block_size,
                                             uint32_t block_count) {}
__attribute__((weak)) void HAL_BE_Block_Write(const uint8_t *buffer,
                                              uint32_t block_size,
                                              uint32_t block_count) {}
```

代码清单 4.20 弱符号桩函数实现示例

4.3.1.4 运行日志记录模块实现

运行日志记录模块通过前述的 Unicorn 钩子机制，将程序执行过程中的关键事件转化为结构化的日志记录。模块在每类钩子回调中将事件信息写入线程安全的日志队列，仿真结束后统一导出为 JSON 格式供故障诊断使用。日志记录包含以下四类信息：（1）指令级信息，利用代码钩子记录指令地址、指令大小和关键寄存器快照；（2）基本块级信息，利用基本块钩子记录每个基本块的入口地址和执行次数，据此构建代码覆盖率统计；（3）函数调用信息，在驱动函数入口和出口地址处设置代码钩子，记录调用地址、被调用函数名、参数值和返回值；（4）中断触发信息，利用中断钩子记录中断号、触发地址和处理器状态。这些日志信息在仿真运行过程中实时采集，为故障诊断和反馈闭环提供运行时数据基础。

4.3.2 仿真执行智能体实现

仿真执行智能体基于 LangGraph 框架实现，其设计见第 3 章。在实现层面，该智能体通过 `StateGraph` 类构建图式状态机，核心状态定义如下：

代码清单 4.21 仿真执行智能体状态定义

```
@dataclass
class EmulatorState(TypedDict):
    messages: Annotated[list, add_messages]
    project_path: str
    build_config: dict
    replacement_functions: list
    iteration_count: int
    max_iterations: int
    processed_functions: list
    emulate_result: Optional[dict]
```

智能体的图结构包含以下节点：`emulate`（仿真运行节点，调用 EmulateProject 工具）、`diagnose`（故障诊断节点，调用 GetMMIOFunctionEmulateInfo 和 GetFunctionCallsEmulateInfo 工具）、`fix`（修复节点，调用 Fixer 子图）、`build`（构建节点，调用 Builder 子图）。节点间的条件转移逻辑根据仿真结果决定：仿真成功时直接终止；失败时进入诊断节点；诊断完成后进入修复节点；修复完成后进入构建节点；构建完成后返回仿真节点进行回归验证。每个节点的执行由 LLM 驱动决策，LLM 根据当前状态和工具返回的结果决定下一步操作。

4.3.3 反馈闭环实现

4.3.3.1 多智能体协同的 LangGraph 实现

第 3 章设计了由仿真执行智能体、修复智能体和构建智能体组成的多智能体架构，本节阐述其基于 LangGraph 的具体实现。

三个智能体通过 LangGraph 的子图（Subgraph）机制集成。仿真执行智能体作为主图，修复智能体和构建智能体分别编译为独立的 `CompiledGraph` 对象，通过工具调用接口被主图节点调用。这种设计使得各智能体拥有独立的状态空间和工具链，同时通过标准化的输入输出接口实现数据交换。

修复智能体接收仿真执行智能体传递的结构化输入，包括标准输出、标准错误、MMIO 函数调用日志和函数调用栈信息，输出为 `ReplacementUpdate` 对象（包含函数名、替换代码和修复原因）。构建智能体接收修复智能体的输出，执行代码应用和编译构建，输出为 `BuildOutput` 对象（包含退出码、编译输出和错误信息）。所有智能体间通信均通过预定义的数据结构进行，避免自由格式文本传递导致的理解偏差。

4.3.3.2 闭环工作流的具体实现

第 3 章给出了闭环协同工作流的核心逻辑，本节阐述其实现中的关键技术细节。

在故障诊断环节，系统实现了基于程序计数器（PC）分析的崩溃定位方法。当仿真运行崩溃时，系统从 Unicorn 引擎获取崩溃时刻的 PC 值，将其与静态分析模块输出的函数地址范围进行匹配，定位崩溃发生的具体函数。对于超时类型的故障，系统分析超时时刻的 PC 分布直方图，识别反复执行的指令区间和对应的循环结构，判断是否由未正确替换的轮询函数引起。

在修复执行环节，修复智能体采用”锁定单函数”策略：每轮修复仅针对诊断出的首个问题函数进行修改，避免同时修改多个函数导致修复方案相互冲突。修复策略包括：为 LOOP 类函数添加超时退出条件、补充遗漏的状态字段更新、将不适配的 Skip 策略调整为 Redirect 策略等。每次修复后，系统通过数据版本管理机制保留替换代码的历史版本，当修复导致状态恶化时支持快速回退。

在回归验证环节，系统在每次构建成功后自动重新生成仿真配置（包括内存映

射、PC/SP 初始值和钩子注册信息），然后执行完整的仿真运行。系统设置最大迭代次数限制（默认值为 10，与第 3 章设计一致），当达到限制后终止闭环并生成包含每轮迭代失败现象和修复方案的详细故障报告。

4.3.3.3 终止条件与错误处理的实现

闭环的终止判断通过 LangGraph 的条件边（conditional edge）实现。在仿真节点之后，条件函数检查仿真结果：若 `emulate_result.success == true`，转移到终止节点；否则检查迭代计数是否达到上限、诊断是否能定位问题函数，决定继续迭代或终止。

系统实现了三层错误处理机制：（1）编译错误处理，由构建智能体在编译失败时触发，错误信息通过正则表达式解析后传递给修复智能体；（2）运行时错误处理，包括内存访问错误（UC_ERR_FETCH_UNMAPPED、UC_ERR_WRITE_UNMAPPED）、非法指令执行（UC_ERR_INSTRUCTION_INVALID）和超时，错误信息从 Unicorn 异常中提取后传递给故障诊断模块；（3）框架级错误处理，当 LangGraph 达到递归深度限制（GraphRecursionError）时，系统捕获异常并分析对话历史，生成故障报告后退出。系统记录每次迭代的完整信息（失败现象、诊断结果、修复方案、验证结果），便于问题追溯和系统优化。

4.3.3.4 修复智能体提示词模板设计

闭环修复智能体的提示词模板针对运行时错误修复场景设计，与分析智能体的分类提示词不同，修复提示词将错误诊断和修复方案生成整合在一个推理流程中。其骨架如代码 4.22所示。

```
**Role**: Embedded software engineer specializing in HAL
development and debugging. Your task is to actively and
independently analyze emulator error logs to identify
problematic functions and fix them.
**CRITICAL**: You must NOT ask for parameters. You MUST
first actively retrieve error logs using available tools.
**Workflow**:
Step 0: Retrieve emulator logs (function_calls_emulate_info,
        mmio_function_emulate_info)
Step 1: Analyze logs to identify problematic functions
Step 2: Get function context (GetReplaceFuncDetailsByFile)
Step 3: Generate fix and apply (UpdateFunctionReplacement)
**Constraints**: Preserve function signature; no new types
```

```
  or headers; preserve OS core functions (scheduler, context
  switch); do not stub VTOR/NVIC/SysTick writes.
```

代码清单 4.22 修复提示词模板骨架

修复提示词的设计要点包括以下四个方面。

主动日志获取。与分析智能体被动接收函数名不同，修复智能体需要主动从仿真运行结果中定位问题。提示词要求 LLM 首先调用 `function_calls_emulate_info()` 获取完整的执行流和错误信息，再调用 `mmio_function_emulate_info()` 获取 MMIO 函数调用日志，从日志中自行判断哪些函数导致了崩溃或超时，而非等待外部指定。

上下文完整获取。修复提示词要求 LLM 在生成修复方案前必须获取完整的上下文，如代码 4.23所示。

```
Before any code changes, call these tools in order:
1. GetReplaceFuncDetailsByFile(file_path) to get:
   - Original function code and signature
   - All previous replacement attempts with exact code
   - All replacement history with timestamps
2. function_calls_emulate_info() to get execution flow
3. mmio_function_emulate_info() to identify MMIO involvement
DO NOT make code changes until you have complete context.
```

代码清单 4.23 修复提示词中的上下文获取要求

这一要求确保 LLM 了解该函数的历次替换历史，避免重复此前已尝试过且失败的修复方案。

OS 核心函数保护。修复提示词中设置了严格的保护规则，禁止 LLM 修改操作系统核心函数（如 FreeRTOS 的 `vTaskStartScheduler`、`PendSV_Handler`，RT-Thread 的 `rt_schedule` 等）。提示词要求：若崩溃发生在 OS 核心函数中，LLM 应查找附近的驱动初始化代码中的问题，而非改写 OS 核心函数本身。

步数预算控制。提示词明确告知 LLM 总步数限制，要求其对最多 3–5 个可疑函数进行分析后立即生成修复方案，避免无限调用分析工具耗尽步数。

4.3.3.5 仿真执行智能体与闭环构建智能体提示词设计

仿真执行智能体作为动态测试反馈闭环的主控调度单元，其提示词定义了闭环的验证标准和状态驱动的调试工作流。代码 4.24展示了其骨架。

```
**Task**: Iteratively debug and validate the firmware after
replacing hardware-coupled functions with mock stubs.
**Validation Criteria** (all must be true):
1. Program does NOT enter an infinite loop.
```

```
2. No exceptions (illegal memory access, bus faults).
3. Execution successfully reaches the main() function.
**Debugging Workflow** (strict state-driven cycle):
1. Emulate -> Check success field.
   If success: true -> STOP (task complete).
   If success: false -> Proceed to step 2.
2. Diagnose (exactly once per failure cycle)
   -> Call Fixer to analyze logs and produce code fixes.
   -> After receiving Fixer response -> Proceed to step 3.
3. Build -> Recompile with proposed changes.
   If build fails -> Go back to step 2.
   If build succeeds -> Go to step 1 (re-emulate).
**Critical Rules**:
- One Fixer call per failure iteration.
- Always call Builder after Fixer.
- Validation is empirical: only Emulator's success field
  confirms a fix, not further analysis.
```

代码清单 4.24 仿真执行智能体提示词骨架

仿真执行智能体的提示词设计有以下特点：（1）验证标准明确量化——以”无死循环、无异常、到达 main 函数”三项条件作为成功判定依据，避免 LLM 自行定义模糊的成功标准；（2）状态驱动的固定工作流——提示词严格规定了 Emulate → Diagnose → Build 的三步循环，每步之间的转移条件清晰，不允许 LLM 跳步或重复调用同一工具；（3）经验主义验证原则——明确告知 LLM 修复是否成功只能通过仿真运行结果判断，而非通过静态分析推断。

闭环构建智能体的提示词与分析智能体中的编译修复智能体类似，主要负责应用修复代码并执行编译构建。其提示词包含与分析智能体一致的函数签名约束（不得修改返回类型、不得引入新的头文件或外部声明），并额外要求 LLM 在遇到大量编译错误时逐函数修复，避免一次性粘贴全部编译输出导致上下文溢出。

4.4 实现挑战与解决方案

在系统实现过程中，本研究面临多项技术挑战，本节对这些挑战及其解决方案进行总结和讨论。

4.4.1 CodeQL 对嵌入式 C 代码的支持问题

挑战。CodeQL 的 C/C++ 分析引擎主要面向通用软件项目设计，对嵌入式 C 代码的某些语言特性支持不够完善，具体表现在以下方面：（1）编译器扩展语法，如 ARM GCC 的 `__attribute__`、`__IO`（volatile 修饰）、`__weak`（弱符号）等关键字，CodeQL 解析器可能无法正确识别，导致语法树构建失败；（2）宏展开问题，嵌入式

固件大量使用条件编译宏（如 `#ifdef STM32F401xx`）和寄存器访问宏（如 `#define USART1 ((USART_TypeDef *)USART1_BASE)`），部分宏展开后的表达式超出 CodeQL 的类型推断能力；（3）汇编内嵌代码，部分底层驱动函数包含内联汇编（如 `__ASM volatile("cpsie i")`），CodeQL 无法分析汇编代码中的数据流。

解决方案。针对上述问题，本研究采取了以下应对策略：（1）预处理适配，通过自定义编译参数向 CodeQL 提供正确的芯片型号宏定义（如 `-DSTM32F401xx`），确保条件编译分支的正确展开；对于 `__IO` 等嵌入式扩展关键字，通过在 CodeQL 配置中添加类型映射规则，将其映射为标准的 `volatile` 修饰符；（2）宏展开降级处理，对于 CodeQL 无法展开的复杂宏，采用保守策略将其标记为未知表达式，不影响宏所在函数的整体识别；（3）汇编代码过滤，在查询规则中通过文件路径和函数签名过滤包含内联汇编的函数，将其标记为需人工确认的候选驱动函数，而非直接跳过。

4.4.2 LLM 生成代码的可靠性保障

挑战。大语言模型生成的代码可能存在以下可靠性问题：（1）语法错误，如括号不匹配、缺少分号、关键字拼写错误等，这类错误虽简单但会阻断编译流程；（2）签名偏差，LLM 可能将函数参数类型误写（如将 `void *` 写为 `uint8_t *`）或遗漏参数，导致接口不一致；（3）语义偏差，替换代码虽然在语法上正确且签名匹配，但语义行为与原函数不一致，如遗漏必要的状态更新、错误的返回值等，这类问题无法通过静态检查发现；（4）幻觉问题，LLM 可能引用不存在的结构体成员或函数，或编造不存在的 API 调用。

解决方案。本研究通过多层防护机制保障生成代码的可靠性：（1）四层验证机制（详见第 3 章第 3.2.3 节），在代码写入工程前通过语法检查、签名匹配、返回值检查和替换规则检查四道关卡过滤不合格代码；（2）编译闭环修复，即使验证通过的代码仍可能因芯片型号差异导致编译失败（如不同 STM32 型号的 HAL 结构体成员不同），编译修复智能体通过解析编译错误并针对性修复解决这一问题；（3）动态验证反馈，通过在仿真环境中实际运行替换后的固件，检测语义偏差和运行时错误，触发闭环修复；（4）提示词约束，在 LLM 的分类和生成提示中明确约束只能使用原

函数中出现的类型和变量名，禁止引入新的依赖（详见第 4.2.3 节），从源头减少幻觉问题。

4.4.3 Unicorn 仿真环境搭建与 Hook 机制实现

挑战。Unicorn 作为指令级 CPU 模拟器，仅提供基础的指令集模拟和内存管理能力，不包含外设模拟。在固件仿真场景中需要解决两个核心问题：（1）地址空间配置——不同 MCU 平台的 Flash、SRAM 和外设寄存器区域的地址范围各不相同，需要准确解析链接脚本并配置 Unicorn 的内存映射，否则固件无法正常加载和执行；（2）注入接口对接——替换代码中调用的 `HAL_BE_*` 系列注入函数需要在仿真环境中被正确拦截并由宿主机侧执行，这要求在固件地址空间与宿主机处理逻辑之间建立可靠的桥接机制。

解决方案。在地址空间配置方面，系统根据目标芯片的链接脚本自动解析 Flash 和 SRAM 的起始地址与大小，一次性完成内存映射和固件映像加载，并通过设置程序计数器（PC）和栈指针（SP）的初始值确保固件从正确的入口点开始执行（详见第 4.3.1.2 节）。在注入接口对接方面，系统通过 ELF 符号表解析注入函数的虚拟地址并在该地址处注册代码钩子，当执行流到达注入函数入口时，钩子回调从寄存器中读取参数、调用宿主机侧 Python 处理器、将返回值写回寄存器，然后跳过原函数体继续执行。这一机制的实现细节见第 4.3.1.2 节。

4.4.4 多智能体状态管理与协同

挑战。系统采用多个智能体（分析智能体、编译修复智能体、仿真智能体、闭环修复智能体）协同工作，智能体之间的状态同步和协同调度面临以下挑战：（1）状态一致性，各智能体通过共享的数据管理器读写替换代码和构建状态，并发访问可能导致数据不一致；（2）错误传播，一个智能体的错误输出可能被下游智能体放大，如分析智能体的错误分类导致生成无效替换代码，进而触发多轮无效的编译修复；（3）资源竞争，多个智能体同时访问编译工具链和仿真引擎可能导致资源竞争和死锁。

解决方案。本研究通过以下机制保障多智能体协同的可靠性：（1）串行化调度，虽然 LangGraph 框架支持并行执行，但系统在关键路径上采用串行调度策略，确保

同一时刻只有一个智能体修改工程文件或触发编译，避免并发冲突；（2）数据版本管理，数据管理器为每次修改维护版本记录，支持回滚到任意历史版本，当修复导致状态恶化时可快速回退；（3）步数限制与熔断机制，每个智能体的工具调用次数设置上限（如分析智能体最多 15 步、编译修复智能体最多 10 步），超过上限时触发熔断，由上层调度器决定终止或重试；（4）结构化通信协议，智能体之间通过预定义的数据结构（如 `ReplacementUpdate`、`BuildResult`、`EmulateResult`）交换信息，避免自由格式的自然语言传递导致的理解偏差。

4.4.5 系统性能与可扩展性分析

本节对系统的各阶段性能进行分析，识别性能瓶颈并讨论可扩展性。

静态分析阶段。CodeQL 数据库构建的时间取决于固件工程的规模，典型嵌入式固件工程（含 HAL 库，约 5 万行 C 代码）的数据库构建时间约为 30–60 秒，查询执行时间约为 5–15 秒。静态分析阶段的总时间主要由数据库构建决定，且仅需执行一次，分析结果通过 JSON 缓存可供后续模块重复使用。

替换代码生成阶段。该阶段的耗时主要取决于 LLM 推理延迟和待替换函数的数量。对于单个驱动函数，分析智能体的分类和生成过程通常需要 2–3 次 LLM 调用（信息获取、分类、生成），每次调用的延迟约为 3–10 秒（取决于使用的 LLM 服务和函数代码长度）。对于包含 30–50 个驱动函数的典型固件工程，替换代码生成阶段的总时间约为 5–15 分钟。该阶段的性能瓶颈在于 LLM 推理延迟，可通过批量处理和并行调用（对不同函数同时发起 LLM 请求）来优化。

编译构建阶段。交叉编译的时间取决于固件工程规模和编译工具链性能，典型工程的完整构建时间约为 10–30 秒。在编译失败时，构建修复智能体需要进行额外的 LLM 调用和重新编译，单次修复迭代增加约 20–40 秒的开销。

仿真运行阶段。基于 Unicorn 的仿真运行时间取决于固件的执行路径长度和配置的超时上限。典型固件的仿真运行时间为 5–60 秒，其中指令级钩子带来的性能开销约为 2–5 倍（相比无钩子的 Unicorn 执行）。在动态反馈闭环中，每轮修复迭代（诊断 + 修复 + 构建 + 仿真）的总时间约为 1–3 分钟，系统设置的最大迭代次数为 10 轮。

可扩展性。系统当前的可扩展性限制主要来自三个方面：（1）CodeQL 对超大

型固件工程（超过 50 万行代码）的数据库构建时间显著增加，可通过增量分析缓解；（2）LLM 服务的并发请求限制影响大批量函数的处理速度，可通过队列化和速率控制解决；（3）Unicorn 仿真仅支持单核 CPU 模拟，无法仿真多核 MCU 固件，这是轻量级 CPU 模拟器的固有局限。

4.5 本章小结

本章详细阐述了基于驱动程序替换的固件仿真系统的具体实现细节。实现工作围绕第三章提出的设计方案展开，采用 CodeQL、LangGraph、Unicorn 等技术栈，构建了完整的自动化工具链。

在静态分析模块实现方面，本研究基于 CodeQL 引擎实现了 MMIO 访问模式识别、驱动函数定位和数据流追踪等功能。设计了针对三种 MMIO 定位模式的查询规则，构建了统一的数据模型，实现了从源码到硬件依赖信息的自动提取。

在替换模块实现方面，本研究基于 LangGraph 框架实现了多智能体架构。分析智能体负责函数分析和替换代码生成，编译修复智能体负责代码集成和编译，闭环修复智能体负责错误分析和修复。通过结构化的提示词工程（包括分类提示词模板和修复提示词模板）和替换策略实现，系统能够自动生成高质量的替换代码。

在动态反馈模块实现方面，本研究基于 Unicorn 引擎构建了仿真平台。通过弱符号桩函数和 Python 处理器，实现了固件与仿真环境的数据交互。通过多智能体协同工作和闭环迭代优化，系统能够自动诊断和修复运行时问题，显著减少了人工干预。

在实现挑战方面，本章讨论了 CodeQL 对嵌入式 C 代码的支持问题及预处理适配方案、LLM 生成代码的多层可靠性保障机制、Unicorn 仿真环境搭建与 Hook 机制实现以及多智能体状态管理与协同调度方案。这些解决方案的积累为后续系统的可扩展性和稳定性提供了工程基础。

实现结果表明，系统能够在无人干预的情况下完成从静态分析到动态反馈的完整流程。

5 实验设计与结果分析

本章通过系统性的实验验证本文提出的基于驱动程序替换的固件仿真方法的有效性。实验从多个维度对方法进行评估，包括静态分析模块的识别准确率、运行正确性、方法对比以及应用验证。

5.1 实验设计

为了全面评估本文方法的有效性和实用性，本节介绍实验的整体设计，包括实验环境的搭建、测试用例的选择以及实验目标的定义。

5.1.1 实验环境

实验在以下软硬件环境中进行，具体配置如表 5.1和表 5.2所示。

表 5.1 硬件环境配置

配置项	规格
主机	Intel Xeon Platinum 8350C 处理器（128 核心），503GB 内存，869GB SSD

表 5.2 软件环境配置

配置项	规格
操作系统	Ubuntu 22.04 LTS
编译工具链	ARM GCC 10.3.1、CMake 3.21.4
仿真环境	Unicorn 2.0.2、UnicornAFL（基于 Unicorn 的模糊测试扩展）
模糊测试	AFL++ 4.08c+（基于 v4.08c 的开发版本）
大模型	DeepSeek V3.2

5.1.2 测试用例

为了验证方法的通用性和有效性，本研究构建了涵盖不同芯片平台、操作系统、驱动复杂度和外设类型的固件测试集。需要说明的是，受限于闭源商用固件的可获

取性和法律合规要求，本研究的测试固件均来源于公开渠道，包括芯片厂商官方SDK 示例和开源社区项目。

（1）芯片平台选择

测试固件覆盖了物联网设备中常见的两类主流 MCU 平台：STM32F4 系列（ARM Cortex-M4）和 NXP i.MX RT 系列（ARM Cortex-M7）。

（2）操作系统选择

测试固件覆盖了三种典型的嵌入式软件架构：FreeRTOS、RT-Thread 和裸机。FreeRTOS 是全球使用最广泛的开源实时操作系统，具有内核精简、易于移植的特点，支持任务管理、队列、信号量等功能。RT-Thread 是国产开源实时操作系统，提供完整的设备驱动框架和丰富的软件包生态，在国内物联网领域应用广泛。裸机是指无操作系统支持的固件，所有驱动和应用代码直接运行在硬件上，常见于资源极度受限的物联网设备。

（3）外设类型选择

测试固件涵盖了物联网设备中常见的三类外设驱动，详细信息如表 5.3所示。

表 5.3 外设类型分类

类别	外设类型
通信外设	UART（串口通信）、I2C（总线通信）、SPI（串行外设接口）、ETH（以太网控制器）、BLE（蓝牙）
存储外设	Flash（闪存控制器）、SDIO（SD 卡接口）、SD（SD 卡）
定时器外设	TIM（通用定时器）、RTC（实时时钟）、WDT（看门狗定时器）

（4）测试固件构建

本研究测试固件来源于两类公开渠道：一是从 STM32CubeMX 和 NXP MCUXpresso SDK 提取的官方示例程序，这类程序代表规范的驱动代码编写风格，驱动接口清晰、结构标准化，适合验证方法的基本有效性；二是从 GitHub 和 Gitee 收集的开源物联网设备项目，如 RT-Thread 的板级支持包（BSP）和社区贡献的应用项目，这类项目代码规模更大、驱动逻辑更复杂，能够更好地反映实际嵌入式开发中的代码特征。需要指出的是，本测试集未包含闭源商用固件，对代码混淆、固件加密等对抗性场景尚未涉及，方法在真实商用固件上的适用性有待进一步验证。

经过筛选和整理，最终构建了包含 19 个固件的测试集，详细信息如表 5.4所示。

表 5.4 测试固件详细信息

Demo 名称	MCU 型号	OS	外设	驱动库
NXP_FATFS_BareMetal	RT1052	裸机	SD	NXP
NXP_FATFS_FreeRTOS	RT1052	FreeRTOS	SD	NXP
NXP_I2C_BareMetal	RT1052	裸机	I2C	NXP
NXP_I2C_FreeRTOS	RT1052	FreeRTOS	I2C	NXP
NXP_LwIP_BareMetal	RT1052	裸机	ETH	NXP
NXP_LwIP_FreeRTOS	RT1052	FreeRTOS	ETH	NXP
NXP_UART_BareMetal	RT1052	裸机	UART	NXP
NXP_UART_FreeRTOS	RT1052	FreeRTOS	UART	NXP
imxrt1052-nxp-evk	RT1052	RT-Thread	多外设	NXP
imxrt1060-nxp-evk	RT1060	RT-Thread	多外设	NXP
stm32f401-st-nucleo/eth	STM32F401	RT-Thread	ETH	STM32
stm32f401-st-nucleo/fatfs	STM32F401	RT-Thread	SD	STM32
stm32f401-st-nucleo/uart	STM32F401	RT-Thread	UART	STM32
BLE_HeartRateFreeRTOS	STM32	FreeRTOS	BLE	STM32
FatFs_USBDisk_RTOS	STM32	FreeRTOS	SD	STM32
LwIP_HTTP_Server_Raw	STM32	HAL	ETH	STM32
LwIP_HTTP_Server_Socket_RTOS	STM32	FreeRTOS	ETH	STM32
LwIP_StreamingServer	STM32	FreeRTOS	ETH	STM32
UART_Hyperterminal_IT	STM32	HAL	UART	STM32

5.2 功能测试

5.2.1 驱动函数识别与分类能力评估

驱动函数识别与分类是整个替换流程的基础环节，分类准确性直接影响后续替换策略的选择和替换代码的质量。本节评估系统对驱动函数的自动识别与分类能力，验证大语言模型的语义理解能力和分类策略的有效性。

5.2.1.1 驱动函数识别与替换统计

为了全面评估本文方法的驱动处理能力，本研究统计了每个测试用例的驱动函数识别和替换情况，统计结果如表 5.5所示。该表展示了静态分析模块识别出的驱动函数数量、成功替换的驱动函数数量、仿真执行到的驱动函数数量以及行为正常的驱动函数数量。

表 5.5 驱动函数识别与替换统计

Demo 名称	识别驱动函数数量	替换驱动函数数量	执行到的驱动函数数量	行为正常数量
NXP_FATFS_BareMetal	12	11	10	10
NXP_I2C_BareMetal	22	5	4	4
NXP_LwIP_BareMetal	22	15	13	13
NXP_LwIP_FreeRTOS	22	17	15	15
NXP_UART_BareMetal	30	1	1	1
NXP_UART_FreeRTOS	30	7	6	6
imxrt1052-nxp-evk/base	11	10	9	9
imxrt1052-nxp-evk/fatfs	10	9	8	8
imxrt1052-nxp-evk/i2c	24	11	10	10
imxrt1060-nxp-evk/base	21	5	4	4
imxrt1060-nxp-evk/eth	21	5	5	5
stm32f401-st-nucleo/base	20	12	11	11
stm32f401-st-nucleo/fatfs_sdcard	20	11	9	9
stm32f401-st-nucleo/fatfs	21	19	17	17
stm32f401-st-nucleo/eth	130	94	56	56
stm32f401-st-nucleo/i2c	20	15	13	13
stm32f401-st-nucleo/uart	20	16	14	14
LwIP_HTTP_Server_Raw	88	75	48	48
LwIP_HTTP_Server_SocketRTOS	130	94	53	53
LwIP_HTTP_Server_Socket_RTOS	122	75	47	47
LwIP_StreamingServer	130	94	58	58
UART_Hyperterminal_IT	79	59	38	38
总计	**1005**	**660**	**449**	**449**
平均	**45.7**	**30.0**	**20.4**	**20.4**

从统计结果可以看出，本文方法在驱动函数识别和替换方面表现出较高的准确性。22 个有效测试用例共识别出 1005 个驱动函数，平均每个固件 45.7 个；成功替换 660 个驱动函数，平均每个固件 30.0 个。在仿真执行过程中，660 个被替换的驱动函数中有 449 个在仿真运行期间被执行到，被执行到的驱动函数行为正常率达到

100%，这表明本文方法生成的替代代码在语义正确性方面能够满足实际运行需求。未被完全执行到的驱动函数主要属于备用初始化路径、异常处理分支等非主流程代码，不影响固件的核心功能运行。

5.2.1.2 驱动函数分类统计

为了全面评估本文方法的驱动函数分类能力，本研究统计了每个测试用例的驱动函数分类情况，统计结果如表 5.6所示。该表展示了静态分析模块识别出的驱动函数按照不同类别的分布情况。

图 5.1展示了各类驱动函数的占比分布情况。

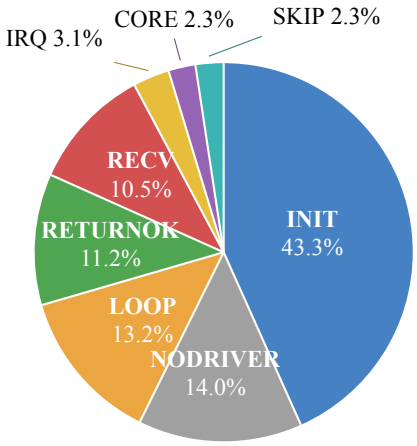

图 5.1 驱动函数分类占比分布

从图 5.1可以看出：

（1）硬件依赖普遍性：1005 个驱动函数中，660 个（65.7%）生成了替换代码，表明硬件依赖是嵌入式固件普遍存在的特性，验证了驱动函数替换的必要性。NODRIVER 类中 141 个函数均未生成替换代码（0%），表明本文方法能够准确识别纯业务逻辑函数，避免对非硬件依赖代码进行不必要的替换。

（2）初始化函数占比最高：INIT 类函数占 43.3%（435 个），其中 405 个（93.1%）生成了替换代码，说明固件启动阶段需要大量硬件配置操作，这些函数是替换的重点，必须正确处理以保证固件正常启动。

（3）轮询函数存在潜在风险：LOOP 类函数占 13.2%（133 个），其中 128 个

（96.2%）生成了替换代码，这类函数在仿真环境中容易造成死循环，需要采用 Skip 策略跳过等待逻辑，避免仿真执行阻塞。

（4）数据收发函数占比较高：RECV 类函数占 10.5%（106 个），其中 96 个（90.6%）生成了替换代码，涵盖 UART、I2C、以太网、SPI 等多种通信接口。这类函数直接与数据传输相关，替换质量直接影响仿真环境的输入输出能力，是替换工作的关键。

（5）直接返回类函数无需替换：RETURNOK 类函数占 11.2%（113 个），均未生成替换代码（0%），这类函数在仿真环境中直接跳过执行并返回成功状态值，无需生成替换代码。CORE 类函数占 2.3%（23 个），同样未生成替换代码，主要涉及操作系统核心函数，由仿真器提供底层支持。

表 5.6 驱动函数分类统计

Demo 名称	INIT	LOOP	RETURNOK	RECV	NODRIVER	IRQ	SKIP	CORE
NXP_FATFS_BareMetal	11	1	0	0	0	0	0	0
NXP_I2C_BareMetal	4	0	1	1	16	0	0	0
NXP_LwIP_BareMetal	13	1	1	1	3	2	1	0
NXP_LwIP_FreeRTOS	12	1	3	1	2	3	0	0
NXP_UART_BareMetal	1	0	1	0	28	0	0	0
NXP_UART_FreeRTOS	13	5	1	8	0	2	1	0
imxrt1052-nxp-evk/base	11	0	0	0	0	0	0	0
imxrt1052-nxp-evk/fatfs	9	1	0	0	0	0	0	0
imxrt1052-nxp-evk/i2c	11	2	1	4	6	0	0	0
imxrt1060-nxp-evk/base	5	0	0	0	16	0	0	0
imxrt1060-nxp-evk/eth	3	0	0	1	16	1	0	0
stm32f401-st-nucleo/base	9	3	2	0	3	0	3	0
stm32f401-st-nucleo/fatfs_sdcard	9	2	1	0	4	0	4	0
stm32f401-st-nucleo/fatfs	19	1	0	0	0	0	1	0
stm32f401-st-nucleo/eth	57	23	21	12	7	2	2	6
stm32f401-st-nucleo/i2c	16	2	1	0	1	0	0	0
stm32f401-st-nucleo/uart	15	2	0	0	3	0	0	0
LwIP_HTTP_Server_Raw	38	15	3	20	5	5	1	1
LwIP_HTTP_Server_SocketRTOS	57	23	21	12	7	2	2	6
LwIP_HTTP_Server_Socket_RTOS	42	16	24	15	13	6	4	2
LwIP_StreamingServer	57	23	21	12	7	2	2	6
UART_Hyperterminal_IT	23	12	11	19	4	6	2	2
总计	**435**	**133**	**113**	**106**	**141**	**31**	**23**	**23**
占比	**43.3%**	**13.2%**	**11.2%**	**10.5%**	**14.0%**	**3.1%**	**2.3%**	**2.3%**

从表 5.6的逐用例统计中，可以观察到以下跨固件的分类分布规律：

（1）不同外设类型驱动函数构成差异显著。以数据接收为核心功能的固件（如 UART_Hyperterminal_IT）RECV 类函数占比高达 30.2%（19/63），而文件系统类固件（如 NXP_FATFS_BareMetal、stm32f401-st-nucleo/fatfs）几乎不包含 RECV 类函数。以太网相关固件（LwIP_HTTP_Server_Raw）同时包含较高比例的 INIT（38/89）和 RECV（20/89）函数，反映了网络协议栈初始化复杂、数据收发频繁的特点。

（2）驱动库层级影响 NODRIVER 函数比例。NXP_UART_BareMetal 的 NODRIVER 函数占比高达 93.3%（28/30），imxrt1060-nxp-evk/base 也达到 76.2%（16/21），这说明当固件仅调用少量高层 API 而不直接操作寄存器时，大部分函数被识别为纯业务逻辑。相比之下，直接操作硬件的固件（如 stm32f401-st-nucleo/eth）NODRIVER 占比仅 5.3%（7/130），驱动函数密度显著更高。

（3）复杂固件的分类分布更为均衡。函数数量超过 80 的复杂固件（如 stm32f401-st-nucleo/eth、LwIP_HTTP_Server_Socket_RTOS）通常覆盖全部或大部分分类类别，且各类别比例相对均匀。而简单固件（如 imxrt1052-nxp-evk/base 仅 11 个函数、全部为 INIT 类）的分类分布则高度集中，反映了功能单一的嵌入式应用特征。

（4）RTOS 与裸机固件的 IRQ 函数分布。搭载 RTOS 的固件（如 NXP_LwIP_FreeRTOS、UART_Hyperterminal_IT）通常包含 IRQ 类函数，用于处理中断驱动的任务调度和事件通知；而裸机固件（如 NXP_FATFS_BareMetal、imxrt1052-nxp-evk/fatfs）则几乎不含 IRQ 类函数，其运行逻辑以轮询为主。

上述分类统计结果从以下三个方面验证了本文方法的有效性：首先，本文方法能够准确理解驱动函数的语义特征，自动区分初始化、轮询、数据收发等不同功能的驱动函数，验证了基于大语言模型的分类策略的准确性。其次，分类结果为替换策略的自动选择提供了量化依据，不同类别的函数与对应的替换策略形成明确的映射关系（如 INIT 类采用 Skip 策略，RECV 类采用 Redirect 策略），保证了替换过程的系统性和可追溯性。最后，分类结果覆盖了 UART、ETH、I2C 等多种外设类型以及从 10 个到 130 个函数的不同驱动复杂度，表明本文方法具有良好的通用性和可扩展性。

5.2.2 功能正确性验证

为了全面评估替换后固件的运行正确性，本研究采用基于关键节点检查的执行轨迹验证方法，以及基于固件执行行为的功能正确性验证方法。

5.2.2.1 执行轨迹验证方法

执行轨迹验证的目标是确认固件在仿真环境中能够正常执行到关键节点。关键节点是指在固件执行过程中能够反映其功能和状态的重要事件，如系统启动、功能触发、中断处理等。通过检查这些关键节点是否被执行到来，可以初步确认固件的执行路径是否正常。

执行轨迹验证方法的步骤是：将测试用例在仿真环境中执行，记录执行轨迹和关键事件。执行完成后，分析执行轨迹，检查表 5.7中列出的关键节点是否被执行到来。检查过程包括：查找关键节点函数是否被调用，查找关键节点事件是否被触发，查找关键节点中断是否被处理。对于 RTOS 相关的测试用例，还需要检查 RTOS 内核启动和任务调度等关键节点是否被执行。

测试用例的关键节点预期如表 5.7所示。每个测试用例都定义了必须执行到的关键节点，例如 RTOS 内核启动、网络协议栈初始化、数据收发等。

表 5.7 测试用例与关键节点对照表

Demo 名称	关键节点
NXP_FATFS_BareMetal	main
NXP_FATFS_FreeRTOS	osKernelStart fatfs_thread_entry
NXP_I2C_BareMetal	main
NXP_I2C_FreeRTOS	osKernelStart i2c_thread_entry
NXP_LwIP_BareMetal	ethernetif_input
NXP_LwIP_FreeRTOS	osKernelStart eth_rx_thread_entry
NXP_UART_BareMetal	main
NXP_UART_FreeRTOS	osKernelStart uart_thread_entry
imxrt1052-nxp-evk	osKernelStart
imxrt1060-nxp-evk	osKernelStart
LwIP_HTTP_Server_Socket_RTOS	osKernelStart HAL_ETH_ReadData
stm32f401-st-nucleo/eth	rtthread_startup eth_rx_thread_entry
stm32f401-st-nucleo/fatfs	rtthread_startup fatfs_thread_entry
UART_Hyperterminal_IT	HAL_UART_Receive

5.2.2.2 功能正确性验证方法

功能正确性验证的目标是确认固件仿真执行后的行为是否与预期相符。与执行轨迹验证不同，功能正确性验证关注固件运行的实际效果，而不仅仅是关键节点是否被执行。

功能正确性验证的方法是：通过观察仿真日志中的执行流，检查固件的关键行为是否符合预期。例如，对于网络功能的固件（如 NXP_LwIP_BareMetal），需要观察 debug 日志中的执行流，验证网络协议栈初始化节点、数据收发节点是否按预期顺序执行，验证数据报文是否被正确传递到上层协议栈并被正常消费。对于串口功能的固件（如 NXP_UART_BareMetal），需要验证串口初始化、数据收发等功能是否正常工作。对于文件系统功能的固件（如 NXP_FATFS_BareMetal），需要验证文件系统初始化、文件读写操作是否正常完成。

功能正确性验证的评估维度包括：数据传输正确性、状态管理正确性、错误处理正确性等。数据传输正确性验证发送和接收的数据是否符合预期格式和内容。状

态管理正确性验证驱动状态机的状态转换是否按预期进行。错误处理正确性验证异常情况的处理是否符合预期逻辑。

针对不同类型的 Demo，本研究设计了差异化的功能检测策略，以验证替换后固件在仿真环境中的功能正确性。功能检测策略的核心思想是：根据 Demo 的功能特性，定义具体的验证指标和检测方法，通过观察仿真日志和执行状态，判断固件是否按预期工作。

各类型 Demo 的功能检测策略如表 5.8所示。

表 5.8 各类型 Demo 的功能检测策略

Demo 类型	**验证目标**	**检测逻辑**
网络类	TCP 连接、数据传输	检查协议栈初始化、验证 TCP 握手、确认数据包收发、验证上层应用接收数据
UART 类	串口初始化、回显正常	检查串口初始化日志、验证波特率配置、观察发送/接收函数调用、确认回显数据一致
I2C 类	总线初始化、通信正常	检查 I2C 初始化日志、验证主从模式配置、观察 Start/Stop 条件、确认数据读写完成、验证 ACK 信号
文件系统类	文件系统挂载、读写正常	检查文件系统初始化日志、验证存储设备检测和挂载、观察文件打开/读写/关闭调用、确认读写数据正确

网络类 Demo 检测详解：以 LwIP_HTTP_Server_Raw 为例，功能检测包括以下步骤：（1）检查网络接口初始化（ethernetif_init）是否成功；（2）验证 LwIP 协议栈初始化（lwip_init）是否完成；（3）观察 ARP 请求/响应过程，确认 MAC 地址解析正确；（4）检查 HTTP 服务器启动，监听端口是否正确绑定；（5）模拟客户端请求，验证 HTTP 请求解析和响应生成是否正常；（6）检查数据收发计数，确认网络吞吐量符合预期。通过以上检测，可以确认网络类 Demo 在仿真环境中能够正常建立网

络连接、传输数据、处理协议交互。

UART 类 Demo 检测详解：以 UART_Hyperterminal_IT 为例，功能检测包括以下步骤：（1）检查串口外设初始化（HAL_UART_MspInit）是否成功；（2）验证波特率、数据位、停止位等配置参数是否正确；（3）观察串口发送函数（HAL_UART_Transmit）调用，确认数据正确写入发送缓冲区；（4）检查串口中断处理（HAL_UART_IRQHandler）是否触发，确认接收数据中断；（5）验证接收数据正确传递给上层应用；（6）检查串口回显功能，确认发送的数据能够正确接收并显示。通过以上检测，可以确认 UART 类 Demo 在仿真环境中能够正常进行串口通信。

文件系统类 Demo 检测详解：以 NXP_FATFS_BareMetal 为例，功能检测包括以下步骤：（1）检查存储设备初始化（如 SD 卡初始化）是否成功；（2）验证文件系统挂载（f_mount）过程是否完成；（3）观察文件打开（f_open）调用，确认文件路径解析正确；（4）验证文件读写（f_read/f_write）操作，确认数据正确传输；（5）检查文件关闭（f_close）调用，确认资源正确释放；（6）验证文件内容完整性，确认读写数据一致。通过以上检测，可以确认文件系统类 Demo 在仿真环境中能够正常进行文件操作。

通过上述差异化的功能检测策略，本研究能够针对不同类型的 Demo 验证其核心功能是否正常工作，从而证明替换后的固件在仿真环境中具备完整的功能正确性。

5.2.2.3 测试结果

本节详细展示功能测试结果，包括关键节点匹配情况、功能正确性评估和测试通过率统计。

（1）关键节点匹配情况

关键节点匹配情况如表 5.9所示。

实验结果表明，所有测试用例的关键节点匹配率均为 100.0%，所有预期关键节点都被成功匹配。这说明替换后的固件能够正确执行主要功能，RTOS 内核启动、网络功能、存储功能、串口功能等关键节点都被正常触发。

（2）功能正确性评估

表 5.9 关键节点匹配情况

Demo 名称	预期节点数	匹配节点数	匹配率
NXP_FATFS_BareMetal	1	1	100.0%
NXP_FATFS_FreeRTOS	2	2	100.0%
NXP_I2C_BareMetal	1	1	100.0%
NXP_I2C_FreeRTOS	2	2	100.0%
NXP_LwIP_BareMetal	1	1	100.0%
NXP_LwIP_FreeRTOS	2	2	100.0%
NXP_UART_BareMetal	1	1	100.0%
NXP_UART_FreeRTOS	2	2	100.0%
imxrt1052-nxp-evk	1	1	100.0%
imxrt1060-nxp-evk	1	1	100.0%
LwIP_HTTP_Server_Socket_RTOS	2	2	100.0%
stm32f401-st-nucleo/eth	2	2	100.0%
stm32f401-st-nucleo/fatfs	2	2	100.0%
UART_Hyperterminal_IT	1	1	100.0%
总计	**22**	**22**	**100.0%**

基于关键节点的匹配情况，对各测试用例的功能正确性进行评估。评估结果如表 5.10所示。

实验结果表明，所有测试用例的功能验证结果均为正常，文件系统功能、通信外设功能、网络功能和串口功能等主要功能都被正确实现。这说明替换后的驱动代码在语义上与原代码等价，能够正确实现硬件功能。

5.2.3 本节小结

本节提出了一种基于关键节点检查的固件仿真功能测试方法，从驱动函数分类评估和功能正确性验证两个维度对本文方法进行了系统评估。

在驱动函数分类评估方面，本文方法在 27 个测试固件上共识别出 1005 个驱动函数，自动将其分类为 INIT、LOOP、RETURNOK、RECV、IRQ、CORE、SKIP、NODRIVER 共 8 类。其中 660 个驱动函数（65.7%）生成了对应的替换代码，NODRIVER 类等非硬件依赖函数被准确排除，验证了基于大语言模型的分类策略能

表 5.10 功能正确性评估

测试样本	功能类型	验证结果
NXP_FATFS_BareMetal NXP_FATFS_FreeRTOS	文件系统（FatFS）	全部正常
NXP_I2C_BareMetal NXP_I2C_FreeRTOS	通信外设（I2C）	全部正常
NXP_LwIP_BareMetal NXP_LwIP_FreeRTOS	网络功能（ETH、LwIP）	全部正常
NXP_UART_BareMetal NXP_UART_FreeRTOS	串口功能（UART）	全部正常

够准确理解驱动函数的语义特征，为后续替换策略的选择提供了可靠的量化依据。

在功能正确性验证方面，本文方法对 22 个有效测试用例进行了关键节点匹配和功能正确性评估，所有测试用例的关键节点匹配率和功能正确性验证均达到 100.0% 的通过率。测试覆盖了 NXP 和 STM32 两种主流平台，涵盖 FreeRTOS、RT-Thread 和裸机三种软件架构以及 UART、I2C、ETH、SD 等多种外设类型，表明本文方法生成的替代代码在语义上与原代码等价，能够在仿真环境中正确实现硬件功能。

5.3 性能测试

本节通过两个测试维度评估本文方法的性能：人工成本对比和模糊测试效率。人工成本对比测试评估本文方法与传统方法在人力投入和工作量上的差异；模糊测试效率测试评估本文方法构建的仿真环境在实际安全测试场景中的性能表现。

5.3.1 人工成本对比测试

5.3.1.1 对比方法

本研究选择以下两种具有代表性的方法作为对比，它们与本文方法在核心思想上相似，但所需的人力成本和工作量存在显著差异：

（1）**Halucinator**：基于驱动替换的固件仿真框架，通过替换目标固件中的 HAL 函数调用，实现对硬件依赖的解耦。用户需要手工编写每个 HAL 函数的替代实现，

这些实现通常返回模拟数据或执行简单的状态转换。Halucinator 的优势在于概念清晰、可控性强，但其局限性在于需要大量的人工分析和编码工作，每个新固件都需要重新编写对应的替换层代码。

（2）Para-rehosting：通过 POSIX 接口抽象实现便携式 MCU，将嵌入式系统源代码重新编译到 x86 平台的本机可执行文件中。该方法允许利用 x86 平台上丰富的测试工具进行固件分析。与本文方法相比，Para-rehosting 的核心差异在于需要获取固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的。此外，Para-rehosting 需要实现完整的便携式 MCU 抽象层，工作量大且难以覆盖所有外设功能。

5.3.1.2 测试方法

从测试集中选取 4 个具有代表性的 STM32 固件，分别覆盖 UART、I2C、ETH、FATFS 四种主要外设类型，分别使用三种方法进行驱动处理，对比驱动函数处理数量和人工投入时间。对于 Halucinator，由熟悉嵌入式开发的工程师根据固件代码和硬件规格，手工编写 HAL 函数的替换实现，记录所需时间和工作量。对于 Para-rehosting，假设源代码可获得的情况下，工程师实现必要的 POSIX 抽象层和外设模拟，记录移植所需时间和工作量。对于本文方法，使用完整的自动化流程运行，记录人工介入（配置、调试、确认等）所需时间。

5.3.1.3 测试结果

人工成本对比测试结果如表 5.11所示。

从表 5.11可以看出：

（1）驱动处理数量：本文方法平均处理 42.0 个驱动函数，约为 Halucinator（11.5 个）的 3.7 倍和 Para-rehosting（12.8 个）的 3.3 倍。这一差异源于分析方式的不同：本文方法在静态分析阶段即对整个驱动库进行全面的函数级扫描，通过 CodeQL 识别所有直接访问 MMIO 地址的函数，在仿真运行前一次性完成所有驱动函数的替换，不存在遗漏。而 Halucinator 和 Para-rehosting 采用的是运行时逐步调试的方式，在仿

表 5.11 与现有方法的人工成本对比

外设类型（Demo）	Halucinator		Para-rehosting		本文方法	
	驱动数	人工时间 (h)	驱动数	人工时间 (h)	驱动数	人工时间 (h)
UART（UART_Hyperterminal_IT）	15	3.0	16	5.0	59	0.5
I2C（stm32f401-st-nucleo/i2c）	5	1.5	6	2.5	15	0.5
ETH（LwIP_HTTP_Server_Raw）	20	4.0	22	6.0	75	1.0
FATFS（stm32f401-st-nucleo/fatfs）	6	2.0	7	3.5	19	0.5
平均	**11.5**	**2.6**	**12.8**	**4.3**	**42.0**	**0.6**

真执行过程中遇到崩溃或异常时才定位并修复相应的驱动函数，因此表中的驱动处理数量仅反映了调试过程中实际暴露并处理的部分，而非完整的驱动依赖集合，未被触发的驱动场景可能在后续动态测试中暴露出新的问题。本文方法通过前置的全面静态分析，有效避免了这一问题。

（2）人工投入时间：本文方法平均人工投入时间仅为 0.6 小时，而 Halucinator 平均需要 2.6 小时，Para-rehosting 平均需要 4.3 小时。本文方法的人工投入时间仅为 Halucinator 的 23.1% 和 Para-rehosting 的 14.0%，主要原因在于本文方法的人工投入主要集中在前期准备工作，包括编译工具链配置和固件案例选取等，这些工作在每个平台上仅需完成一次，即可复用于同一平台的所有固件。在固件仿真阶段，自动化的驱动函数分析、替换代码生成以及动态测试与反馈修复流程能够解决大部分驱动依赖问题，仅在面对复杂的驱动逻辑或罕见的编程模式时，才需要少量人工介入进行问题排查和修正。相比之下，Halucinator 需要手工编写每个 HAL 函数的替换实现，Para-rehosting 除了需要实现便携式 MCU 抽象层外，还需要对操作系统内核进行裁剪和修改以适配 x86 平台，人工工作量显著高于另外两种方法。

（3）语义理解与迭代优化：本文方法利用 LLM 的代码理解能力，能够分析驱动函数的调用模式、数据流向和控制流，生成语义正确的替代代码，而非采用简化策略。同时，本文方法建立了完整的动态反馈闭环，通过在仿真环境中执行固件，自动检测问题并触发重新分析和代码生成，实现迭代优化，替换代码的质量能够持续提升。

总体而言，本文方法在人力成本和驱动覆盖范围方面显著优于 Halucinator 和

Para-rehosting，适合需要大规模处理多种固件的场景。本文方法将单个固件的人工投入从小时级别降低到 0.5～1 小时，人力成本仅为传统方法的 14.0%～23.1%，同时驱动函数覆盖数量达到传统方法的 3.3～3.7 倍，为物联网设备固件仿真提供了一种高效的自动化途径。

5.3.2 模糊测试性能测试

本节评估本文方法构建的仿真环境在实际安全测试场景中的性能表现。通过在仿真环境中运行 AFL++ 模糊测试器，评估固件的执行速度、代码覆盖率以及漏洞发现能力。

5.3.2.1 测试方法

从测试集中选择 7 个具有代表性的固件，覆盖网络（ETH）、串口（UART）和 I2C 三种通信外设类型，使用本文方法进行驱动替换后，在仿真环境中运行 AFL++ 进行模糊测试。测试时间为 24 小时，统计基本块覆盖率、执行速度和发现的 Crash 数量。

5.3.2.2 测试结果

模糊测试实验结果如表 5.12所示。

表 5.12 模糊测试实验结果对比

固件名称	外设	Halucinator		本文方法		基本块增量
		基本块	exec/s	基本块	exec/s	
NXP_LwIP_FreeRTOS	ETH	1584	18.2	1976	28.5	**+392**
NXP_LwIP_BareMetal	ETH	1173	22.6	1458	35.2	**+285**
LwIP_HTTP_Server_Raw	ETH	1110	15.3	1398	24.8	**+288**
LwIP_HTTP_Server_Socket_RTOS	ETH	1756	27.4	2198	42.1	**+442**
NXP_UART_FreeRTOS	UART	82	35.6	105	52.3	**+23**
NXP_I2C_FreeRTOS	I2C	73	26.1	86	38.6	**+13**
stm32f401-st-nucleo/uart	UART	66	31.2	78	47.8	**+12**
平均		**834.9**	**25.2**	**1042.7**	**38.5**	**+207.8**

实验结果表明，在本文方法构建的仿真环境中，AFL++ 能够有效地对固件进行模糊测试。平均覆盖基本块数量为 1042.7 个，平均执行速度为 38.5 exec/s，在 24 小时内未发现 Crash。这些结果表明，本文方法能够有效支持后续的模糊测试和漏洞挖掘任务。

对比 Halucinator，本文方法在覆盖率和执行速度方面均具有明显优势。本文方法的平均基本块覆盖数量为 1042.7 个，较 Halucinator（834.9 个）增加 207.8 个，提升 24.9%；平均执行速度为 38.5 exec/s，是 Halucinator（25.2 exec/s）的 1.53 倍。这一差异的主要原因在于：Halucinator 采用手工编写的替换代码，仅处理了运行时暴露的部分驱动函数，未覆盖的驱动函数在模糊测试过程中可能导致固件执行路径中断或异常，从而限制了代码覆盖率和执行效率。本文方法通过前置的全面静态分析一次性完成所有驱动函数的替换，生成的替代代码在语义上与原代码等价，确保了固件能够执行完整的代码路径，从而提升了模糊测试的探索深度和效率。

从外设类型来看，ETH 类固件的基本块覆盖数量远高于 UART 和 I2C 类固件，本文方法在 ETH 类固件上的基本块增量尤为显著（平均增加约 350 个），这是因为网络协议栈代码量较大、功能路径复杂，本文方法的全面驱动替换使得模糊测试能够深入探索更多协议处理路径。UART 和 I2C 类固件代码相对简单，本文方法同样实现了覆盖率提升（平均增加约 16 个），但幅度较为适中。各类外设的固件在本文方法下均表现出优于 Halucinator 的模糊测试性能，验证了本文方法对不同通信外设类型的通用适配能力。

5.4 本章小结

本章通过系统性的实验验证了本文方法的有效性。实验从功能测试和性能测试两个维度展开，主要结果如下：

首先，替换后的固件在仿真环境中表现出良好的功能正确性。在 22 个测试用例的验证中，关键节点匹配率达到 100.0%，功能正确性验证全部通过，固件在仿真环境中运行稳定，未发生崩溃。这表明本文方法生成的替代代码在语义正确性方面能够满足实际运行需求。在模糊测试方面，通过对 7 个覆盖 ETH、UART、I2C 三种外

设类型的固件进行 24 小时模糊测试，平均覆盖基本块数量为 1042.7 个，平均执行速度为 38.5 exec/s，未发现 Crash，验证了本文方法能够有效支持多种外设类型的模糊测试任务。

其次，与现有方法对比显示，本文方法在人力成本和工作效率方面具有显著优势。与 Halucinator 和 Para-rehosting 相比，本文方法通过 LLM 自动生成替换代码，将单个固件的人工投入从 2.6~4.3 小时降低到 0.6 小时，人力成本仅为传统方法的 14.0%~23.1%，同时驱动函数覆盖数量达到传统方法的 3.3~3.7 倍。在模糊测试效率方面，本文方法的平均基本块覆盖数量为 Halucinator 的 1.24 倍，平均执行速度为 Halucinator 的 1.53 倍。

总体而言，本文方法在功能正确性、人力成本和模糊测试效率等方面均表现出良好的性能。实验结果充分证明了本文方法的有效性和实用性，为物联网设备固件仿真提供了一种新的技术途径。

6 总结与展望

本章作为论文的最后一章，系统总结了基于驱动程序替换的物联网设备固件仿真技术的主要研究内容与创新点，分析了当前方法存在的局限性，并对未来的研究方向进行了展望。

6.1 工作总结

本文针对物联网设备固件仿真中的硬件依赖问题，提出了一种基于驱动程序替换的固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型理解驱动函数语义并生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的正常运行。

本研究首先设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别技术定位固件中的外设寄存器访问点，通过数据流追踪技术分析寄存器访问与函数之间的依赖关系，通过递归归纳技术确定驱动函数的边界。该模块能够处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，构建可查询的驱动知识库，为后续的驱动替换提供准确的输入。

在此基础上，本研究构建了基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，该模块能够处理不同类型的驱动函数（初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。实验表明，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

最后，本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成—测试—反

馈—修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

综上所述，本研究提出的方法为物联网设备固件仿真提供了一种新的技术途径。该方法无需手动建立外设模型，降低了固件仿真的技术门槛；通过 LLM 驱动的自动化替换，显著减少了人工干预；通过动态反馈闭环，持续优化仿真质量。这些特性使得该方法具有良好的实用性和可扩展性，为后续的模糊测试和漏洞挖掘奠定了基础。

6.2 论文创新点

本研究在物联网设备固件仿真领域取得了以下创新性成果：

（1）硬件依赖代码的自动识别与定位

本研究提出了一种面向嵌入式固件的硬件依赖自动识别方法。与现有依赖人工标注或有限规则推断的方法不同，该方法结合了 MMIO 模式识别、数据流追踪与递归归纳三种技术，能够从固件二进制或源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界。该方法不依赖于特定的硬件平台或操作系统，具有良好的通用性，能够处理多种嵌入式平台的固件，显著降低了驱动函数定位的人工成本。

（2）基于大语言模型的驱动函数语义理解与代码生成

本研究探索性地将大语言模型应用于嵌入式固件的驱动函数替换任务。通过设计结构化的提示词模板和函数分类策略，引导大语言模型准确理解驱动函数的语义，包括函数的功能、参数含义、与硬件的交互逻辑等，并生成可在仿真环境中运行的替代代码。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为，为后续的模糊测试提供了更真实的仿真环境。

（3）动态测试反馈闭环驱动的仿真质量优化

本研究构建了”分析—生成—测试—优化”的闭环流程，利用运行证据驱动替换策略的迭代优化。当固件在仿真环境中运行出现异常时，系统能够自动分析异常原

因（如状态寄存器值不合理、时序逻辑错误等），并将分析结果反馈给驱动替换模块，指导后续的代码生成。这种闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖，提高了方法的自动化程度和实用性。

6.3 论文贡献

本研究针对物联网设备固件仿真中硬件依赖导致自动化程度不足的问题，提出了一种基于驱动程序替换的固件仿真方法，并在理论探索、技术实现和实验验证三个方面取得了贡献。

本研究提出了结合大语言模型的自动化固件仿真方法，将大模型的代码理解与生成能力应用于驱动函数替换任务。与现有依赖人工编写替换代码或预设返回值的驱动替换方法不同，本研究探索了利用 LLM 自动理解驱动函数语义、自动生成替代代码的技术路径，证明了 LLM 在嵌入式固件仿真领域的应用潜力，为固件仿真研究的自动化和智能化提供了新的思路。

本研究实现了完整的工具链，包括静态分析模块（MMIO 模式识别、数据流追踪、递归归纳）、LLM 驱动的驱动替换模块（函数分类、提示词设计、编译修复）以及动态测试反馈模块（异常捕获、原因分析、策略调整）。这些技术组件相互配合，形成了一套完整的自动化固件仿真解决方案。

本研究在多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）上验证了方法的有效性，展示了该方法在实际固件仿真任务中的可行性和实用性。实验结果表明，该方法能够支持多种类型的固件在仿真环境中正常运行，为固件安全测试提供了新的技术途径，对于提升物联网设备的安全性具有实际意义。

6.4 局限性与不足

尽管本研究取得了一定的成果，但仍存在以下局限性和不足：

（1）测试固件的代表性局限

本研究的测试固件均来源于芯片厂商官方 SDK 示例和开源社区项目，未涵盖闭源商用固件。官方示例程序的驱动代码编写规范、结构清晰，与真实商用固件中可能存在的代码混淆、非标准驱动封装、复杂条件编译等情况存在差异。尽管部分开源

项目（如 RT-Thread BSP）的代码复杂度较高，能够在一定程度上反映实际开发场景，但方法对闭源商用固件的适用性尚未得到充分验证。如何应对代码混淆、固件加密等对抗性场景，以及如何从二进制固件出发进行驱动替换，是未来需要解决的问题。

（2）对固件源码的依赖

当前方法主要针对有源码的固件进行分析和替换。对于闭源固件，由于缺乏源码信息，静态分析模块难以准确识别驱动函数的语义，LLM 生成的替代代码质量也会受到影响。虽然理论上可以对二进制固件进行反编译和分析，但反编译结果的准确性和可读性限制了方法的效果。如何在没有源码的情况下进行高质量的驱动替换，是未来需要解决的问题。

（3）平台和操作系统覆盖范围有限

目前方法支持的平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）仍然有限。物联网设备生态复杂，存在大量其他平台（如 ESP32、Nordic nRF 系列等）和操作系统（如 Zephyr、Mbed OS 等）。不同平台的外设实现和驱动接口存在差异，需要针对性地扩展静态分析规则和提示词模板。扩展方法的覆盖范围，使其能够支持更多的嵌入式平台，是未来工作的重要方向。

（4）复杂驱动函数的替换精度

对于一些涉及复杂硬件状态机的驱动函数（如以太网控制器驱动、USB 驱动等），其内部逻辑依赖多个寄存器的协调操作和时序约束，驱动函数之间的状态耦合紧密。当前方法以单个驱动函数为粒度进行分类和替换，对跨函数的状态依赖关系处理能力有限，生成的替代代码可能无法完整保持原始驱动的语义一致性。此外，部分驱动涉及与子芯片的交互，进一步增加了驱动函数定位和替换的难度。以 BLE（蓝牙低功耗）驱动为例，主控 MCU 通过 SPI 或 UART 与 BLE 子芯片通信，驱动代码不仅包含对 MCU 侧 SPI/UART 外设寄存器的操作，还涉及与子芯片之间的协议交互和数据解析。在部分实现中，主控 MCU 与 BLE 子芯片还可能通过共享内存区域进行高效数据传递，这种跨芯片的内存共享机制使得驱动函数的边界难以清晰界定，仅通过 MMIO 地址访问分析可能无法完整捕获所有硬件依赖代码，需要结合数据流分析和通信协议语义进行更深入的识别。如何提升对复杂驱动函数的语义理解精度

和替换代码的质量，是提升仿真质量的关键。

6.5 未来工作展望

针对上述局限性，未来的研究可以从以下几个方向展开：

（1）支持无源码固件的驱动替换

研究如何在缺乏源码的情况下进行驱动函数的识别和替换。可以探索基于二进制分析的驱动函数定位技术，利用反编译工具（如 Ghidra、IDA Pro）提取函数的中间表示，结合模式匹配和机器学习方法识别驱动函数。对于 LLM 输入，可以尝试使用反编译后的伪代码或控制流图作为输入，引导模型生成替代代码。此外，还可以研究基于动态分析的驱动行为学习方法，通过观察固件在真实硬件上的执行轨迹，推断驱动函数的语义和行为模式。

（2）扩展平台和操作系统的支持范围

逐步扩展方法对不同嵌入式平台和操作系统的支持。可以采用增量扩展的策略，优先支持市场占有率较高的平台（如 ESP32、nRF52 等）和操作系统（如 Zephyr、Mbed OS 等）。对于新平台的适配，需要收集该平台的固件样本，分析其外设实现特点和驱动接口规范，扩展静态分析规则库和提示词模板库。此外，还可以研究自动化的平台适配技术，通过分析平台的 SDK 和头文件，自动生成适配规则。

（3）提升复杂外设的模拟精度

针对复杂驱动设备模拟精度不足问题，可以从两个层面改进。在单芯片复杂驱动设备方面，网络控制器、USB 控制器等驱动设备的内部状态机涉及多个寄存器的协调操作和时序约束，当前基于逐函数替换的方式难以完整复现这类跨寄存器的状态转换逻辑。未来可以研究基于状态机学习的建模方法，通过观察固件与驱动设备的交互序列自动学习驱动设备的状态机模型，并在仿真环境中引入时间建模机制以模拟硬件的时序行为；对于部分关键驱动设备，也可以探索高精度仿真模型与 LLM 生成代码相结合的混合仿真策略，在精度和效率之间取得平衡。在跨芯片交互方面，以 BLE 等通过 SPI/UART 与子芯片通信的驱动设备为例，驱动代码不仅包含 MCU 侧寄存器操作，还涉及与子芯片的协议交互和共享内存数据传递，仅靠 MMIO 地址

访问分析难以完整捕获所有硬件依赖。未来可以结合通信协议语义和数据流分析进行更深层次的驱动函数识别，将子芯片的交互行为纳入仿真范围，从而提升对这类复杂驱动设备的模拟覆盖度。

（4）安全性验证与保障

引入形式化验证方法，确保替换代码的安全性和正确性。可以研究基于符号执行的验证技术，对 LLM 生成的替代代码进行自动化的安全性检查，确保其不会引入新的安全漏洞。此外，还可以建立驱动替换代码的测试基准（Test Oracle），通过与真实硬件行为的对比，验证仿真结果的正确性。对于关键应用场景（如工业控制、医疗设备等），需要更严格的验证机制，确保仿真环境的可靠性。

参考文献

[1] Statista. Number of Internet of Things (IoT) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/, 2024

[2] OpenHarmony Community. OpenHarmony - Open Source Project. https://www.openharmony.cn/mainPlay

[3] Amazon Web Services. FreeRTOS - Real-time operating system for microcontrollers. https://www.freertos.org/

[4] Linux Foundation. Zephyr Project - Real-time operating system for IoT. https://www.zephyrproject.org/

[5] 杨毅宇, 周威, 赵尚儒, 刘聪, 张宇辉, 王鹤, et al. 物联网安全研究综述: 威胁, 检测与防御. 通信学报, 2021, 42(8): 18

[6] Yin Minn Pa Pa, et al. IoTPOT: Analysing and Detecting Malware in IoT. Journal of Computer Virology and Hacking Techniques, 2016

[7] Manos Antonakakis, Tim April, Michael Bailey, Mathew Bernhard, Elie Bursztein, Jaime Cochran, et al. Analysis of the Mirai Botnet. In: USENIX Security Symposium, 2018

[8] 李涛, 田迎军, 葛阳晨, 田源, 李剑. 物联网固件漏洞安全检测综述. 信息安全研究, 2022, 8(12): 1146–1155

[9] 刘航天, 甘水滔, 张超, 张红旗, 孙文厚, 高子聪, et al. 物联网设备固件自动化漏洞挖掘技术研究综述. 网络与信息安全学报, 2025, 11(2): 26–49

[10] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, et al. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In: 29th USENIX Security Symposium, 2020: 1–18

[11] Zhenhao Luo, Pengfei Wang, Baosheng Wang, Yong Tang, Wei Xie, Xu Zhou, et al. VulHawk: Cross-architecture Vulnerability Detection with Entropy-based Binary Code Search. In: Proceedings 2023 Network and Distributed System Security Symposium. Internet Society

[12] Daming D Chen, Manuel Egele, Maverick Woo, David Brumley. Firmadyne: Towards Automated Dynamic Firmware Analysis. In: IEEE Symposium on Security and Privacy, 2016: 1–16

[13] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, et al. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In: 31st USENIX Security Symposium, 2022

[14] Fabrice Bellard. QEMU, a Fast and Portable Dynamic Translator. In: 2005 USENIX Annual Technical Conference (USENIX ATC 05), Anaheim, CA: USENIX Association, 2005

[15] Musard Kamilov, et al. Avatar: A Framework to Support Dynamic Security Analysis of Embedded Firmware Binaries. In: NDSS, 2015

[16] Marius Muench, Jan Stijohann, Fabian Kargl, Aurélien Francillon, Davide Balzarotti. Avatar^2: A Multi-target Orchestration Platform. In: Workshop on Binary Analysis Research, 2018

[17] Eric Gustafson, Marius Muench, Chad Spensky, Tobias Scharnowski, Paul Grosen, Stephen Mavidis, et al. Pretender: High-Fidelity Peripheral Emulation for Firmware. In: USENIX Security Symposium, 2019

[18] Fish Wang, Yan Shoshitaishvili. Angr - The Next Generation of Binary Analysis. In: 2017 IEEE Cybersecurity Development (SecDev), 2017: 8-9

[19] Wei Zhou, Lan Zhang, Le Guan, Peng Liu, Yuqing Zhang. What Your Firmware Tells You Is Not How You Should Emulate It: A Specification-Guided Approach for Firmware Emulation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. ACM, : 3269–3283

[20] Bo Feng, Alejandro Mera, Long Lu. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In: Proceedings of Usenix Security Symposium, 2020

[21] Chen Cao, Le Guan, Peng Cui, Limin Cheng, Mingyu Zhu. Laelaps: An Efficient and Fast Dynamic Analysis Tool for IoT Firmware. In: IEEE International Conference on Embedded Software and Systems, 2021

[22] Wei Zhou, et al. uEmu: A Framework for Firmware Emulation and Vulnerability Discovery. In: ACM CCS, 2021

[23] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, Giovanni Vigna. Firmalice: Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware. In: NDSS, 2015

[24] Evan Johnson, Maxwell Bland, YiFei Zhu, Joshua Mason, Stephen Checkoway, Stefan Savage, et al. Jetset: Targeted Firmware Rehosting for Embedded Systems. : 321–338

[25] Alejandro Mera, Bo Feng, Long Lu, Engin Kirda. DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. In: 2021 IEEE Symposium on Security and Privacy (SP). IEEE, : 1938–1954

[26] Wenqiang Li, Libo Li, Shenglin Zhang, Tao Xie, Lei Ma, et al. Para-rehosting: Automatic Payload Generation for Embedded Inter-Process Communication. In: USENIX Security Symposium, 2022

[27] Lukas Seidel, Dominik Maier, Marius Muench. Forming Faster Firmware Fuzzers. In: 32nd {USENIX} Security Symposium, {USENIX} Security 2023, Anaheim,CA, USA, August 9-11, 2023. {USENIX} Association, : 2903–2920

[28] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Guo, et al. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In: NDSS, 2018

[29] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, et al. Snipuzz: Black-box fuzzing of iot firmware via message snippet inference. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security, 2021: 337–350

[30] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, et al. Labrador: Response Guided Directed Fuzzing for Black-box IoT Devices. In: 2024 IEEE Symposium on Security and Privacy (SP). IEEE, : 1920–1938

[31] Tobias Scharnowski, Simon Wörner, Felix Buchmann, Nils Bars, Moritz Schloegel, Thorsten Holz. Hoedur: Embedded Firmware Fuzzing using Multi-Stream Inputs. 2023

[32] Wei Zheng, et al. FirmAFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation. In: USENIX Security, 2019

[33] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, et al. SFuzz: Slice-based Fuzzing for Real-Time Operating Systems. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. ACM, : 485–498

[34] Alejandro Mera, Changming Liu, Ruimin Sun, Engin Kirda, Long Lu. SHiFT: Semi-hosted Fuzz Testing for Embedded Applications. In: Davide Balzarotti, Wenyuan Xu, editors, 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association, : 5323–5340

[35] Nick Stephens, Jack Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, et al. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In: NDSS, 2016

[36] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, Taesoo Kim. QSYM: A Practical Concolic Execution Engine Tailored for Hybrid Fuzzing. In: USENIX Security Symposium, 2018

[37] Peng Chen, Hao Chen. Angora: Efficient Fuzzing by Principled Search. In: IEEE Symposium on Security and Privacy, 2018

[38] Jiameng Shi, Wenqiang Li, Wenwen Wang, Le Guan. Facilitating Non-Intrusive In-Vivo Firmware Testing with Stateless Instrumentation. In: Proceedings 2024 Network and Distributed System Security Symposium. Internet Society

[39] Anthropic. Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet, 2024

[40] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, et al. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities, 2025. https://arxiv.org/abs/2507.06261

[41] DeepSeek-AI, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature, 2025, 645: 633–638

[42] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, et al. GLM: General Language Model Pretraining with Autoregressive Blank Infilling. In: Smaranda Muresan, Preslav Nakov, Aline Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Dublin, Ireland: Association for Computational Linguistics, 2022: 320–335

[43] MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, et al. MiniMax-01: Scaling Foundation Models with Lightning Attention, 2025. https://arxiv.org/abs/2501.08313

[44] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, et al. Finetuned Language Models Are Zero-Shot Learners. arXiv preprint arXiv:2109.01652, 2022

[45] Weize Wang, Qipeng Chen, Zihao Zhang, Yang Li, Xin Cui, Jing Chen, et al. DolphCoder: Echo-Locating Code with Diverse and Multi-Objective Instruction Tuning. arXiv preprint, 2024

[46] Tianyu Zhang, et al. CodeLSI: Leveraging Foundation Models for Automated Code Generation with Low-Rank Optimization and Domain-Specific Instruction Tuning. arXiv preprint, 2024

[47] Xinyun Chen, Maxwell Lin, Dale Schuurmans, Quoc Le. Teaching Large Language Models to Self-Debug. arXiv preprint arXiv:2304.05128, 2023

[48] Kechi Liu, Shusheng Zhang, Ge Li, Zhi Jin, Li Zhao, Yang Li. Tuning Models of Code with Compiler-Generated Reinforcement Learning Feedback. arXiv preprint, 2024

[49] Yuxiang Zhang, et al. CodeDPO: Aligning Code Models with Self-Generated and Verified Source Code. arXiv preprint, 2024

[50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903, 2022

[51] Kechi Zhang, et al. MoTCoder: Elevating Large Language Models with Modular of Thought for Challenging Programming Tasks. arXiv preprint, 2024

[52] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv preprint arXiv:2302.02531, 2023

[53] Wei Chen, et al. MCCoder: Streamlining Motion Control with LLM-Assisted Code Generation and Rigorous Verification. arXiv preprint, 2024

[54] Zhi Li, et al. RGD: Multi-LLM Based Agent Debugger via Refinement and Generation Guidance. arXiv preprint, 2024

[55] Tianyu Wang, et al. Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification in Industrial Control Systems Using LLM-Based Agents. arXiv preprint, 2024

[56] RT-Thread Community. RT-Thread: An Open Source Embedded Real-Time Operating System. https://www.rt-thread.org/

[57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2022

[58] Y. Dong, X. Jiang, Z. Jin, G. Li. Self-Collaboration Code Generation via ChatGPT. ACM Transactions on Software Engineering and Methodology, 2024, 33(7): 1–38

[59] Nguyen, Anh Quynh, et al. Unicorn: The ultimate CPU emulator. https://www.unicorn-engine.org/

[60] Valentin J.M. Manes, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, et al. The Art, Science, and Engineering of Fuzzing: A Survey. 47(11): 2312–2331

[61] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, Marc Heuse. AFL++ : Combining Incremental Steps of Fuzzing Research. In: 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association, 2020